

# Gram–Schmidt Methods for Unsupervised Feature Extraction and Selection

Bahram Yaghoobi<sup>1</sup>, Graduate Student Member, IEEE, Netanel Raviv<sup>2</sup>, Senior Member, IEEE, and Bruno Sinopoli<sup>3</sup>, Fellow, IEEE

**Abstract**—Feature extraction and selection in the presence of nonlinear dependencies among the data is a fundamental challenge in unsupervised learning. We propose using a Gram–Schmidt (GS) type orthogonalization process over function spaces to detect and map out such dependencies. Specifically, by applying the GS process over some family of functions, we construct a series of covariance matrices that can either be used to identify new large-variance directions, or to remove those dependencies from known directions. In the former case, we provide information-theoretic guarantees in terms of entropy reduction. In the latter, we provide precise conditions by which the chosen function family eliminates existing redundancy in the data. Each approach provides both a feature extraction and a feature selection algorithm. Our feature extraction methods are linear, and can be seen as natural generalization of principal component analysis (PCA). We provide experimental results for synthetic and real-world benchmark datasets which show superior performance over state-of-the-art (linear) feature extraction and selection algorithms. Surprisingly, our linear feature extraction algorithms are comparable and often outperform several important nonlinear feature extraction methods such as autoencoders, kernel PCA, and UMAP. Furthermore, one of our feature selection algorithms strictly generalizes a recent Fourier-based feature selection mechanism (Heidari et al., IEEE Transactions on Information Theory, 2022), yet at significantly reduced complexity.

**Index Terms**—Feature extraction, feature selection, principal component analysis, Gram–Schmidt orthogonalization.

## I. INTRODUCTION

**D**IMENSIONALITY reduction is a set of techniques used to reduce the number of features (or dimensions) in a dataset, while retaining the important information. In the context of unsupervised learning, the goal is to simplify the data and make it easier to analyze or visualize, while reducing the computational complexity and memory requirements of any further processing. The two main approaches to

dimensionality reduction are *feature selection*, which involves selecting a subset of the original features based on some criteria, and *feature extraction*, which involves transforming the original features into a new set of features, preferably in a linear fashion, so that the extracted features capture the most important information [3].

In a sense, the ultimate goal of feature extraction is to identify and remove *redundancy*, that is, parts of the data which do not carry intrinsic information but rather can be determined by other parts of the data. It is well known that whenever such redundancy is linear, i.e., when certain features are linear functions of other ones, then PCA identifies and removes those redundant features. The paper at hand advances the state-of-the-art by presenting new linear feature extraction and selection techniques which are capable of removing *non-linear* redundancy from the data. We focus on *unsupervised* feature extraction, i.e., that does not take into account any label information.

The well-known PCA technique works by analyzing the covariance matrix of the data (or an empirical approximation thereof), and extracts new features called *principal components*; the principal components are uncorrelated and capture the variance in the original data in descending order. Each principal components is a projection of the data on an eigenvector of its covariance matrix, referred to as *principal direction*. In contrast, we propose a family of Gram–Schmidt based algorithms which rely on eigenvector analysis of *alternative* covariance matrices, introduced next.

Suppose our data is sampled i.i.d from some unknown random variable  $X = (X_1, \dots, X_d)^\top$  over  $\mathbb{R}^d$  with distribution  $P_X$ . Our algorithms begin by fixing a finite family  $\mathcal{F}(\mathbf{z})$  of linearly independent functions in some (non-random) variables  $\mathbf{z} = (z_1, z_2, \dots)$ , e.g., low-degree polynomials. In an iterative process, the variables  $z_i$  are substituted one-by-one with linear projections of  $X$  (i.e., of the form  $X^\top \mathbf{v}$  for some  $\mathbf{v} \in \mathbb{R}^d$ ), effectively turning those (ordinary) functions into random ones. Then, after applying the *Gram–Schmidt* (GS) process over the resulting random functions, (Section II-B), we subtract from  $X$  its projections on those functions, thereby creating new data distributions with less redundancy. The covariance matrices of these new data distributions are the *alternative covariance matrices* mentioned earlier. Note that this approach requires no particular assumption regarding  $\mathcal{F}(\mathbf{z})$  or  $P_X$ , other than the assumption that the functions in the GS process belong to  $\mathcal{L}^2(P_X)$ , i.e., have finite variance when computed according to  $P_X$  [4].

Received 21 August 2024; revised 10 July 2025; accepted 10 July 2025. Date of publication 17 July 2025; date of current version 17 September 2025. An earlier version of this paper was presented in part at the 2024 IEEE Conference on Decision and Control (CDC) [1]; and in part at the 2024 Annual Allerton Conference on Communication, Control, and Computing [2]. (Corresponding author: Netanel Raviv.)

Bahram Yaghoobi and Bruno Sinopoli are with the Department of Electrical and Systems Engineering, Washington University in St. Louis, St. Louis, MO 63130 USA (e-mail: byaghoobi@wustl.edu; bsinopoli@wustl.edu).

Netanel Raviv is with the Department of Computer Science and Engineering, Washington University in St. Louis, St. Louis, MO 63130 USA (e-mail: netanel.raviv@wustl.edu).

Communicated by R. Venkataramanan, Associate Editor for Machine Learning and Statistics, Communications, Signal Processing and Source Coding.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TIT.2025.3589174>.

Digital Object Identifier 10.1109/TIT.2025.3589174

0018-9448 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

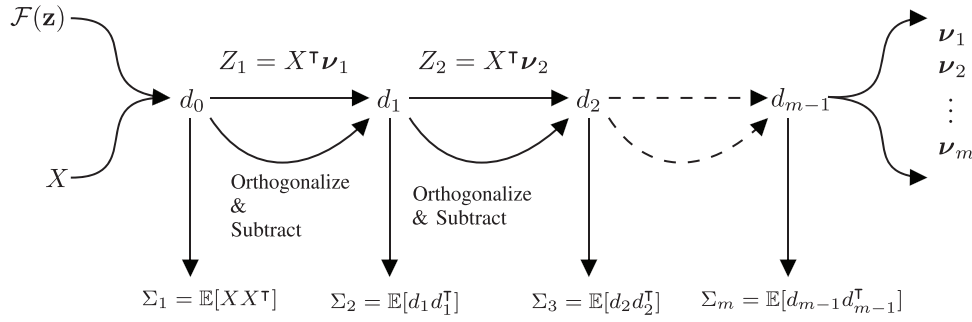


Fig. 1. A schematic description of all algorithms in this paper. The algorithm receives a function family  $\mathcal{F}(\mathbf{z})$  in variables  $\mathbf{z} = (z_1, z_2, \dots)$  and data sampled i.i.d from an unknown random variable  $X$ . The algorithm iteratively substitutes the (non-random) variables  $z_i$  in  $f(\mathbf{z}) \in \mathcal{F}(\mathbf{z})$  with random variables  $Z_i$ , which are linear functions of  $X$ , orthogonalizes the resulting  $\mathcal{L}^2(P_X)$  functions, and subtracts from  $X$  its projections on them. This creates new random variables  $d_j(X)$ , and the  $\Sigma_j$ 's are their covariance matrices. Different algorithms vary in the way the  $\nu_i$ 's are specified: either as the largest eigenvectors of the  $\Sigma_i$ 's (Gram-Schmidt Functional Reduction, GFR, Section III) or as high-variance principal directions (Gram-Schmidt Component Analysis, GCA, Section IV) in the feature extraction case, or their unit-vector counterparts in Gram-Schmidt Functional Selection (GFS, Section V) and Gram-Schmidt Feature Analysis (GFA, Section VI) in the feature selection case. The output of the algorithm is  $\nu_1, \dots, \nu_m$ .

In the context of feature extraction, eigen-analysis of these alternative covariance matrices enables to either define new high variance directions (Section III), or remove nonlinear redundancy from the principal components (Section IV). In the context of feature selection, similar analysis of the main diagonal of those matrices enables two different feature selection algorithms which either select high-variance features (Section V) or remove nonlinear redundancy from the given features (Section VI).

In more detail, the Gram-Schmidt process is a classic algebraic technique to convert a given basis of a subspace to an orthonormal basis of the same subspace, i.e., where all vectors are of length 1 and any pair of vectors are orthogonal. We employ a well-known generalization of GS to function spaces, where vectors are replaced by  $\mathcal{L}^2(P_X)$  functions, i.e., where the inner product between two functions is the expectation of their product, computed according to  $P_X$ .

The use of GS in our algorithms results in a new set of orthonormal  $\mathcal{L}^2(P_X)$  functions. The orthonormality of these functions enables to define new random variables  $d_j(X)$ —one at each iteration  $j$  of the algorithm—by subtracting from  $X$  its projections on those orthonormal functions. This amounts to “nonlinear redundancy removal,” in the sense that these  $d_j$ 's capture whatever remains in  $X$  should one set to zero the part of the redundancy which can be described by the orthonormal functions.

As mentioned earlier, those  $d_j$ 's give rise to alternative covariance matrices  $\Sigma_{j+1} = \mathbb{E}[d_j(X)d_j(X)^T]$ . Standard eigenvector analysis over  $\Sigma_{j+1}$  provides insights into choosing the next feature to be extracted or selected. The extraction/selection process stops once sufficiently many features have been extracted/selected, or a maximum variance threshold has been reached. This general framework, which is illustrated in Figure 1, is manifested in two complementing approaches that are described next.

In the first approach, we provide algorithms with entropy reduction guarantees that apply to any data distribution over a finite-alphabet. To this end, at step  $j$  we extract the largest

eigenvector<sup>1</sup> of  $\Sigma_j$  (in the feature extraction case), or select the most variant feature according to  $\Sigma_j$  (in the feature selection case). To prove these entropy reduction guarantees, we borrow ideas from a recently proposed Fourier-based feature selection method due to [5], and show that the conditional entropy  $H(X|Z)$ , where  $Z$  is the resulting extracted/selected random variable, is bounded by a function of the dimension and a threshold parameter. We term these algorithms *Gram-Schmidt Functional Reduction* (GFR, feature extraction), and *Gram-Schmidt Functional Selection* (GFS, feature selection).

In the second approach, we provide algorithms with clear guarantees of their ability to *eliminate redundancy*. Roughly speaking, a “redundancy” is as an equation that is satisfied, or approximately satisfied, by every datapoint drawn from  $X$ . Each such redundancy implies a functional connection between the features/components of the data, and by “eliminating redundancy” we mean that one of those features/components is not extracted/selected since it can be determined by other features/components. This paradigm should be preferred in cases where prior knowledge regarding the structure of the redundancy is available, even if not exact. In particular, knowing a priori that some linear mapping of the data (e.g., the principal components or the features themselves) has redundancy, and that the redundancy can be approximated by some function family, can provide strong guarantees on the ability to remove that redundancy.

To implement the second approach, we employ similar GS-type orthogonalization, but instead of choosing new directions/features according to the  $\Sigma_j$ 's as in the previous approach, we first subtract all low-variance directions/features from the data, and then choose variance maximizing directions/features. This subtraction process enables a clearer analysis of the effect the algorithms have on the redundancy structures in the data. We term these algorithms *Gram-Schmidt Functional Selection* (GFS, for eliminating redundancy from the principal components) and *Gram-Schmidt Feature Analysis* (GFA, for eliminating redundancy from the features themselves).

<sup>1</sup>I.e., the eigenvector of largest eigenvalue in absolute value.

Finally, the complexity of all algorithms is roughly quadratic in the size of the chosen function family (which can be determined by the user), linear in the size of the dataset, and at most cubic in the data dimension. We also comment that GFS strictly generalizes the recently proposed Unsupervised Fourier Feature Selection (UFFS) method [5], [6] at significantly reduced complexity, and the details are given in the appendix. We summarize all our contributions in the following subsection.

*Remark 1:* Roughly speaking, our approach falls under the broad category of “maximum variance pursuit.” Algorithms in this family, which includes PCA and kernel PCA, scan the feature space in search of projections which maximize some form of variance. Indeed, our GFR algorithm can be seen as a strict generalization of PCA, in the sense it specifies to PCA if one chooses  $\mathcal{F}(\mathbf{z}) = \{z_1, \dots, z_d\}$ . Kernel PCA, however, is a nonlinear method, whereas all our methods are linear. A comprehensive literature review, including a detailed comparison to PCA and its variants, as well as to other state-of-the-art linear and nonlinear methods, is given in Section VII. In Appendix E, a detailed explanation of the underlying mechanisms and procedural distinctions between GFR and kernel PCA is provided, with particular emphasis on their differences during both the training and inference phases.

#### A. Our Contributions

- We present *Gram-Schmidt Functional Reduction* (GFR), a **linear feature extraction** technique that identifies directions of high-variance (different from the principal ones). We provide information-theoretic guarantees of bounded conditional entropy for the case of discrete data distributions (Section III).
- We present *Gram-Schmidt Component Analysis* (GCA), a **linear feature extraction** technique that identifies and removes nonlinear dependencies among the principal components. We provide conditions on the chosen function family by which GCA can eliminate redundancy (Section IV).
- We present *Gram-Schmidt Functional Selection* (GFS), a **feature selection** technique which inherits similar structure and similar information-theoretic guarantees from GFR (Section V).
- We present *Gram-Schmidt Feature Analysis* (GFA), a **feature selection** technique which inherits similar formal guarantees from GCA (Section VI).
- We prove that GFS generalizes the Unsupervised Fourier Feature Selection (UFFS) algorithm due to [5], yet at significantly lower complexity. Specifically, UFFS arises as a special case of GFS when a certain function family is chosen (Appendix C).
- We support our theoretical findings and show the applicability of our techniques by performing broad experiments. The experiments validate our findings in several aspects:
  - GFR achieves similar residual variance<sup>2</sup> using significantly fewer extracted features in comparison to

<sup>2</sup>I.e., the maximum amount of variance that is left in the data distribution after extracting said features. This is captured in PCA by the  $(m+1)$ ’th largest eigenvalue of the covariance matrix when extracting  $m$  features.

PCA, by almost an order of magnitude in certain cases.

- GFR preserves classification accuracy in both synthetic and real-world datasets, in comparison to PCA and state-of-the-art linear methods. It also performs comparatively well or better than some powerful nonlinear methods such as autoencoders, kernel PCA, and Uniform Manifold Approximation and Projection (UMAP).
- GFS shows superior performance in classification accuracy on real-world and synthetic datasets in comparison to state-of-the-art feature selection algorithms, including UFFS [5].
- The theoretical guarantees of GCA and GFA are shown to hold over synthetic datasets.

The implementations of our algorithms are simple, and source code is available at [https://github.com/byaghooti/Gram\\_schmidt\\_feature\\_extraction](https://github.com/byaghooti/Gram_schmidt_feature_extraction).

## II. NOTATIONS AND PRELIMINARIES

### A. Notations

We denote vector random variables using capital letters  $X, Y, Z$ , etc., and in particular we let  $X = (X_1, \dots, X_d)$  be the (unknown, zero mean) random variable over  $\mathbb{R}^d$  (i.e., each  $X_i$  is over  $\mathbb{R}$ ) from which the data is sampled i.i.d. We use  $\mathbf{x} = (x_1, \dots, x_d)$ ,  $\mathbf{y} = (y_1, \dots, y_d)$ , etc. to denote ordinary (that is, non-random) variables, and  $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_d)$ ,  $\boldsymbol{\beta} = (\beta_1, \dots, \beta_d)$  etc. to denote vectors of scalars (i.e.,  $\alpha_i, \beta_i \in \mathbb{R}$ ). Further, for  $S \subseteq [d]$  (where  $[d] \triangleq \{1, 2, \dots, d\}$ ), we use  $X_S$  (resp.  $\mathbf{x}_S$ , etc.) to denote a vector of length  $|S|$  which contains the entries of  $X$  (resp.  $\mathbf{x}$ , etc.) that are indexed by  $S$ .

We denote by  $\mathcal{F}(\mathbf{z})$  a generic function family in the (ordinary) variables  $z_1, \dots, z_d$ , which are linearly independent over  $\mathbb{R}$ . For  $S \subseteq [d]$  we let  $\mathcal{F}(\mathbf{z}_S)$  be the subset of  $\mathcal{F}$  which contains all functions in  $\mathcal{F}$  which depend only on a subset of the variables in  $\mathbf{z}_S$ . Our Gram-Schmidt approach strongly relies on substituting the ordinary variables  $z_i$  by random ones  $Z_i$ , and hence for  $S = \{\sigma_1, \dots, \sigma_{|S|}\} \subseteq [d]$  (with  $\sigma_1 < \dots < \sigma_{|S|}$ ) and a vector of random variables  $Z = (Z_1, \dots, Z_d)$ , we denote by  $\mathcal{F}(Z_S)$  the result of substituting  $z_i \leftarrow Z_{\sigma_i}$  for every  $i \in [|S|]$  in  $\mathcal{F}(\mathbf{z}_{|S|})$ .

*Example 1:* Let  $d = 3$  and  $\mathcal{F}(\mathbf{z}) = \{z_1, z_2, z_3, z_1 z_2, z_1 z_2 z_3\}$ . For  $S = [2]$  we have that  $\mathcal{F}(\mathbf{z}_S) = \{z_1, z_2, z_1 z_2\}$ , and for  $\mathcal{T} = \{1, 3\}$  we have  $\mathcal{F}(Z_{\mathcal{T}}) = \{Z_1, Z_3, Z_1 Z_3\}$ .

All function families  $\mathcal{F}(\mathbf{z})$  we use in this paper contain all the “singleton” functions, i.e.,  $\{f_i(\mathbf{z}) = z_i\}_{i=1}^d \subseteq \mathcal{F}(\mathbf{z})$ . Our algorithms require no particular assumption on  $\mathcal{F}(\mathbf{z})$  or on  $P_X$  other than  $\mathcal{F}(Z_S) \subseteq \mathcal{L}^2(P_X)$  for every  $S \subseteq [d]$ , i.e., that the functions which result from the substitution have finite variance, and as such belongs to a Hilbert space in which the inner product between two functions  $f$  and  $g$  is  $\mathbb{E}[fg]$ . We note that all random variables in this paper are some functions of  $X$ , the random variable from which the data is sampled i.i.d, and all expectations are computed with respect to  $P_X$ .

*Remark 2:* The random variables we use in this paper (including  $X$ ) need not be known. We manipulate random variables as symbolic expressions, and only use them in the

context of computing expectations. In practice, these expectations are approximated using empirical means. For example, we may manipulate  $X$  to obtain a function  $\ell(X) = X_1 + X_2^2$ , without the need to know how  $X$  is distributed. To avoid notational clutter, expressions of the form  $\mathbb{E}[\ell(X)]$  are used in lieu of the empirical approximation  $\mathbb{E}[\ell(X)] \approx \sum_{i=1}^N \ell(\alpha_i) = \sum_{i=1}^N (\alpha_{i,1} + \alpha_{i,2}^2)$ , where  $\{\alpha_i\}_{i=1}^N$  is the dataset at hand.

As mentioned previously, our algorithms rely on the Gram-Schmidt process in a Hilbert space. To this end, we denote by  $\hat{\mathcal{F}}(Z_S)$  the result of applying the Gram-Schmidt process over a function family  $\mathcal{F}(Z_S)$ ; notice that  $\text{span } \hat{\mathcal{F}}(Z_S) = \text{span } \mathcal{F}(Z_S)$ . The full details of this Gram-Schmidt process are given in the following section. Finally, we use the algorithmic notation  $a \leftarrow b$  to denote that the expression  $b$  is computed, and then assigned into the variable  $a$ .

*Remark 3:* All our proposed algorithms can be implemented using purely algebraic operations. However, we chose to present them within a probabilistic framework in order to convey the core ideas more clearly, as well as to establish the theoretical guarantees more easily. Nevertheless, to support practical implementation, algebraic representations of all proposed algorithms, including orthogonalization, GFR, GCA, GFS, and GFA, are provided in Appendix D.

### B. Gram-Schmidt Orthogonalization Over Function Spaces

In this section we describe the core GS orthogonalization process in a function space on which all our algorithms are based. As mentioned earlier, we begin by fixing a function family  $\mathcal{F}(\mathbf{z})$  in variables  $\mathbf{z} = (z_1, \dots, z_d)$ ; other than being square integrable according to the data distribution we do not make any particular assumptions on those functions, but in our experimental sections we focus on various polynomial functions.

Our algorithms (Sections III–VI) apply in an iterative manner, where in iteration  $j$  we choose some new feature  $Z_{\ell_j}$  to select/extract, substitute  $z_j \leftarrow Z_{\ell_j}$  in the functions of  $\mathcal{F}(\mathbf{z}_{[j]})$ , and perform GS orthogonalization. To this end, Algorithm 1 operates at each iteration  $j$  of either of our algorithms by receiving an already orthogonalized  $\hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_{j-1}})$ , new functions in  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]})$ , and a new random variable  $Z_{\ell_j}$ .

Then, for each member of  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]})$ , the algorithm substitutes  $z_i \leftarrow Z_{\ell_i}$  for every  $i \in [j]$  (Line 6), subtracts its projection on already-orthogonalized functions (Line 7), normalizes (Line 8), and adds the result to  $\mathcal{T}$  (Line 9). After the subtraction process we define a new (vector) random variable  $d_j(X)$  from  $X$  (Line 12), whose covariance matrix is computed (Line 13) and returned as output. An equivalent explicit algebraic representation of this orthogonalization process, referred to as *Algebraic-Orthogonalize*, is provided in Algorithm 7 in Appendix D-A.

### III. GRAM-SCHMIDT FUNCTIONAL REDUCTION (GFR)

In Algorithm 2 below we propose *Gram-Schmidt Functional Reduction* (GFR), which uses the alternative covariance matrices  $\Sigma_j$  in order to extract new high variance directions. In each iteration  $j$ , GFR begins by identifying the largest eigenvector  $\mathbf{v}_j$  of  $\Sigma_j$  (this would be the highest variance principal direction of

**Algorithm 1** Orthogonalize( $\hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_{j-1}}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{Z_{\ell_i}\}_{i=1}^j$ )

---

```

1: Input:
   •  $\hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_{j-1}})$ : an orthogonalized function family in
     random variables  $Z_{\ell_1}, \dots, Z_{\ell_{j-1}}$ .
   •  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]})$ : all the functions in  $\mathcal{F}(\mathbf{z})$  which depend
     on  $\mathbf{z}_{[j]}$  but not only on  $\mathbf{z}_{[j-1]}$ .
   •  $\{Z_{\ell_i}\}_{i=1}^j$ : the random variables  $Z_{\ell_1}, \dots, Z_{\ell_{j-1}}$  which appear
     in  $\hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_{j-1}})$ , and a new random variable  $Z_{\ell_j}$ .
2: Output: Orthogonalized function family  $\hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_j})$ ,
   and a covariance matrix  $\Sigma_{j+1}$ .
3: Initialize:  $\mathcal{T} \leftarrow \hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_{j-1}})$ .
4: Denote  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}) \triangleq \{f_1(\mathbf{z}), \dots, f_\ell(\mathbf{z})\}$ , where
    $f_1(\mathbf{z}) = z_j$ .
5: for  $k \leftarrow 1$  to  $\ell$  do
6:    $g(Z_{\ell_1}, \dots, Z_{\ell_j}) \leftarrow f_k(Z_{\ell_1}, \dots, Z_{\ell_j})$  (create  $g$  from  $f_k$  by
     substituting  $z_i \leftarrow Z_{\ell_i}$  for  $i \in [j]$ .)
7:    $g(Z_{\ell_1}, \dots, Z_{\ell_j}) \leftarrow g(Z_{\ell_1}, \dots, Z_{\ell_j}) - \sum_{f \in \mathcal{T}} \mathbb{E}[gf]f$ 
     (orthogonalize  $g$  with respect to  $\mathcal{T}$ .)
8:    $g(Z_{\ell_1}, \dots, Z_{\ell_j}) \leftarrow \frac{g(Z_{\ell_1}, \dots, Z_{\ell_j})}{\sqrt{\mathbb{E}[g(Z_{\ell_1}, \dots, Z_{\ell_j})^2]}}$  (normalize  $g$ .)
9:    $\mathcal{T} \leftarrow \mathcal{T} \cup \{g(Z_{\ell_1}, \dots, Z_{\ell_j})\}$  (add  $g$  to  $\mathcal{T}$ )
10: end for
11: Define  $\hat{\mathcal{F}}(Z_{\ell_1}, \dots, Z_{\ell_j}) = \mathcal{T}$ .
12: Define  $d_j(X) = X - \sum_{f \in \mathcal{T}} \mathbb{E}[Xf]f$ .
13: Define  $\Sigma_{j+1} = \mathbb{E}[d_j(X)d_j^\top(X)]$ .
14: Return  $\Sigma_{j+1}, \mathcal{T}$ .
```

---

**Algorithm 2** Gram-Schmidt Functional Reduction (GFR)

---

```

1: Input: A function family  $\mathcal{F}(\mathbf{z})$  and a threshold  $\epsilon > 0$ .
2: Output: Extracted directions  $\mathbf{v}_1, \dots, \mathbf{v}_m$  ( $m$  is a varying
   number that depends on  $\mathcal{F}(\mathbf{z}), \epsilon$ , and  $X$ ).
3: Initialize:  $\Sigma_1 = \mathbb{E}[XX^\top]$ .
4: for  $j \leftarrow 1$  to  $d$  do
5:   Let  $\mathbf{v}_j$  be the largest unit-norm eigenvector of  $\Sigma_j$ , i.e.,
      $\mathbf{v}_j = \arg \max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \Sigma_j \mathbf{v}$ .
6:   if  $\mathbf{v}_j^\top \Sigma_j \mathbf{v}_j \leq \epsilon^2$  then
7:     break.
8:   else
9:     Define  $Z_j = X^\top \mathbf{v}_j$ .
10:  end if
11:   $\Sigma_{j+1}, \hat{\mathcal{F}}(Z_1, \dots, Z_j) = \text{Orthogonalize}(\hat{\mathcal{F}}(Z_1, \dots, Z_{j-1}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{Z_{\ell_i}\}_{i=1}^j)$ .
12: end for
```

---

$d_{j-1}(X)$ , computed in Line 12 of Algorithm 1). If the variance of this direction is less than  $\epsilon^2$ , the algorithm stops, and otherwise it continues by specifying the next random variable  $Z_j$  as  $Z_j = X^\top \mathbf{v}_j$ , which enables the next call to Orthogonalize (Algorithm 1). An equivalent and fully algebraic version of this algorithm, referred to as *Algebraic-GFR*, is provided in Algorithm 8 in Appendix D-B.

Next, we state the information-theoretic guarantees of GFR. While GFR shows significant gains over state-of-the-art linear (and some nonlinear) methods with real-world datasets, its information-theoretic guarantees require the data distribution  $X$



to be over a finite alphabet. In our experiments (Section VIII) it is shown that setting  $\mathcal{F}(\mathbf{z})$  to be multilinear polynomials of low degree reduces the residual variance in real-world datasets by up to an order of magnitude in comparison with PCA.

**Theorem 1:** Suppose  $X$  is over a discrete domain  $\mathcal{X}^d$ , for some  $\mathcal{X} \subseteq \mathbb{R}$ , and let  $Z = (Z_1, \dots, Z_m) = (X^\top \mathbf{v}_1, \dots, X^\top \mathbf{v}_m)$ , where  $\mathbf{v}_1, \dots, \mathbf{v}_m$  is the output of GFR whose input is a given  $\epsilon$  and  $\mathcal{F}(\mathbf{z})$ . Then  $H(X|Z) \leq dO(\epsilon)$ .

The proof of Theorem 1 is similar to a proof from [5], but requires several additional statements. These statements essentially show that once GFR stops, the variance of the data in *all potential subsequent directions* is smaller than  $\epsilon^2$ . As a result, a tradeoff between  $\mathcal{F}(\mathbf{z})$  and  $\epsilon$  is revealed—taking a more powerful  $\mathcal{F}(\mathbf{z})$  (e.g., higher degree polynomials) or a smaller  $\epsilon$  would decrease the number  $m$  of extracted features. To prove this statement, we complete the vectors  $\mathbf{v}_1, \dots, \mathbf{v}_m$  with  $\mathbf{v}_{m+1}, \dots, \mathbf{v}_d$  as follows.

**Definition 1:** Let  $\mathbf{v}_1, \dots, \mathbf{v}_m$  be the output of GFR with input  $\epsilon > 0$  and  $\mathcal{F}(\mathbf{z})$ . Let  $\mathbf{v}_1, \dots, \mathbf{v}_{m'}$  be the output of GFR with the same  $\mathcal{F}(\mathbf{z})$  and  $\epsilon = 0$ , and let  $\Sigma_1, \dots, \Sigma_{m'}$  be the respective covariance matrices. In cases where  $m' < d$  we let  $\mathbf{v}_{m'+1}, \dots, \mathbf{v}_d$  be an arbitrary orthonormal completion of  $\mathbf{v}_1, \dots, \mathbf{v}_{m'}$ . In these cases we also artificially complete the orthogonalization process to produce  $\hat{\mathcal{F}}(Z_1, \dots, Z_d)$ , with  $Z_j = X^\top \mathbf{v}_j$  for all  $j \in [d]$ , and let  $\{\Sigma_j\}_{j=1}^d$  be the respective covariance matrices. Finally, for  $j \in [d]$  we denote  $\lambda_j = \mathbf{v}_j^\top \Sigma_j \mathbf{v}_j$ .

**Lemma 1:** For every  $j \in \{m+1, \dots, d\}$  we have that  $\mathbf{v}_j^\top \Sigma_j \mathbf{v}_j \leq \epsilon^2$ .

The following technical lemma is required for the subsequent proof of Lemma 1. For brevity, in the subsequent lemmas for every  $j \in [d]$  we denote  $\hat{\mathcal{F}}_j \triangleq \hat{\mathcal{F}}(Z_1, \dots, Z_j)$ .

**Lemma 2:** In GFR, we have the following.

- (a) For  $i < j$  in  $[d]$  we have  $\Sigma_j = \Sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX] \mathbb{E}[X^\top f]$ .
- (b) The vectors  $\mathbf{v}_1, \dots, \mathbf{v}_d$  are orthonormal.

**Proof of Lemma 2(a):** It is readily verified that  $d_{j-1}(X)$  (see Algorithm 1) can be written as

$$d_{j-1}(X) = d_{i-1}(X) - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f, \quad (1)$$

and that  $d_{i-1}(X)$  can be written as

$$d_{i-1}(X) = X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f. \quad (2)$$

Hence, it follows from (1) that

$$\begin{aligned} \Sigma_j &= \mathbb{E}[d_{j-1}(X)d_{j-1}(X)^\top] \\ &= \mathbb{E}\left[\left(d_{i-1}(X) - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f\right) \left(d_{i-1}(X) - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f\right)^\top\right] \\ &= \Sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fd_{i-1}(X)]\mathbb{E}[X^\top f] \\ &\quad - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX]\mathbb{E}[d_{i-1}(X)^\top f] \\ &\quad + \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]\mathbb{E}[X^\top f], \end{aligned} \quad (3)$$

where the last summand follows from the orthonormality of  $\hat{\mathcal{F}}_{j-1}$ . Further, it follows from (2) that every  $f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}$  satisfies

$$\begin{aligned} \mathbb{E}[fd_{i-1}(X)] &= \mathbb{E}\left[f \cdot \left(X - \sum_{g \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xg]g\right)\right] = \mathbb{E}[fX], \\ \mathbb{E}[d_{i-1}(X)^\top f] &= \mathbb{E}\left[\left(X - \sum_{g \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xg]g\right)^\top \cdot f\right] = \mathbb{E}[X^\top f], \end{aligned}$$

and therefore (3) =  $\Sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]\mathbb{E}[X^\top f]$  as required.  $\square$

**Proof of Lemma 2(b):** Since all  $\mathbf{v}_i$ 's are normalized it suffices to show that for any  $j \in [d]$ , the vector  $\mathbf{v}_j$  is orthogonal to  $\mathbf{v}_1, \dots, \mathbf{v}_{j-1}$ . If  $j > m'$ , however, the latter statement follows from Definition 1, since  $\mathbf{v}_{m'+1}, \dots, \mathbf{v}_d$  is an orthogonal completion of  $\mathbf{v}_1, \dots, \mathbf{v}_{m'}$ . If  $j \leq m'$  let  $i < j$ , and observe that  $\mathbf{v}_i^\top \Sigma_j \mathbf{v}_j = \lambda_j \mathbf{v}_i^\top \mathbf{v}_j$ . It follows from part (a) that

$$\Sigma_j = \Sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX]\mathbb{E}[X^\top f],$$

and hence

$$\begin{aligned} \mathbf{v}_i^\top \Sigma_j \mathbf{v}_j &= \mathbf{v}_i^\top \left( \Sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX]\mathbb{E}[X^\top f] \right) \mathbf{v}_j \\ &= \lambda_i \mathbf{v}_i^\top \mathbf{v}_j - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[f \cdot \mathbf{v}_i^\top X] \mathbb{E}[X^\top \mathbf{v}_j \cdot f]. \end{aligned} \quad (4)$$

Let  $\tilde{Z}_i$  and  $\hat{Z}_i$  be the orthogonalized and orthonormalized versions of  $Z_i$ , respectively. By definition, we have that  $\mathbf{v}_i^\top X = Z_i = \tilde{Z}_i + \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f]f$ , and hence every  $f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}$  satisfies

$$\begin{aligned} \mathbb{E}[f \cdot \mathbf{v}_i^\top X] &= \mathbb{E}\left[f \cdot \left(\tilde{Z}_i + \sum_{g \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i g]g\right)\right] \\ &= \mathbb{E}[f\tilde{Z}_i] = \begin{cases} \|\tilde{Z}_i\| & \text{if } f = \hat{Z}_i \\ 0 & \text{otherwise.} \end{cases} \end{aligned} \quad (5)$$

Therefore,

$$\begin{aligned} (4) &= \lambda_i \mathbf{v}_i^\top \mathbf{v}_j - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[f\tilde{Z}_i] \mathbb{E}[X^\top \mathbf{v}_j \cdot f] \\ &= \lambda_i \mathbf{v}_i^\top \mathbf{v}_j - \|\tilde{Z}_i\| \mathbb{E}[X^\top \mathbf{v}_j \cdot \hat{Z}_i] \\ &= (\lambda_i \mathbf{v}_i^\top - \mathbb{E}[X^\top \tilde{Z}_i]) \mathbf{v}_j. \end{aligned} \quad (6)$$

Now observe that

$$\begin{aligned} \mathbb{E}[X\tilde{Z}_i] &= \mathbb{E}\left[X \left(Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f]f\right)\right] \\ &= \mathbb{E}\left[X \left(\mathbf{v}_i^\top X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[\mathbf{v}_i^\top X f]f\right)\right] \\ &= \mathbb{E}[(XX^\top)\mathbf{v}_i] - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]\mathbb{E}[X^\top \mathbf{v}_i f] \\ &= \left(\mathbb{E}[XX^\top] - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX]\mathbb{E}[X^\top f]\right) \mathbf{v}_i \\ &\stackrel{(*)}{=} \Sigma_i \mathbf{v}_i = \lambda_i \mathbf{v}_i, \end{aligned}$$

where  $(*)$  follows by applying part (a) on  $\Sigma_i$  and  $\Sigma_1$ , and hence (6)=0. Subsequently, we have  $\lambda_j \mathbf{v}_i^\top \mathbf{v}_j = \mathbf{v}_i^\top \Sigma_j \mathbf{v}_j = 0$ . Since  $\lambda_j \neq 0$  by Definition 1, it follows that  $\mathbf{v}_j$  is orthogonal to  $\mathbf{v}_i$  for all  $i \in [j-1]$ , and the claim follows.  $\square$

**Proof of Lemma 1:** According to the stopping criterion in GFR, it follows that

$$\begin{aligned} \mathbf{v}_j^\top \Sigma_j \mathbf{v}_j &> \epsilon^2 \text{ for all } j \in [m] \\ \max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \Sigma_{m+1} \mathbf{v} &\leq \epsilon^2. \end{aligned} \quad (7)$$

Assume for contradiction that there exists  $j \in \{m+1, \dots, d\}$  such that  $\mathbf{v}_j^\top \Sigma_j \mathbf{v}_j > \epsilon^2$ , and observe that  $j > m+1$ , since otherwise the existence of  $\mathbf{v}_{m+1}$  contradicts (7). It follows from Lemma 2.(a) that

$$\Sigma_j = \Sigma_{m+1} - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[fX] \mathbb{E}[X^\top f],$$

and therefore

$$\begin{aligned} \mathbf{v}_j^\top \Sigma_j \mathbf{v}_j &= \mathbf{v}_j^\top \Sigma_{m+1} \mathbf{v}_j - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[f \cdot \mathbf{v}_j^\top X]^2 \\ &\stackrel{(7)}{\leq} \epsilon^2 - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[f \cdot \mathbf{v}_j^\top X]^2. \end{aligned}$$

Therefore, since  $\mathbf{v}_j^\top \Sigma_j \mathbf{v}_j > \epsilon^2$ , it follows that

$$- \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[f \cdot \mathbf{v}_j^\top X]^2 > 0,$$

which is a contradiction since  $\mathbb{E}[f \cdot \mathbf{v}_j^\top X]^2 \geq 0$  for every  $f$ .  $\square$

*Proof of Theorem 1:* Since the matrix whose rows are the  $\mathbf{v}_i$ 's has determinant 1, we have:

$$\begin{aligned} H(X|Z) &= H(\{X^\top \mathbf{v}_j\}_{j=1}^d | \{X^\top \mathbf{v}_j\}_{j=1}^m) \\ &= H(\{X^\top \mathbf{v}_j\}_{j=m+1}^d | \{X^\top \mathbf{v}_j\}_{j=1}^m) \\ &= H(Z^\perp | Z) = \sum_{i=m+1}^d H(Z_i | (Z_j)_{j=1}^{i-1}), \end{aligned} \quad (8)$$

where the last equality follows from the chain rule for information entropy. Since

$$\tilde{Z}_i = Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f, \quad (9)$$

it follows that

$$\begin{aligned} H(Z_i | (Z_j)_{j=1}^{i-1}) &= H\left(\tilde{Z}_i + \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f | (Z_j)_{j=1}^{i-1}\right) \\ &\stackrel{(a)}{=} H(\tilde{Z}_i | (Z_j)_{j=1}^i) \stackrel{(b)}{\leq} H(\tilde{Z}_i), \end{aligned} \quad (10)$$

where (a) follows since the expression  $\sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f$  is uniquely determined by the variables  $Z_1, \dots, Z_{i-1}$  (every  $f \in \hat{\mathcal{F}}_{i-1}$  is a deterministic function of  $Z_1, \dots, Z_{i-1}$ , and the coefficients  $\mathbb{E}[Z_i f]$  are constants), and (b) follows since conditioning reduces entropy. Combining (8) with (10) we have that  $H(X|Z) \leq \sum_{i=m+1}^d H(\tilde{Z}_i)$ , and hence it remains to bound  $H(\tilde{Z}_i)$  for all  $i \in \{m+1, \dots, d\}$ .

To this end let  $i \in \{m+1, \dots, d\}$ , define<sup>3</sup>  $\alpha_i \triangleq \min\{|\tilde{Z}_i(\alpha)| : \alpha \in \mathcal{X}^d, \tilde{Z}_i(\alpha) \neq 0\}$  and  $\alpha_{\min} \triangleq \min\{\alpha_i\}_{i=m+1}^d$ , and observe that by Markov's inequality we have

$$\begin{aligned} \Pr(\tilde{Z}_i \neq 0) &= \Pr(|\tilde{Z}_i| \geq \alpha_i) \\ &= \Pr(\tilde{Z}_i^2 \geq \alpha_i^2) \leq \frac{\mathbb{E}[\tilde{Z}_i^2]}{\alpha_i^2} \leq \frac{\mathbb{E}[\tilde{Z}_i^2]}{\alpha_{\min}^2}. \end{aligned} \quad (11)$$

Moreover, recall that

$$\begin{aligned} d_{i-1}(X) &= X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf] f, \text{ and} \\ \Sigma_i &= \mathbb{E}[d_{i-1}(X) d_{i-1}(X)^\top], \end{aligned}$$

<sup>3</sup>To clarify the notation  $\tilde{Z}_i(\alpha)$  for  $\alpha \in \mathcal{X}^d$ , recall that every  $Z_i$  is a function of  $X$ , and therefore so is its orthogonalized version  $\tilde{Z}_i = Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f$ . Therefore, the notation  $\tilde{Z}_i(\alpha)$  stands for the scalar given by the substitution  $X \leftarrow \alpha$  in  $\tilde{Z}_i$ .

and therefore

$$\begin{aligned} \mathbf{v}_i^\top \Sigma_i \mathbf{v}_i &= \mathbf{v}_i^\top \mathbb{E}[d_{i-1}(X) d_{i-1}(X)^\top] \mathbf{v}_i \\ &= \mathbf{v}_i^\top \mathbb{E}\left[\left(X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf] f\right) \left(X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf] f\right)^\top\right] \mathbf{v}_i \\ &= \mathbb{E}\left[\left(\mathbf{v}_i^\top X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[\mathbf{v}_i^\top X f] f\right) \left(X^\top \mathbf{v}_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[X^\top \mathbf{v}_i f] f\right)\right] \\ &= \mathbb{E}\left[\left(Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f\right) \left(Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f\right)\right] \\ &\stackrel{(9)}{=} \mathbb{E}[\tilde{Z}_i^2]. \end{aligned}$$

Consequently, it follows that

$$(11) = \frac{\mathbf{v}_i^\top \Sigma_i \mathbf{v}_i}{\alpha_{\min}^2} \stackrel{\text{Lemma 1}}{\leq} \frac{\epsilon^2}{\alpha_{\min}^2}.$$

Now, by using the grouping rule [7, Ex. 2.27] we have

$$H(\tilde{Z}_i) \leq h_b(\epsilon^2/\alpha_{\min}^2) + \frac{\epsilon^2}{\alpha_{\min}^2} \log |\mathcal{X}|,$$

where  $h_b$  is the binary entropy function, defined as  $h_b(p) = -p \log(p) - (1-p) \log(1-p)$ . Finally, using the bound  $h_b(p) \leq 2\sqrt{p(1-p)} \leq 2\sqrt{p}$ , it follows that

$$\begin{aligned} H(X|Z) &\leq \sum_{i=m+1}^d H(\tilde{Z}_i) \\ &\leq (d-m)(h_b(\epsilon^2/\alpha_{\min}^2) + (\epsilon^2/\alpha_{\min}^2) \log |\mathcal{X}|) \\ &\leq (d-m)(2\epsilon/\alpha_{\min} + (\epsilon/\alpha_{\min}) \log |\mathcal{X}|) \\ &= dO(\epsilon). \end{aligned}$$

$\square$

#### IV. GRAM-SCHMIDT COMPONENT ANALYSIS (GCA)

In this section we present *Gram-Schmidt Component Analysis* (GCA) in Algorithm 3, which demonstrates how orthogonalization techniques similar to those used in GFR (Section III) can eliminate redundancy among the principal components. Specifically, it is shown that if the principal components (i.e., the projections of the data on its principal directions) satisfy some nonlinear equations, and if these equations are well approximated by the function family  $\mathcal{F}$  in a sense that will be made formal shortly, then GCA identifies that redundancy, i.e., it eliminates the dependent components among the principal ones.

Recall that  $X = (X_1, \dots, X_d)$  is the random variable from which the data is sampled, and let  $Z = (Z_1, \dots, Z_d)$  be the principal components of  $X$  in decreasing order of variance. That is,  $Z_i = X^\top \rho_i$  where  $\rho_i$  is the  $i$ 'th largest principal direction of  $X$ , and hence  $\text{Var}(Z_1) \geq \text{Var}(Z_2) \geq \dots \geq \text{Var}(Z_d)$ <sup>4</sup>. We comment that GCA can be applied using any orthonormal basis (other than the principal directions), and yet we focus on the principal

<sup>4</sup>The  $\rho_1, \rho_2, \dots$  notations in this section, which denote the principal directions (output of PCA), are not to be confused with  $\mathbf{v}_1, \mathbf{v}_2, \dots$  in Section III, which denote the outputs of GFR.

**Algorithm 3** Gram-Schmidt Component Analysis (GCA)

---

```

1: Input: A function family  $\mathcal{F}(\mathbf{z})$  and a threshold  $\epsilon > 0$ .
2: Output: A set  $\mathcal{L} \subseteq [d]$  of indices of extracted principal
   directions.
3: Initialize:  $\Sigma_1 = \mathbb{E}[XX^\top]$  and  $\mathcal{L}_0 = \emptyset$ .
4: for  $j \leftarrow 1$  to  $d$  do
5:   Let  $\mathcal{E}_j = \{i | \rho_i^\top \Sigma_j \rho_i < \epsilon\}$ .
6:   if  $\mathcal{E}_j = [d]$  then
7:     return  $\mathcal{L}_{j-1}$ .
8:   end if
9:   Define  $\tilde{X}_j = X - \sum_{i \in \mathcal{E}_j} X^\top \rho_i \cdot \rho_i$ .
10:  Let  $\ell_j = \arg \max_{i \in [d] \setminus \mathcal{E}_j} \rho_i^\top \mathbb{E}[\tilde{X}_j \tilde{X}_j^\top] \rho_i$  (breaking ties
   arbitrarily), and  $\mathcal{L}_j = \mathcal{L}_{j-1} \cup \{\ell_j\}$ .
11:  Define  $Z_{\ell_j} = \rho_{\ell_j}^\top X$ .
12:   $\Sigma_{j+1}, \hat{\mathcal{F}}(Z_{\mathcal{L}_j}) = \text{Orthogonalize}(\hat{\mathcal{F}}(Z_{\mathcal{L}_{j-1}}), \mathcal{F}(\mathbf{z}_{[\ell_j]}) \setminus$ 
    $\mathcal{F}(\mathbf{z}_{[\ell_{j-1}]})$ ,  $\{Z_{\ell_i}\}_{i=1}^j$ ).
13: end for
14: return  $\mathcal{L}_d$ .

```

---

components for their prominent role in data analysis. Applying GCA for the special case of the standard basis results in a (slightly simplified) feature selection algorithm, and the full details are given in Section VI. We now turn to formalize the notion of redundancy.

**Definition 2:** For a function  $R : \mathbb{R}^d \rightarrow \mathbb{R}$  and  $\epsilon > 0$ , we say that  $X$  contains  $\epsilon$ -redundancy  $R$  among its principal components if  $\mathbb{E}[R(Z)^2] \leq \epsilon$ .

Intuitively, data which contains  $\epsilon$ -redundancy  $R$  lies close to the set of zeros of  $R(\mathbf{z})$  in some portion of the space. Roughly speaking, each  $\epsilon$ -redundancy implies that some variable  $z_j$  which participates in  $R$  is *redundant*, as it can be “isolated” from  $R(\mathbf{z}) = 0$  in the form  $z_j = r(\mathbf{z}^{-j})$ , where  $\mathbf{z}^{-j} \triangleq (z_1, \dots, z_{j-1}, z_{j+1}, \dots, z_d)$ , for some function  $r$ . This separation of  $z_j$  requires some additional conditions that will be discussed shortly.

The purpose of GCA is to eliminate such redundancies, i.e., for each  $\epsilon$ -redundancy  $R$  we wish to skip the component which is made redundant by it, and output to the user only the non-redundant components. GCA can be seen as an additional redundancy removal algorithm that can be applied on top of PCA for further dimension reduction; PCA identifies if the data lies in a subspace, and GCA identifies if further (nonlinear) dependencies exist among the principal components. In Section VI, we apply similar principals to feature selection.

**Example 2:** Suppose the data has a 0-redundancy  $R(z_1, z_2, z_3) = z_3 - z_1 z_2$ , meaning that  $R(Z) = Z_3 - Z_1 Z_2 = 0$ . It is clear that extracting  $Z_1$  and  $Z_2$  suffices in order to determine  $Z_3$ , and hence extracting  $Z_1, Z_2$  and skipping  $Z_3$  can be understood as “eliminating” the redundancy  $R$ .

Gram-Schmidt Component Analysis (GCA) is given in Algorithm 3. In iteration  $j$ , it begins by identifying the principal components  $\{\rho_i\}_{i \in \mathcal{E}_j}$  in which the reduced data distribution  $d_j(X)$  (see Algorithm 1) has less than  $\epsilon$  variance, and terminates if  $\mathcal{E}_j = [d]$ . The set  $\mathcal{E}_j$  contains the indices of principal components that were either selected in previous iterations of GCA, or can be described as functions from  $\mathcal{F}(\mathbf{z})$  of the

selected principal components. In Line 9 GCA subtracts the components in  $\mathcal{E}_j$  from  $X$  to define  $\tilde{X}_j$ , and the most variant component of  $\tilde{X}_j$  is selected (the selected components are gathered in a set  $\mathcal{L}_j$ , which is later returned as output). Finally, a call to Orthogonalize (Algorithm 1) is made, in order to construct an orthogonal basis  $\hat{\mathcal{F}}(Z_{\mathcal{L}_j})$  to the function family  $\text{span}(\mathcal{F}(Z_{\mathcal{L}_j}))$ , and in order to obtain the covariance matrix  $\Sigma_{j+1}$  of  $d_j(X)$ . The corresponding equivalent algebraic implementation, denoted *Algebraic-GCA*, is provided in Algorithm 9 in Appendix D-B.

We now turn to formally describe and explain the ability of GCA to eliminate redundancy in the data. For some  $\epsilon > 0$ , suppose  $X$  contains the  $\epsilon$ -redundancies  $R^{(1)}, \dots, R^{(t)}$  among its principal components (i.e.,  $\mathbb{E}[R^{(1)}(Z)^2] \leq \epsilon, \dots, \mathbb{E}[R^{(t)}(Z)^2] \leq \epsilon$ ), and for every  $i \in [t]$  let  $Z_{j_i}$  be the least variant component among those participating in  $R^{(i)}$ . We consider the components  $\{Z_{j_i}\}_{i=1}^t$  as *redundant*, and show that they can be eliminated by GCA if the function family  $\mathcal{F}(\mathbf{z})$  is in some sense sufficiently descriptive. We make the following simplifying assumptions.

**Assumption 1:** The integers  $j_1, \dots, j_t$  are distinct, and there is no “inter-dependence” among  $Z_{j_1}, \dots, Z_{j_t}$ . That is, for every  $i$  the variable  $z_{j_i}$  does not appear in  $R^{(j)}(\mathbf{z})$  for every  $j \neq i$ . A justification for this assumption using the well-known implicit-function theorem is given in the sequel.

We also assume without loss of generality that  $j_1 < \dots < j_t$ , and denote the redundant index-set by  $\mathcal{R}^\epsilon \triangleq \{j_1, \dots, j_t\}$  and the non-redundant one by  $\mathcal{N}^\epsilon \triangleq [d] \setminus \mathcal{R}^\epsilon$ . Further, we denote  $\mathcal{N}_i^\epsilon = [j_i - 1] \setminus \{j_\ell\}_{\ell=1}^{i-1}$  for all  $i \in [t]$ , i.e.,  $\mathcal{N}_i^\epsilon$  is the index-set of non-redundant components up to  $j_i - 1$ . Recall that for a set  $\mathcal{S} = \{\sigma_1, \dots, \sigma_{|\mathcal{S}|}\} \subseteq [d]$ , with  $\sigma_1 < \dots < \sigma_{|\mathcal{S}|}$ , we denote by  $\mathcal{F}(Z_{\mathcal{S}})$  the result of taking  $\mathcal{F}(z_1, \dots, z_{|\mathcal{S}|})$  and substituting  $z_i \leftarrow Z_{\sigma_i}$  for every  $f \in \mathcal{F}(z_1, \dots, z_{|\mathcal{S}|})$  and every  $i \in [|\mathcal{S}|]$ . Also recall that  $\hat{\mathcal{F}}(Z_{\mathcal{S}})$  is the result of applying the Gram-Schmidt process over  $\mathcal{F}(Z_{\mathcal{S}})$ , and let  $\text{proj}_{\mathcal{F}(Z_{\mathcal{S}})} Z_j$  be the projection of  $Z_j$  on  $\text{span} \mathcal{F}(Z_{\mathcal{S}})$ , given by  $\sum_{f \in \hat{\mathcal{F}}(Z_{\mathcal{S}})} \mathbb{E}[Z_j f] f$ .

Using these notations, we proceed to define what it means for a function family  $\mathcal{F}(\mathbf{z})$  to be sufficiently descriptive in order for GCA to eliminate said redundancies. For a given  $\epsilon > 0$  we say that the  $\epsilon$ -redundancy  $R^{(i)}$  is  $\epsilon$ -captured by  $\mathcal{F}(\mathbf{z})$  if  $\mathbb{E}[Z_{j_i} - \text{proj}_{\mathcal{F}(Z_{\mathcal{N}_i^\epsilon})} Z_{j_i}]^2 \leq \epsilon$ . In simple terms,  $R^{(i)}$  is captured by  $\mathcal{F}(\mathbf{z})$  if the least variant component of  $R^{(i)}$  can be “well approximated” by the closest function to  $Z_{j_i}$  in  $\mathcal{F}(\mathbf{z})$  (this function take the remaining components in  $R^{(i)}$  as inputs). Both the closeness and the approximation are measured by the expected  $\ell_2$ -deviation.

**Theorem 2:** Under Assumption 1, let  $R^{(1)}, \dots, R^{(t)}$  be the  $\epsilon$ -redundancies of  $X$ . If all  $R^{(i)}$ ’s are  $\epsilon$ -captured by  $\mathcal{F}(\mathbf{z})$ , then Gram-Schmidt Component Analysis (Algorithm 3) outputs the non-redundant directions  $\{\rho_i\}_{i \in \mathcal{N}^\epsilon}$ .

**Proof:** We ought to show that  $\mathcal{L} = \mathcal{N}^\epsilon$ , i.e., that every element in  $\mathcal{N}^\epsilon$  is chosen to  $\mathcal{L}_j$  at some step  $j$  of GCA, and no element in  $\mathcal{R}^\epsilon$  is chosen to any such  $\mathcal{L}_j$ . We begin by observing that since  $X = \sum_{i=1}^d \rho_i Z_i$ , it follows from the definition of Orthogonalize (Algorithm 1) that for every  $j \in [d]$ ,

$$\begin{aligned} d_j(X) &= X - \sum_{f \in \hat{\mathcal{F}}(Z_{\mathcal{L}_{j-1}})} \mathbb{E}[X f] f \\ &= \sum_{i=1}^d \rho_i \left( Z_i - \sum_{f \in \hat{\mathcal{F}}(Z_{\mathcal{L}_{j-1}})} \mathbb{E}[Z_i f] f \right) \end{aligned}$$

$$= \sum_{i=1}^d \rho_i (Z_i - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})} Z_i).$$

Since  $\rho_i$  represent the principal directions, they are orthogonal. Therefore, for every  $i, j \in [d]$ , we have:

$$\begin{aligned} \rho_i^\top \Sigma \rho_i &= \rho_i^\top \mathbb{E}[d_j(X) d_j(X)^\top] \rho_i \\ &= \mathbb{E}[(Z_i - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})} Z_i)^2]. \end{aligned} \quad (12)$$

Therefore, in each iteration  $j$  the set  $\mathcal{E}_j$  contains all indices  $i$  such that  $i \in \mathcal{L}_{j-1}$  (since for those indices we have  $\mathbb{E}[(Z_i - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})} Z_i)^2] = 0$ ). In addition,  $\mathcal{E}_j$  also contains all indices  $i'$  such that  $i' \notin \mathcal{L}_{j-1}$  but  $\mathbb{E}[(Z_{i'} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})} Z_{i'})^2] \leq \epsilon$ , i.e.,  $Z_{i'}$  can be well approximated by its own projection on  $\text{span } \mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})$ .

To prove that  $\mathcal{L} = \mathcal{N}^\epsilon$ , we prove by induction over  $i \in [t]$  that every  $j \in [j_i]$  is assigned appropriately by GCA, meaning,  $j$  is assigned to  $\mathcal{L}$  if and only if it is in  $\mathcal{N}^\epsilon$ .

*Base Case:* We begin by showing that the first redundant component  $Z_{j_1}$  is not chosen as non-redundant, i.e., that  $j_1 \notin \mathcal{L}$ . Suppose for contradiction that  $j_1 \in \mathcal{L}$ , and let  $j \in [d]$  be the smallest integer such that  $j_1 \in \mathcal{L}_j$ , i.e.,  $j$  is the step in which  $Z_{j_1}$  is selected. It follows from Line 10 that  $Z_{j_1}$  is the most variant component of  $\bar{X}_j$ . In addition, it also follows from Line 9, that  $j_1 \notin \mathcal{E}_j$ , since otherwise  $Z_{j_1}$  would not have been a component of  $\bar{X}_j$ . Therefore, since  $j_1 \notin \mathcal{E}_j$ , it follows from Line 5 and from (12) that

$$\mathbb{E}[(Z_{j_1} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})} Z_{j_1})^2] \geq \epsilon. \quad (13)$$

Since  $Z_{j_1}$  is the most variant component of  $\bar{X}_j$ , it follows that neither of the more variant  $Z_1, \dots, Z_{j_1-1}$  is a component of  $\bar{X}_j$ , meaning, every  $i \in \{1, \dots, j_1 - 1\}$  was either selected at some earlier step of the algorithm as the most variant component in Line 10, or not selected in Line 10 but rather subtracted from  $\bar{X}_{j'}$  at some earlier step  $j'$  of the algorithm in Line 9 (recall that a component  $i'$  is subtracted in Line 9 of step  $j'$  if belongs to  $\mathcal{E}_{j'}$ , or equivalently by (12), if  $\mathbb{E}[(Z_{i'} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j'-1}})} Z_{i'})^2] < \epsilon$ ).

If the latter is true, i.e., if there exists  $i' < j_1$  and  $j' < j$  such that  $\mathbb{E}[(Z_{i'} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j'-1}})} Z_{i'})^2] < \epsilon$ , it follows that there exists a function  $g \in \text{span } \mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j'-1}})$  such that  $\mathbb{E}[(Z_{i'} - g)^2] < \epsilon$ . That is, the function  $Z_{i'} - g$  is an  $\epsilon$ -redundancy, whose least variant component  $Z_{i'}$  is more variant than  $Z_{j_1}$ , a contradiction to Assumption 1. Therefore, the latter must hold, i.e., every  $i \in \{1, \dots, j_1 - 1\}$  was selected at some earlier step of the algorithm as the most variant component in Line 10, hence  $\mathcal{L}_{j-1} = \{1, \dots, j_1 - 1\}$ , or in other words  $\mathcal{L}_{j-1} = \mathcal{N}_1^\epsilon$ . However,  $\mathcal{L}_{j-1} = \mathcal{N}_1^\epsilon$  implies by (13) that  $\mathbb{E}[(Z_{j_1} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{N}_1^\epsilon})} Z_{j_1})^2] \geq \epsilon$ , contradicting the fact that  $R^{(1)}$  is captured by  $\mathcal{F}(\mathbf{z})$ . This concludes showing that  $j_1 \notin \mathcal{L}$ .

It remains to show that  $\{1, \dots, j_1 - 1\} \subseteq \mathcal{L}$ , which is similar. Assuming otherwise, by following steps similar to above we obtain that some  $i' \in \{1, \dots, j_1 - 1\}$  is the least variant component of some  $\epsilon$ -redundancy, contradicting Assumption 1.

*Induction Hypothesis:* Assume that  $j_1, \dots, j_{i-1} \notin \mathcal{L}$ , and that  $[j_{i-1}] \setminus \{j_1, \dots, j_{i-1}\} \subseteq \mathcal{L}$  for some  $i \geq 2$ .

*Induction Step:* We ought to show that  $j_i \notin \mathcal{L}$  and that  $\{j_{i-1} + 1, \dots, j_i - 1\} \subseteq \mathcal{L}$ . We begin by showing that  $j_i \notin \mathcal{L}$ . Assume for contradiction that  $j_i \in \mathcal{L}$ , and let  $j \in [d]$  be the smallest integer such that  $j_i \in \mathcal{L}_j$ . It follows from Line 10 that

$Z_{j_i}$  is the most variant component of  $\bar{X}_j$ , and similarly to the base case, that  $j_i \notin \mathcal{E}_j$  and

$$\mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j-1}})} Z_{j_i})^2] \geq \epsilon. \quad (14)$$

Since  $Z_{j_i}$  is the most variant component of  $\bar{X}_j$ , it follows that neither of the more variant  $Z_{j_{i-1}+1}, \dots, Z_{j_i-1}$  is a component of  $\bar{X}_j$ , meaning, every  $i' \in \{j_{i-1} + 1, \dots, j_i - 1\}$  was either selected to  $\mathcal{L}$  at some earlier step  $j'$  in Line 10, or it was not selected in Line 10 of any step, but it belongs to  $\mathcal{E}_{j'}$  for some  $j' < j$ . If it is the latter, i.e., if there exists  $i' \in \{j_{i-1} + 1, \dots, j_i - 1\}$  and  $j' < j$  such that  $i' \in \mathcal{E}_{j'}$  but  $i'$  was not selected in Line 10, then  $\mathbb{E}[(Z_{i'} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{L}_{j'-1}})} Z_{i'})^2] < \epsilon$ , which implies the existence of a function  $g \in \text{span } \mathcal{F}(\mathcal{L}_{j'-1})$  such that  $Z_{i'} - g$  is an  $\epsilon$ -redundancy. Therefore, we have that  $Z_{i'} - g$  is an  $\epsilon$ -redundancy whose least variant component  $Z_{i'}$  is neither  $Z_{j_{i-1}}$  nor  $Z_{j_i}$ , a contradiction to Assumption 1. Therefore, the former case must hold, i.e., we must have that each  $i' \in \{j_{i-1} + 1, \dots, j_i - 1\}$  was selected in Line 10, which implies that  $\mathcal{L}_{j-1} = \mathcal{N}_i^\epsilon$  by the induction hypothesis. However,  $\mathcal{L}_{j-1} = \mathcal{N}_i^\epsilon$  implies by (14) that  $\mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{\mathcal{N}_i^\epsilon})} Z_{j_i})^2] \geq \epsilon$ , contradicting the fact that  $R^{(i)}$  is captured by  $\mathcal{F}(\mathbf{z})$ . Showing that each of  $j_{i-1} + 1, \dots, j_i - 1$  are selected to  $\mathcal{L}$  is once again similar.  $\square$

We continue by drawing several conclusions from Theorem 2. In particular, we would like to obtain a better understanding of the term “ $R^{(i)}$  is  $\epsilon$ -captured by  $\mathcal{F}(\mathbf{z})$ ”, and we do so using the well-known *implicit function theorem*. To gain some intuition, observe that since we extract components in decreasing order of variance, eliminating redundancy  $R$  depends on the representation of the least variant variable in  $R$  as a function of all other variables which participate in  $R$ . For example, if  $R(\mathbf{z})$  depends only on  $z_1, z_2, z_3$ , we would like to extract  $z_3$  from  $R(z_1, z_2, z_3) = 0$  in the form of  $z_3 = r(z_1, z_2)$  for some function  $r$  such that  $r(z_1, z_2)$  can be well-approximated by a function in  $\text{span } \mathcal{F}(Z_1, Z_2)$ . Then, since  $Z_1, Z_2$  are extracted prior to  $Z_3$  due their higher variance, the orthogonalization and subtraction processes enable GCA to identify that  $Z_3$  can be determined by  $Z_1, Z_2$ , and hence skip it.

Recall that the implicit function theorem [8], [9] provides sufficient conditions for extracting a variable  $z_j$  from a function  $R(\mathbf{z})$  inside its zero set  $\mathcal{Z}_R \triangleq \{\alpha | R(\alpha) = 0\}$ . This theorem states that if a continuously differentiable<sup>5</sup> function  $R(\mathbf{z})$  satisfies  $\frac{\partial}{\partial z_j} R(\mathbf{z})|_{\mathbf{z}=\alpha} \neq 0$  for some  $j \in [d]$  and some  $\alpha \in \mathcal{Z}_R$ , then there exists a continuous function  $r : \mathbb{R}^{d-1} \rightarrow \mathbb{R}$  such that

$$R(\beta_1, \dots, \beta_{j-1}, r(\beta^{-j}), \beta_{j+1}, \dots, \beta_d) = 0$$

for every  $\beta$  in an environment of  $\alpha$  in  $\mathcal{Z}_R$ , where  $\beta^{-j} \triangleq (\beta_1, \dots, \beta_{j-1}, \beta_{j+1}, \dots, \beta_d)$ . That is, for any point  $\alpha \in \mathcal{Z}_R$  in which the derivative of  $R(\mathbf{z})$  according to the variable  $z_j$  is nonzero, there exists an environment  $\mathcal{V} \subseteq \mathcal{Z}_R$  which contains  $\alpha$  and a function  $g$  such that  $\beta_j = g(\beta^{-j})$  for all  $\beta \in \mathcal{V}$ . We will require a continuous extension of the implicit function theorem, which applies a similar principal to entire connected

<sup>5</sup>That is,  $R$  is differentiable, and its gradient is a continuous function. Further, for  $\alpha \in \mathbb{R}^d$ , the notation  $\frac{\partial}{\partial z_j} R(\mathbf{z})|_{\mathbf{z}=\alpha}$  signifies the result of deriving  $R(\mathbf{z})$  according to  $z_j$ , and then substituting  $z_i \leftarrow \alpha_i$  for all  $i \in [d]$ .



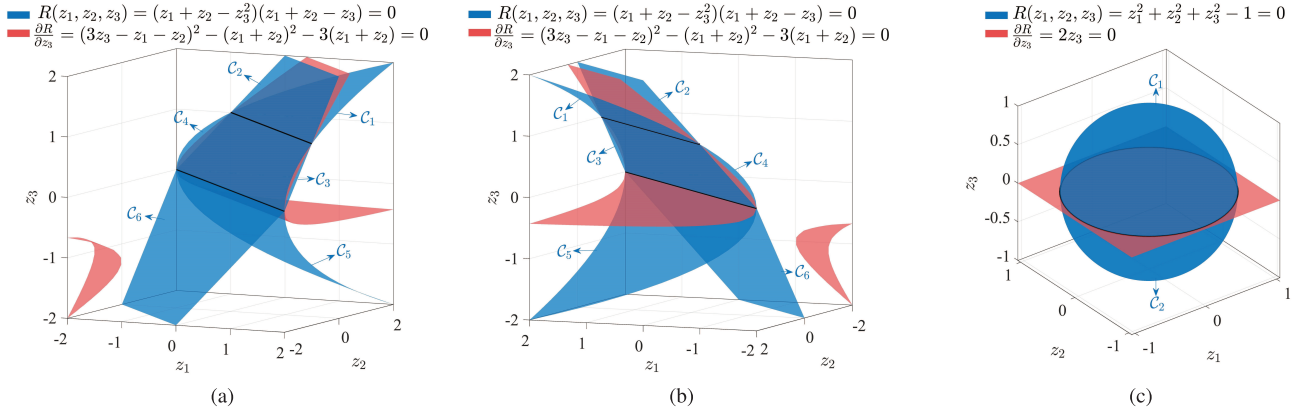


Fig. 2. (a) and (b): Two views of the zero set  $\mathcal{Z}_R$  of  $R(\mathbf{z}) = (z_1 + z_2 - z_3^2)(z_1 + z_2 - z_3)$  (in blue), the zero set of its derivative by  $z_3$  (in red), and their intersection (in black). The respective subset  $\mathcal{B}_3 \subseteq \mathcal{Z}_R$  in which Theorem 3 applies is partitioned to six connected components  $\mathcal{C}_1, \dots, \mathcal{C}_6$ , in each of which  $z_3$  can be isolated as a function of  $z_1, z_2$ . It can be seen that in  $\mathcal{C}_1$  and  $\mathcal{C}_4$  we have  $z_3 = \sqrt{z_1 + z_2}$ , in  $\mathcal{C}_2, \mathcal{C}_3$ , and  $\mathcal{C}_6$  we have  $z_3 = z_1 + z_2$ , and in  $\mathcal{C}_5$  we have  $z_3 = -\sqrt{z_1 + z_2}$ . (c) A similar figure for  $R(\mathbf{z}) = z_1^2 + z_2^2 + z_3^2 - 1$ , where in  $\mathcal{C}_1$  we have  $z_3 = \sqrt{z_1^2 + z_2^2}$  and in  $\mathcal{C}_2$  we have  $z_3 = -\sqrt{z_1^2 + z_2^2}$ .

regions of  $\mathcal{Z}_R$  in which the derivative according to  $z_j$  is nonzero.

**Theorem 3 (Extended Implicit Function Theorem):** [10, Lemma 2.7], [11], [12] Let  $R : \mathbb{R}^d \rightarrow \mathbb{R}$  be a continuously differentiable function, and for  $j \in [d]$  let  $\mathcal{B}_j \subseteq \mathcal{Z}_R$  be the collection of all points  $\beta \in \mathcal{Z}_R$  such that  $\frac{\partial}{\partial z_j} R(\mathbf{z})|_{\mathbf{z}=\beta} \neq 0$ . Further, let  $\mathcal{C}$  be a connected component<sup>6</sup> of  $\mathcal{B}_j$ . Then, there exists a continuous  $g : \mathbb{R}^{d-1} \rightarrow \mathbb{R}$  such that  $\gamma_j = g(\gamma^{-j})$  for all  $\gamma \in \mathcal{C}$ .

In simple terms, Theorem 3 states that the representation of a variable  $z_j$  as a function of  $\mathbf{z}^{-j}$  near a point  $\beta \in \mathcal{Z}_R$  with  $\frac{\partial}{\partial z_j} R(\mathbf{z})|_{\mathbf{z}=\beta} \neq 0$  (as guaranteed by the implicit function theorem), can be continuously extended to the entire portion  $\mathcal{C}$  of  $\mathcal{Z}_R$  reachable from  $\beta$  such that  $\frac{\partial}{\partial z_j} R(\mathbf{z})|_{\mathbf{z}=\gamma} \neq 0$  for all  $\gamma \in \mathcal{C}$  (see examples in Figure 2).

Going back to drawing conclusions from Theorem 2, we begin with the case  $\epsilon = 0$ , in which the data satisfies  $R^{(1)}(Z) = \dots = R^{(t)}(Z) = 0$ . We would like to identify precise conditions by which Theorem 2 holds, i.e., to properly interpret the conditions that the redundancies  $R^{(i)}$  are  $\epsilon$ -captured by  $F(\mathbf{z})$ . To this end, consider the case in which  $\frac{\partial}{\partial z_{j_i}} R^{(i)}(\mathbf{z})|_{\mathbf{z}=\alpha} \neq 0$  for every  $\alpha \in \text{Supp}(X)$  and every  $i \in [t]$ . In this case Theorem 3 implies that there exists a function  $r^{(i)} : \mathbb{R}^{d-1} \rightarrow \mathbb{R}$  which enables  $z_{j_i}$  to be “isolated” from  $R^{(i)}(\mathbf{z}) = 0$  in the form  $z_{j_i} = r^{(i)}(\mathbf{z}^{-j_i})$ . More precisely, since  $R^{(i)}$  need not depend on all  $z_i$ ’s, let  $\mathcal{Z}_i \subseteq [d]$  be the set of variables which appear in  $R^{(i)}$  (notice that  $j_i \in \mathcal{Z}_i$ ). Then, the assumption  $\frac{\partial}{\partial z_{j_i}} R^{(i)}(\mathbf{z}_{\mathcal{Z}_i})|_{\mathbf{z}=\alpha} \neq 0$  for every  $\alpha \in \text{Supp}(X)$  implies the ability to write  $z_{j_i} = r^{(i)}(\mathbf{z}_{\mathcal{Z}_i}^{-j_i})$ . The existence of these  $r^{(i)}$ ’s enables two conclusions:

**C1.** By composing functions, we can justify the “inter-dependence” in Assumption 1. For example, if  $r^{(2)}$  depends on  $z_{j_1}$ , one can substitute  $z_{j_1}$  for its functional representation  $r^{(1)}(\mathbf{z}_{\mathcal{Z}_1}^{-j_1})$ , and eliminate the (direct) dependence of  $r^{(2)}$  on  $z_{j_1}$ . By applying this argument repeatedly for all “inter-dependencies” among the  $z_{j_i}$ ’s,

one can write  $z_{j_i} = r^{(i)}(\mathbf{z}_{\mathcal{N}_i})$  for all  $i$ , where  $\mathcal{N}_i = [j_i - 1] \setminus \{j_\ell\}_{\ell=1}^{i-1}$ .

**C2.** Following **C1**, we obtain a precise interpretation of the term “the 0-redundancy  $R^{(i)}$  is 0-captured by  $\mathcal{F}(\mathbf{z})$ .” Clearly, in the case where  $z_{j_i} = r^{(i)}(\mathbf{z}_{\mathcal{N}_i})$  we have that  $\mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathbf{z}_{\mathcal{N}_i})} Z_{j_i})^2] = 0$  if and only if  $r^{(i)}(\mathbf{z}_{\mathcal{N}_i}) \in \text{span } \mathcal{F}(\mathbf{z}_{\mathcal{N}_i})$ .

Conclusions **C1** and **C2** readily imply the first corollary of Theorem 2.

**Corollary 1:** Let  $R^{(1)}, \dots, R^{(t)}$  be the 0-redundancies of  $X$ . If (1) the derivative of each  $R^{(i)}$  according to its least variant variable  $z_{j_i}$  is nonzero on the data support; and (2) the resulting functions  $r^{(i)}(\mathbf{z}_{\mathcal{N}_i})$  are in  $\text{span } \mathcal{F}(\mathbf{z}_{\mathcal{N}_i})$  for all  $i$ , then GCA outputs  $\mathcal{N}^0 = [d] \setminus \{j_1, \dots, j_t\}$ . In words, if the least variant variable in each 0-redundancy can be isolated as a function of the remaining variables, and if that function belongs to  $\text{span } \mathcal{F}$ , then GCA eliminates *all* redundancies.

Next, to extend Corollary 1, we consider cases in which the 0-redundancies satisfy the assumptions of the implicit function theorem only up to measure zero. As shown next, under additional assumptions GCA can guarantee successful elimination of the redundancies in this case as well.

Suppose as before that the data contains the 0-redundancies  $R^{(1)}, \dots, R^{(t)}$ , each with least variant variable  $z_{j_i}$ . However, we relax the assumption of Corollary 1, and only assume that the respective derivatives vanish on a subset of measure zero. That is, for every  $i \in [t]$  we assume that  $\Pr(Z \in \mathcal{B}_i) = 1$ , where  $\mathcal{B}_i = \{\beta \in \mathbb{R}^d : \frac{\partial}{\partial z_{j_i}} R(\mathbf{z})|_{\mathbf{z}=\beta} \neq 0\}$ . This assumption is not overly restrictive since in many cases the zero set of the derivative will have Lebesgue measure zero, hence measure zero for many continuous distributions. For example, consider Figure 2c under the normalized Gaussian distribution. It is readily verified that the probability to sample a point on the black line is zero.

Clearly, the region  $\mathcal{B}_i$  need not be connected (see Figure 2), and hence we let  $\mathcal{C}_{i,1}, \dots, \mathcal{C}_{i,\ell_i}$  be its connected components. Using Theorem 3 on each one of  $\mathcal{C}_{i,1}, \dots, \mathcal{C}_{i,\ell_i}$  separately guarantees that there exists functions  $g_{i,1}, \dots, g_{i,\ell_i}$  which enable

<sup>6</sup>A connected component is an equivalence class of a relation  $\sim$  over  $\mathcal{B}_j$ , where  $\beta \sim \beta'$  if there exists a continuous path  $q(t)$  such that  $q(0) = \beta$ ,  $q(1) = \beta'$ , and  $q(t) \in \mathcal{B}_j$  for all  $t \in [0, 1]$ .

to extract  $z_{j_i}$  as a function of  $\mathbf{z}^{-j_i}$  in each of  $\mathcal{C}_{i,1}, \dots, \mathcal{C}_{i,\ell_i}$ , respectively. That is, we have that  $\alpha_{j_i} = g_{i,j}(\alpha^{-j_i})$  for every  $i \in [\ell]$  and every  $\alpha \in \mathcal{C}_{i,j}$ . The success of GCA in this case will follow if  $\mathcal{F}(\mathbf{z})$  can approximate each function  $g_{i,j}$  *locally*, meaning, in the connected component  $\mathcal{C}_{i,j}$  in which it applies. To this end, for  $i \in [t]$  we say that the 0-redundancy  $R^{(i)}$  is  $\epsilon$ -*locally captured* by  $\mathcal{F}(\mathbf{z})$  if  $\mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{N_i^\epsilon})} Z_{j_i})^2 | Z \in \mathcal{C}_{i,j}] \leq \epsilon$  for every  $j \in [\ell_i]$ .

*Corollary 2:* For  $\epsilon > 0$ , suppose that  $X$  contains redundancies  $R^{(1)}, \dots, R^{(t)}$  of either of two types:

- A.  $\epsilon$ -redundancies; or
- B. 0-redundancies whose derivatives by the least variant variable vanishes only on a set of measure zero.

Further, suppose that every redundancy of type **A** is  $\epsilon$ -captured by  $\mathcal{F}(\mathbf{z})$ , and every redundancy of type **B** is  $\epsilon$ -locally captured by  $\mathcal{F}(\mathbf{z})$ . Then, GCA outputs<sup>7</sup>  $\mathcal{N}^\epsilon = [d] \setminus \{j_1, \dots, j_t\}$ .

*Proof:* Observe that for every redundancy  $R^{(i)}$  of type **B** we have

$$\begin{aligned} & \mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{N_i^\epsilon})} Z_{j_i})^2] \\ &= \sum_{a=1}^{\ell_i} \Pr(Z \in \mathcal{C}_{i,a}) \mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{N_i^\epsilon})} Z_{j_i})^2 | Z \in \mathcal{C}_{i,a}] \\ &\leq \epsilon \sum_{a=1}^{\ell_i} \Pr(Z \in \mathcal{C}_{i,a}) \leq \epsilon, \end{aligned}$$

where the latter inequality follows since  $\Pr(Z \notin \cup_{a=1}^{\ell_i} \mathcal{C}_{i,a}) = 0$  by the assumption on the respective derivative of redundancies of type **B**. Therefore, it follows that each of  $R^{(1)}, \dots, R^{(t)}$  is an  $\epsilon$ -redundancy which is captured by  $\mathcal{F}(\mathbf{z})$ , and hence the lemma follows from Theorem 2.  $\square$

In particular, for a redundancy  $R^{(i)}$  of type **B** in Corollary 2 we have that  $Z_{j_i} = r_j^{(i)}(\mathcal{Z}_{N_i^\epsilon})$  for some function  $r_j^{(i)}$  in the connected component  $\mathcal{C}_{i,j}$ , and therefore the term “ $R^{(i)}$ ” is  $\epsilon$ -locally captured by  $\mathcal{F}(\mathbf{z})$  can be interpreted as

$$\begin{aligned} & \mathbb{E}[(Z_{j_i} - \text{proj}_{\mathcal{F}(\mathcal{Z}_{N_i^\epsilon})} Z_{j_i})^2 | Z \in \mathcal{C}_{i,j}] \\ &= \mathbb{E}[(r_j^{(i)}(\mathcal{Z}_{N_i^\epsilon}) - \text{proj}_{\mathcal{F}(\mathcal{Z}_{N_i^\epsilon})} r_j^{(i)}(\mathcal{Z}_{N_i^\epsilon}))^2 | Z \in \mathcal{C}_{i,j}] \leq \epsilon, \end{aligned}$$

meaning,  $\mathcal{F}(\mathcal{Z}_{N_i^\epsilon})$  can  $\epsilon$ -approximate the function  $r_j^{(i)}$  in the connected component  $\mathcal{C}_{i,j}$ .

In summary, Theorem 2 implies that whenever Assumption 1 is satisfied, GCA successfully removes all  $\epsilon$ -redundancies from the data for any  $\mathcal{F}(\mathbf{z})$  which is sufficiently descriptive, i.e., it can approximate the least variant component in each redundancy. To interpret what this approximation means, in Corollary 1 it was shown that for  $\epsilon = 0$ , the implicit function theorem can further imply that the descriptiveness of  $\mathcal{F}$  reduces to  $\text{span}(\mathcal{F})$  containing the “isolation” of each least variant variable from its respective redundancy; this required nonzero derivative by that variable in all of the data support. The latter requirement was alleviated in Corollary 2 in the case where nonzero derivatives existed almost everywhere, where strict containment of the isolation in  $\text{span}(\mathcal{F})$  is relaxed to  $\epsilon$ -approximation.

For  $\epsilon$ -redundancies with  $\epsilon > 0$ , interpreting Theorem 2 is more challenging. The main reason is that the existence of such redundancy  $R$  does not necessarily imply that  $R(Z) = 0$ ,

and hence Theorem 3 cannot be directly applied. Therefore, the term “capturing” does not translate to containing some function in  $\text{span}(\mathcal{F})$  (Corollary 1), or containing some local  $\epsilon$ -approximation (Corollary 2).

However, an interesting extension of Corollary 1 arises in the case  $\epsilon > 0$ , which applies for the case with 0-redundancies and  $\mathcal{F}(\mathbf{z})$  that is *not* sufficiently descriptive in order to capture, but can only  $\epsilon$ -approximate, those redundancies. The next corollary follows immediately from Theorem 2 since a 0-redundancy is also an  $\epsilon$  redundancy for every  $\epsilon > 0$ .

*Corollary 3:* Suppose there exists 0-redundancies  $R^{(1)}, \dots, R^{(t)}$  in  $X$  such that  $Z_{j_i} = r^{(i)}(\mathcal{Z}_{N_i^0})$  for every  $i \in [t]$ . Further, suppose that for  $\mathcal{F}(\mathbf{z})$  and some  $\epsilon > 0$  we have

$$\mathbb{E}[(r^{(i)}(\mathcal{Z}_{N_i^0}) - \text{proj}_{\mathcal{F}(\mathcal{Z}_{N_i^0})} r^{(i)}(\mathcal{Z}_{N_i^0}))^2] \leq \epsilon. \quad (15)$$

Then GCA can eliminate all the redundancies.

Corollary 3 states that if  $\mathcal{F}(\mathbf{z})$  is insufficiently descriptive in order to *contain* the 0-redundancies in its span (as in Corollary 1), but can reasonably *approximate* them, then GCA still succeeds in eliminating the redundancy. One can extend Corollary 3 by employing the implicit function theorem globally (as in Corollary 1), or locally (as in Corollary 2), and the details are omitted.

*Example 3:* Suppose that  $X$  satisfies some 0-redundancies such that the respective  $r^{(i)}$  from Corollary 3 are polynomials of degree 5. Further suppose that  $\mathcal{F}(\mathbf{z})$  are polynomials of degree 3 which can  $\epsilon$ -approximate the  $r^{(i)}$ 's in the sense of (15). Then GCA with  $\mathcal{F}(\mathbf{z})$  and  $\epsilon$  eliminates the redundancy.

A complexity tradeoff is evident from Corollary 3 and Example 3. One can use simpler function family  $\mathcal{F}(\mathbf{z})$  with  $\epsilon > 0$  instead of a more complex one with  $\epsilon = 0$ , and reduce the runtime of GCA. As long as the simpler  $\mathcal{F}(\mathbf{z})$  is able to  $\epsilon$ -approximate the redundancies, GCA will succeed. If the simpler  $\mathcal{F}(\mathbf{z})$  cannot  $\epsilon$ -approximate some redundancy, it will be treated as non-redundant.

*Remark 4:* In order to corroborate Corollaries 1, 2, and 3, GCA was implemented and tested on synthetic data sets successfully, and the results are reported in Section VIII-C.

## V. GRAM-SCHMIDT FUNCTIONAL SELECTION (GFS)

Throughout the feature selection algorithms (in this section and in the following one), we make use of the Hadamard product  $\otimes$ , where for two vectors  $\mathbf{x} = (x_i)_{i=1}^n, \mathbf{y} = (y_i)_{i=1}^n$  we have  $\mathbf{x} \otimes \mathbf{y} = (x_i y_i)_{i=1}^n$ ; similarly  $\mathbf{x}^{\otimes 2} = (x_i^2)_{i=1}^n$ .

In what follows, it is shown that a slight variant of GFR provides a feature selection algorithm with similar information-theoretic guarantees. Specifically, we replace the maximization step (which finds the eigenvector  $\mathbf{v}_j$ , Line 5 of Algorithm 2) by a discrete version, and similarly terminate if the resulting variance is smaller than the threshold  $\epsilon^2$ . The output of the algorithm is a subset  $\mathcal{S}_\epsilon \subseteq [d]$  of selected features. In feature selection algorithms, we only need to compute the alternative *variance vectors*  $\sigma_j$  (a variance vector of a random variable is the diagonal of its covariance matrix) instead of covariance matrices  $\Sigma_j$  in their entirety. Using  $\sigma_j$ 's instead of  $\Sigma_j$ 's allows some simplifications. For instance, if

<sup>7</sup>Note that a 0-redundancy is also an  $\epsilon$ -redundancy for every  $\epsilon > 0$ , and hence one can denote  $\mathcal{R}^\epsilon, \mathcal{N}^\epsilon, \mathcal{N}_i^\epsilon$  in accordance with Assumption 1.

**Algorithm 4** Gram-Schmidt Functional Selection (GFS)

---

```

1: Input: A function family  $\mathcal{F}(\mathbf{z})$  and a threshold  $\epsilon > 0$ .
2: Output: Selected features  $s_1, \dots, s_m$  ( $m$  is a varying
   number that depends on  $\mathcal{F}(\mathbf{z})$ ,  $\epsilon$ , and  $X$ )
3: Initialize:  $\sigma_1 = \mathbb{E}[X^{\otimes 2}]$ .
4: for  $j \leftarrow 1$  to  $d$  do
5:   Let  $s_j \triangleq \arg \max_{i \in [d]} \{\sigma_{j,i}\}_{i=1}^d$ , where  $\sigma_j = (\sigma_{j,i})_{i=1}^d$ .
6:   if  $\|\sigma_j\|_\infty \leq \epsilon^2$  then
7:     break.
8:   else
9:     Define  $Z_j = X_{s_j}$ .
10:  end if
11:   $\sigma_{j+1}, \hat{\mathcal{F}}(Z_1, \dots, Z_j) = \text{Orthogonalize}(\hat{\mathcal{F}}(Z_1, \dots, Z_{j-1}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{Z_i\}_{i=1}^j)$ 
12: end for

```

---

Algorithm 1 is used for feature *selection* purposes, we make the following simple changes:

(Line 13) Define  $\Sigma_{j+1} = \mathbb{E}[d_j(X)d_j^T(X)]$   
 $\rightarrow$  Define:  $\sigma_{j+1} = \mathbb{E}[d_j(X)^{\otimes 2}]$ .  
 (Line 14) Return  $\Sigma_{j+1}, \mathcal{T} \rightarrow$  Return  $\sigma_{j+1}, \mathcal{T}$ .

In Algorithm 4 below we propose *Gram-Schmidt Functional Selection* (GFS), which at every step  $j$  uses the variance vector  $\sigma_j$  to select the most variant feature of  $d_{j-1}$ . If the variance of this feature is less than some threshold  $\epsilon^2$ , the algorithm stops, and otherwise it continues by specifying  $Z_j$  as the highest variance feature of  $d_{j-1}$ , which enables the next call to “Orthogonalize.” The equivalent algebraic counterpart of GFS, called *Algebraic-GFS*, is presented in Algorithm 10 in Appendix D-C. The formal guarantees are very similar to those of GFR (Section III), and also require a discrete  $X$ . Yet, experiments in Section VIII show significant gains in real-world setting in which the data is continuous. The information-theoretic guarantee of GFS is as follows.

**Theorem 4:** Suppose  $X$  is over a discrete domain  $\mathcal{X}^d$ , for some  $\mathcal{X} \subseteq \mathbb{R}$ , and let  $\mathcal{S}_\epsilon = \{s_1, \dots, s_m\}$  be the out of GFS whose input is a given  $\epsilon$  and  $\mathcal{F}(\mathbf{z})$ . Then  $H(X|X_{\mathcal{S}_\epsilon}) \leq dO(\epsilon)$ .

The proof of Theorem 4 is very similar to that of Theorem 1, and is given in full in Appendix A.

## VI. GRAM-SCHMIDT FEATURE ANALYSIS (GFA)

As mentioned in Section IV, Gram-Schmidt Component Analysis (GCA) can serve as an additional layer of redundancy removal beyond PCA. While PCA identifies if the data lies on a subspace (or closely so, if one wishes to ignore principal directions whose respective eigenvalue is small), GCA can further identify if nonlinear redundancies exist within that subspace. However, in GCA it is merely assumed that  $X = \sum_{i=1}^d \rho_i Z_i$ , and the fact that the  $\rho_i$ ’s are the principal directions does not play a role other than them being orthonormal. Therefore, one can substitute the principal directions  $\{\rho_i\}_{i=1}^d$  in GCA by any other orthonormal basis of  $\mathbb{R}^d$ , and obtain similar results.

One such orthonormal basis which is of interest, other than the principal directions, is the standard basis. By

**Algorithm 5** Gram-Schmidt Feature Analysis (GFA)

---

```

1: Input: A function family  $\mathcal{F}(\mathbf{z})$  and a threshold  $\epsilon > 0$ .
2: Output: A set  $\mathcal{S} \subseteq [d]$  of indices of selected features.
3: Initialize:  $\sigma_1 = \mathbb{E}[X^{\otimes 2}]$  and  $\mathcal{S}_0 = \emptyset$ .
4: for  $j \leftarrow 1$  to  $d$  do
5:   Let  $\mathcal{E}_j = \{i | \sigma_{j,i} < \epsilon\}$ .
6:   if  $\mathcal{E}_j = [d]$  then
7:     return  $\mathcal{S}_{j-1}$ .
8:   end if
9:   Let  $s_j \triangleq \arg \max_{i \in [d] \setminus \mathcal{E}_j} \{\sigma_{j,i}\}$ 
   and  $\mathcal{S}_j = \mathcal{S}_{j-1} \cup \{s_j\}$ .
10:   $\sigma_{j+1}, \hat{\mathcal{F}}(Z_{\mathcal{S}_j}) = \text{Orthogonalize}(\hat{\mathcal{F}}(Z_{\mathcal{S}_{j-1}}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{Z_{s_i}\}_{i=1}^j)$ 
11: end for
12: return  $\mathcal{S}_d$ .

```

---

substituting the principal directions in GCA by the standard basis, the guarantees of GCA pertaining to redundancy elimination along the principal directions are substituted by similar redundancy elimination guarantees along the *features themselves*. Arguably, the ability to eliminate redundancy along the features themselves is also of great importance. Therefore, in this section we present *Gram-Schmidt Feature Analysis* (GFA), an extension of GCA which serves a different purpose. GFA is given in Algorithm 5, which is obtained directly from GCA by substituting principal directions by the standard basis, and covariance matrices by covariance vectors. To maintain notational consistency with GCA we denote  $Z_i = X_i$ . The corresponding equivalent algebraic formulation, denoted *Algebraic-GFA*, is presented in Algorithm 11 in Appendix D-C.

The proof of the following theorem can be easily verified.

**Theorem 5:** Gram-Schmidt Feature Analysis (GFA, Algorithm 5) satisfies Theorem 2, as well as Corollaries 1, 2, and 3, while replacing each principal direction  $\rho_i$  by the standard basis vector  $\mathbf{e}_i$  (and in particular replacing  $Z_i = X^\top \rho_i$  by  $Z_i = X_i$ ).

## VII. LITERATURE REVIEW

Unsupervised dimension reduction is an old topic, dating as far back as 1901. The literature in this area is vast, out of which we summarize the main key contributions below. PCA is a well-known technique [13]; for a contemporary introduction see [14]. Multidimensional scaling (MDS) [15] is a class of methods (which includes Isomap [16]) whose objective is to maximize the scatter of the projection, under the rationale that doing so would yield the most informative projection. There exists a duality between PCA and MDS where scatter is measured by Euclidean distance [15], [17]. Maximum Autocorrelation Factors (MAF) and Slow Feature Analysis are linear feature extraction methods tailored to multivariate time series data [18], [19]. Locality Preserving Projections (LPP) aim to find a linear transformation of the data that preserves the local pairwise relationships among data points [20]. Locally Linear Embedding (LLE) [21] finds representations by preserving local linear relationships between

adjacent points. Independent component analysis (ICA) [22] is used to separate a multivariate signal into independent, non-Gaussian components, but is also popular for feature extraction. t-Distributed Stochastic Neighbor Embedding (t-SNE) is a nonlinear technique widely used for visualizing high-dimensional data. It minimizes the divergence between two probability distributions: one representing pairwise similarities of data points in the original space and the other in the reduced space. This method excels at preserving local structures and revealing intricate patterns, making it an invaluable tool for exploring and interpreting complex datasets in two or three dimensions [23]. Uniform Manifold Approximation and Projection (UMAP) is a dimensionality reduction technique that can be used for visualization purposes similar to t-SNE, as well as for general nonlinear dimensionality reduction tasks. It is particularly good at preserving both local and global structure of the data. UMAP is based on three key assumptions: the data is uniformly distributed on a Riemannian manifold, the Riemannian metric is locally constant or can be approximated as such, and the manifold is locally connected [24]. Autoencoders are hourglass shaped neural networks usually trained to minimize the Euclidean deviation of their input from their output; the output of the narrowest intermediate layer is the ensuing nonlinear dimension reduction mechanism. Their ability to learn nonlinear transformations makes them a powerful tool for reducing the dimensionality of complex datasets [25].

Main techniques in the area include Laplacian score [26], [27], Fisher score [28], [29], and trace ratio [30], which assess the importance of features by their ability to preserve data similarity; Multi-cluster feature selection [31], [32], nonnegative discriminative feature selection [33], and unsupervised discriminative feature selection [34], [35], which assume sparsity and employ lasso type minimization; and minimal-redundancy-maximal-relevance [36], mutual information based feature selection [37], and fast correlation based filter [38], which maximize mutual information between the original and selected features.

Reference [39] proposed to use information entropy to evaluate the importance of features, and used the trace criterion to select the most relevant feature subset. Reference [40] proposes to use a competitive learning to divide the original feature into subsets, and the subset which minimizes dispersion is selected. Reference [41] analyzed the distribution of the feature data by use of the probability density of different features; a feature is selected by the data distribution relation among the other features. Reference [42] uses the maximum information compression index to measure the similarity between features. Reference [43] first calculates the maximal information coefficient for each attribute pair to construct an attribute distance matrix, and then clusters all attributes and selects features according to their cluster affiliation. Some further techniques use the Laplacian score [26], [44], [45], which measures the local topology of the data clusters.

Additionally, several generalizations of PCA were proposed in the literature. First, kernel PCA [14, Ch. 19.9.1] is a popular method for *nonlinear* dimension reduction, in which data is embedded into a higher dimensional space using an embedding

function  $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ , which is in general not known a priori. Then, principal directions are found in  $\mathbb{R}^D$ , aided by the kernel trick to reduce computational complexity. The resulting dimension reduction is obtained by projecting data transformed by  $\Phi$  on the principal directions in  $\mathbb{R}^D$ ; since each entry in  $\mathbb{R}^D$  is a nonlinear function of the entries in  $\mathbb{R}^d$ , this yields a *nonlinear* dimension reduction method. Therefore, kernel PCA is different from all methods proposed in this paper, which are linear. Yet, for completeness we discuss several similarities and differences between our approach and kernel PCA. In terms of similarities, both methods seek high variance directions in a space transformed by a nonlinear function—the function  $\Phi$  mentioned above in the case of kernel PCA, and the function  $\mathbf{z} \mapsto (f(\mathbf{z}))_{f \in \mathcal{F}}$  in our case—in which each entry is a nonlinear function of the entries in  $\mathbb{R}^d$ . However, our methods rely on devising a *linear* transform  $V$  in  $\mathbb{R}^D$  which is applied before the function  $\mathbf{z} \mapsto (f(\mathbf{z}))_{f \in \mathcal{F}}$ , i.e., in a sense our methods are based on exploring high variance directions in a space transformed by  $\mathbf{z} \mapsto (f(\mathbf{z}^T V))_{f \in \mathcal{F}}$ . More so, the linear transform  $V$  is not determined a priori (unlike kernel PCA, in which  $\Phi$  must be determined a priori), but discovered throughout the algorithm—each row  $\mathbf{v}_i$  of  $V$  is determined according to variance properties in the space  $(f(X^T \mathbf{v}_1), \dots, X^T \mathbf{v}_{i-1}))_{f \in \mathcal{F}_{i-1}}$  (where  $\mathcal{F}_i \subseteq \mathcal{F}$  is a subset which depends only on the first  $i-1$  variables). The resulting dimension reduction in our case is given by projecting the original data on the rows of  $V$ , which is fundamentally different from kernel PCA where it is given by computing  $\Phi$  and projecting over the principal directions in  $\mathbb{R}^D$ .

Second, generalized PCA [46] is a framework which assumes the data lies on a *union* of subspaces of  $\mathbb{R}^d$ . Using De-Morgan's laws, it is shown [46, Thm. 2] that data which lies on a union of  $\ell$  subspaces also lies in the kernel of a homogeneous polynomial system. The number of polynomials in this system is the product of the co-dimensions of the subspaces, and all polynomials only contain monomials of degree exactly  $\ell$ . Then, using differential calculus, the bases of these subspaces are found. Whenever  $\mathcal{F}(\mathbf{z})$  is a set of monomials, the definition of redundancy in Section IV (also Section VI) implies that  $X$  satisfies a certain polynomial system as well, albeit not necessarily homogeneous nor containing only monomials of a certain degree. The similarities to generalized PCA do not seem to extend beyond that, as generalized PCA framework addresses only a very particular form of redundancy.

Finally, similar to our work, function-space orthogonality is also employed in the theory of *Principal Inertia Components* (PIC) [47], [48], where it is used to define the *principal functions* of a pair of random variables  $(X, Y)$ . These principal functions can then be used to compute functions of  $X$  from samples of  $Y$ , and vice versa. While evaluations of the principal functions can be seen as a dimension reduction method, there are several substantial difference between the PIC paradigm and this paper: (a) the principal functions are highly nonlinear and in practice computed via a neural network (as opposed to our linear methods); (b) PIC is tailored for simultaneous analysis of two variables (rather than one variable in our methods); and (c) in our methods the orthogonal functions are a tool in the dimension reduction process (e.g., to define



the alternative covariance matrices), and not the dimension reduction itself as in PIC.

### VIII. EXPERIMENTAL RESULTS

In this section the theoretical results and performance of the proposed algorithms are evaluated through simulation studies over synthetic and real-world datasets. Below we detail the methods and the rationale behind each set of experiments.

Section VIII-A supports the claim that GFR significantly reduces the data's residual variance in comparison with PCA. Intuitively, residual variance captures how variant the data is after extracting features. Formally, in PCA residual variance after extracting  $m$  features is captured by the  $(m + 1)$ 'th eigenvalue of the covariance matrix  $\Sigma_{m+1}$ , and in GFR it is the largest eigenvalue of the alternative covariance matrix  $\Sigma_{m+1}$ . In both cases, residual variance is the largest eigenvalue of the covariance matrix, after setting to zero the parts of the data which were extracted so far. We show that for a similar threshold  $\epsilon^2$ , GFR achieves similar residual variance using significantly fewer extracted features in comparison to PCA, by almost an order of magnitude in certain cases.

In section VIII-B we evaluate GFR for classification tasks over benchmark datasets and compare the results to other well-known unsupervised feature extraction algorithms: Kernel PCA, FastICA, Locality Preserving Projections (LPP), UMAP, and autoencoders; this is a representative list of the best known linear methods, and the most popular nonlinear ones. We use each method for feature extraction while disregarding the labels. The labels are later re-attached, a classifier is trained over the extracted features, and the resulting accuracies are compared.

In section VIII-C, we study the performance of GCA based on its success in removing all redundant features among principal components in synthetic datasets. Although its correctness is formally proven, we study the effect of employing empirical covariance matrices, and its connection to dataset size. We show that GCA can indeed remove all redundant principal components from the data, given moderately large datasets. On the other hand, GCA's performance degrades for smaller datasets, particularly in ones with higher-degree multilinear polynomial redundancy.

In section VIII-D we turn to experiment on our feature selection techniques, and similar to the experiments for feature extraction, we measure variance reduction and classification accuracy. We begin by showing that selecting higher degree multilinear polynomials as the redundant functions  $\mathcal{F}(\mathbf{z})$  in GFS results in a significant reduction in maximum variance among the entries of  $d_j(X)$  in each iteration (i.e.,  $\max\{\text{Var}[d_j(X)_i]\}_{i=1}^d$ ). We proceed in Section VIII-E to validate the performance of GFS for classification tasks on the benchmark datasets. The numerical experiments show superior performance of GFS in comparison to other algorithms.

In Section VIII-F, We corroborate GFS's superiority over Unsupervised Fourier Feature Selection (UFFS) [5] in terms of running time, the ability to capture redundant features, and classification accuracy, over synthetic datasets. Finally, we study the performance of GFA over synthetic datasets in section VIII-G.

TABLE I  
PROPERTIES OF THE TESTED BENCHMARK DATASETS

| Dataset  | USPS | MNIST | COIL-20 | Musk | Credit Approval |
|----------|------|-------|---------|------|-----------------|
| Features | 256  | 784   | 1024    | 166  | 15              |
| Samples  | 7291 | 60000 | 1440    | 6597 | 690             |

The experiments were performed on a laptop with Intel(R) Core(TM) i9-9880H CPU @ 2.30GHz and 64GB RAM, and the source code is available at [https://github.com/byaghooti/Gram\\_schmidt\\_feature\\_extraction](https://github.com/byaghooti/Gram_schmidt_feature_extraction).

#### A. Residual Variance Reduction of GFR

To support our claim that GFR results in significant variance reduction in comparison to PCA, we apply GFR to benchmark datasets taken from UCI repository [49] and [50]. The properties of the tested benchmark datasets are provided in Table I.

We compare PCA against GFR, where  $\mathcal{F}(\mathbf{z})$  in GFR is set as multilinear polynomials of degrees up to 2, 3, or 4, and the results are shown in Table II. We compare the amount of residual variance left in the dataset with the number of extracted features. Specifically, Table II shows the number of features that had to be extracted to achieve a certain amount of residual variance.

Table II clearly shows a drastic reduction of residual variance as the degree of the multilinear polynomials increases, showcasing that the redundancies in these datasets are well described by them. We emphasize the particular power of multilinear polynomials of degree 4, achieving up to 6-10 times fewer extracted features in USPS (as opposed to 2-5 times in other cases with USPS), up to 24 fewer features for MNIST, and up to 6 times fewer features for Musk.

We also compare GFR with Kernel PCA, where  $\mathcal{F}(\mathbf{z})$  in GFR consists of multilinear polynomials of degrees up to 4 and general polynomials of degrees up to 4, and Kernel PCA is used with a degree 4 polynomial kernel. Kernel PCA maps the features to a higher-dimensional space using a nonlinear kernel and computes the principal components in the transformed space, resulting in a feature representation with a different statistical distribution compared to the original space. Thus, we cannot directly compare the variance of the extracted features from GFR and Kernel PCA. Instead, when extracting  $m$  features, we compare the ratio of the variance of the  $(m + 1)$ 'th feature to the variance of the first extracted feature, i.e.,  $\text{Var}(Z_{m+1}) / \text{Var}(Z_1)$ , where  $Z_1$  and  $Z_{m+1}$  are the first and  $(m + 1)$ 'th extracted features, respectively. This ratio normalizes the variance of the features with respect to the first extracted feature. The lower this ratio, the more effectively the algorithm removes redundant features, as it indicates how much of the variance of the first extracted feature remains in the data after extracting  $m$  features. Table III shows this ratio for different numbers of extracted features on benchmark datasets. The results show that, in most benchmark datasets (except Credit Approval) GFR achieves a lower ratio, demonstrating its ability to remove redundant features better than Kernel PCA.

TABLE II

NUMBER OF EXTRACTED FEATURES BY GFR WITH MULTILINEAR POLYNOMIALS ON BENCHMARK DATASETS FOR DIFFERENT VALUES OF THE THRESHOLD  $\epsilon^2$

| Degree of Polynomial | $\epsilon^2$ |      |     |     |    |
|----------------------|--------------|------|-----|-----|----|
|                      | 0.1          | 0.15 | 0.2 | 0.5 | 1  |
| 1 (PCA)              | 208          | 130  | 113 | 70  | 45 |
| 2                    | 61           | 52   | 47  | 30  | 21 |
| 3                    | 29           | 27   | 25  | 20  | 16 |
| 4                    | 20           | 19   | 19  | 17  | 14 |

(a) USPS Dataset

| Degree of Polynomial | $\epsilon^2$ |      |     |     |     |      |
|----------------------|--------------|------|-----|-----|-----|------|
|                      | 0.01         | 0.05 | 0.1 | 0.2 | 0.5 | 0.75 |
| 1 (PCA)              | 696          | 576  | 451 | 344 | 233 | 191  |
| 2                    | 205          | 145  | 114 | 84  | 53  | 43   |
| 3                    | 55           | 49   | 45  | 39  | 31  | 26   |
| 4                    | 29           | 28   | 27  | 25  | 22  | 20   |

(b) MNIST Dataset

| Degree of Polynomial | $\epsilon^2$ |      |      |     |     |     |
|----------------------|--------------|------|------|-----|-----|-----|
|                      | 0.01         | 0.02 | 0.05 | 0.1 | 0.2 | 0.5 |
| 1 (PCA)              | 577          | 477  | 353  | 264 | 189 | 120 |
| 2                    | 50           | 47   | 42   | 38  | 33  | 27  |
| 3                    | 21           | 20   | 19   | 18  | 17  | 15  |
| 4                    | 14           | 14   | 14   | 14  | 13  | 12  |

(c) COIL-20 Dataset

| Degree of Polynomial | $\epsilon^2$ |      |      |     |     |     |    |
|----------------------|--------------|------|------|-----|-----|-----|----|
|                      | 0.01         | 0.02 | 0.05 | 0.1 | 0.2 | 0.5 | 1  |
| 1 (PCA)              | 122          | 104  | 83   | 68  | 52  | 37  | 26 |
| 2                    | 51           | 46   | 38   | 32  | 27  | 20  | 16 |
| 3                    | 27           | 26   | 23   | 21  | 19  | 15  | 13 |
| 4                    | 20           | 19   | 18   | 17  | 16  | 14  | 12 |

(d) Musk Dataset

| Degree of Polynomial | $\epsilon^2$ |      |   |
|----------------------|--------------|------|---|
|                      | 0.5          | 0.75 | 1 |
| 1 (PCA)              | 14           | 11   | 7 |
| 2                    | 11           | 8    | 6 |
| 3                    | 9            | 8    | 6 |
| 4                    | 9            | 8    | 5 |

(e) Credit Approval Dataset

## B. Classification Accuracy of GFR

### 1) Classification Accuracy of GFR in Benchmark Datasets:

We validate GFR for classification tasks on the benchmark datasets in Table I. In Table IV, we apply GFR with multilinear polynomials of degree up to 4 against Kernel PCA, FastICA, PCA, and LPP. The classification accuracy in the benchmark datasets shows overall superior performance of GFR in comparison to other algorithms. It is worth mentioning that as we were expecting, the classification accuracy of GFR is higher than PCA in all the experiments in the mentioned benchmark datasets.

We used a support vector machine classifier with radial basis function as kernel. 5-fold cross-validation

TABLE III

COMPARISON OF THE RESIDUAL VARIANCE REDUCTION OF GFR WITH MULTILINEAR POLYNOMIALS OF DEGREE UP TO 4 AND POLYNOMIALS OF DEGREE UP TO 4 AND KERNEL PCA WITH A POLYNOMIAL KERNEL OF DEGREE 4 ON BENCHMARK DATASETS. THE ENTRIES REPRESENT THE RATIO BETWEEN THE VARIANCE OF THE  $(m + 1)$ 'TH EXTRACTED FEATURE AND THAT OF THE FIRST EXTRACTED FEATURE, I.E.,  $\text{Var}(Z_{m+1}) / \text{Var}(Z_1)$ , FOR DIFFERENT NUMBER OF EXTRACTED FEATURES  $M$

| Method                          | Number of Extracted Features |      |      |       |
|---------------------------------|------------------------------|------|------|-------|
|                                 | 20                           | 19   | 17   | 14    |
| GFR with multilinear polynomial | 0.26                         | 0.52 | 1.32 | 2.61  |
| GFR with polynomial             | 0.08                         | 0.09 | 0.11 | 1.03  |
| Kernel PCA                      | 7.95                         | 8.50 | 9.92 | 11.59 |

(a) USPS Dataset

| Method                          | Number of Extracted Features |      |      |      |      |      |
|---------------------------------|------------------------------|------|------|------|------|------|
|                                 | 29                           | 28   | 27   | 25   | 22   | 20   |
| GFR with multilinear polynomial | 0.02                         | 0.12 | 0.24 | 0.49 | 1.22 | 1.82 |
| GFR with polynomial             | 0.01                         | 0.04 | 0.17 | 0.34 | 0.91 | 1.48 |
| Kernel PCA                      | 0.59                         | 0.61 | 0.62 | 0.68 | 0.90 | 0.91 |

(b) MNIST Dataset

| Method                          | Number of Extracted Features |       |       |
|---------------------------------|------------------------------|-------|-------|
|                                 | 14                           | 13    | 12    |
| GFR with multilinear polynomial | 0.45                         | 0.91  | 2.27  |
| GFR with polynomial             | 0.55                         | 0.063 | 0.087 |
| Kernel PCA                      | 15.08                        | 16.15 | 16.62 |

(c) COIL-20 Dataset

| Method                          | Number of Extracted Features |      |       |       |       |       |       |
|---------------------------------|------------------------------|------|-------|-------|-------|-------|-------|
|                                 | 20                           | 19   | 18    | 17    | 16    | 14    | 12    |
| GFR with multilinear polynomial | 0.02                         | 0.04 | 0.10  | 0.21  | 0.41  | 1.03  | 2.06  |
| GFR with polynomial             | 0.01                         | 0.02 | 0.04  | 0.051 | 0.057 | 0.31  | 0.76  |
| Kernel PCA                      | 8.71                         | 8.93 | 10.02 | 10.45 | 11.22 | 12.93 | 14.17 |

(d) Musk Dataset

| Method                          | Number of Extracted Features |      |      |
|---------------------------------|------------------------------|------|------|
|                                 | 9                            | 8    | 5    |
| GFR with multilinear polynomial | 1.87                         | 2.81 | 3.74 |
| GFR with polynomial             | 1.69                         | 2.05 | 2.56 |
| Kernel PCA                      | 0.07                         | 0.10 | 0.21 |

(e) Credit Approval Dataset

on the entire datasets is used to validate the performance of the algorithms. In multi-class classification tasks, we used One-vs-Rest strategy. To implement Kernel PCA, FastICA, LPP, and UMAP, we used KernelPCA, FastICA, LocalityPreservingProjection, and UMAP

TABLE IV  
CLASSIFICATION ACCURACY (%) OF GFR WITH MULTILINEAR POLYNOMIALS OF DEGREE UP TO 4, KERNEL PCA, FASTICA, PCA, AND LPP ON BENCHMARK DATASETS

| Method     | Number of Extracted Features |           |              |              |
|------------|------------------------------|-----------|--------------|--------------|
|            | 20                           | 19        | 17           | 14           |
| GFR        | <b>97.02</b>                 | <b>97</b> | <b>96.86</b> | <b>96.04</b> |
| Kernel PCA | 96.90                        | 96.69     | 96.52        | 95.86        |
| FastICA    | 96.14                        | 95.77     | 95.61        | 95.54        |
| PCA        | 96.18                        | 95.89     | 95.72        | 95.33        |
| LPP        | 93.54                        | 93.48     | 93.29        | 92.32        |

(a) USPS Dataset

| Method     | Number of Extracted Features |              |              |              |              |              |
|------------|------------------------------|--------------|--------------|--------------|--------------|--------------|
|            | 29                           | 28           | 27           | 25           | 22           | 20           |
| GFR        | <b>96.91</b>                 | <b>96.90</b> | <b>96.87</b> | <b>96.79</b> | <b>96.63</b> | <b>96.46</b> |
| Kernel PCA | 93.11                        | 93.02        | 92.87        | 92.72        | 92.69        | 92.58        |
| FastICA    | 95.01                        | 94.98        | 94.94        | 94.85        | 94.52        | 94.35        |
| PCA        | 90.11                        | 90.02        | 89.54        | 89.12        | 88.77        | 88.35        |
| LPP        | 87.13                        | 86.95        | 86.8         | 86.31        | 86.03        | 85.94        |

(b) MNIST Dataset

| Method     | Number of Extracted Features |              |              |
|------------|------------------------------|--------------|--------------|
|            | 14                           | 13           | 12           |
| GFR        | <b>97.99</b>                 | <b>97.92</b> | <b>97.85</b> |
| Kernel PCA | 96.67                        | 96.81        | 96.25        |
| FastICA    | 95.63                        | 95.28        | 92.85        |
| PCA        | 96.04                        | 95.13        | 92.68        |
| LPP        | 90.56                        | 90.35        | 89.03        |

(c) COIL-20 Dataset

| Method     | Number of Extracted Features |              |              |             |              |              |              |
|------------|------------------------------|--------------|--------------|-------------|--------------|--------------|--------------|
|            | 20                           | 19           | 18           | 17          | 16           | 14           | 12           |
| GFR        | 83.32                        | 83.24        | 83.57        | <b>83.1</b> | 82.21        | 82.24        | <b>82.21</b> |
| Kernel PCA | <b>83.97</b>                 | <b>84.31</b> | 83.99        | 82.79       | 81.37        | 79.25        | 78.87        |
| FastICA    | 83.24                        | 82.9         | <b>84.44</b> | 82.65       | <b>83.78</b> | <b>82.49</b> | 82.1         |
| PCA        | 81.04                        | 81.04        | 81.81        | 82.55       | 82.12        | 81.34        | 80.68        |
| LPP        | 82.67                        | 82.69        | 81.49        | 81.49       | 82.57        | 81.78        | 81.1         |

(d) Musk Dataset

| Method     | Number of Extracted Features |              |                        |
|------------|------------------------------|--------------|------------------------|
|            | 9                            | 8            | 5                      |
| GFR        | <b>84.78</b>                 | <b>84.49</b> | 81.59 ( $2^{nd}$ best) |
| Kernel PCA | 81.30                        | 79.13        | 80                     |
| FastICA    | 82.46                        | 82.46        | 81.44                  |
| PCA        | 82.46                        | 82.46        | 81.44                  |
| LPP        | 84.34                        | 84.34        | <b>82.31</b>           |

(e) Credit Approval Dataset

from `sklearn.decomposition`, `lpproj` packages, and `umap-learn`, respectively. In all classification tasks utilizing Kernel PCA, polynomial kernels are employed, whose degree matches that of the function family  $\mathcal{F}$ .

Since autoencoders and UMAP have some hyperparameters, we tuned them to find some rough optimum points, and then changed their values around those points to show their behavior for different values of hyperparameters. Specifically, in autoencoders, we change the number of hidden layers, the number of nodes in hidden layers, and the number of

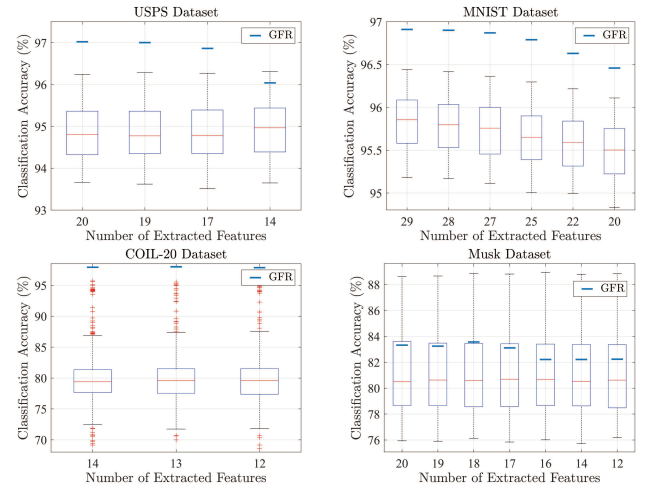


Fig. 3. Classification accuracy (%) of GFR with multilinear polynomials of degree up to 4 versus UMAP on benchmark datasets.

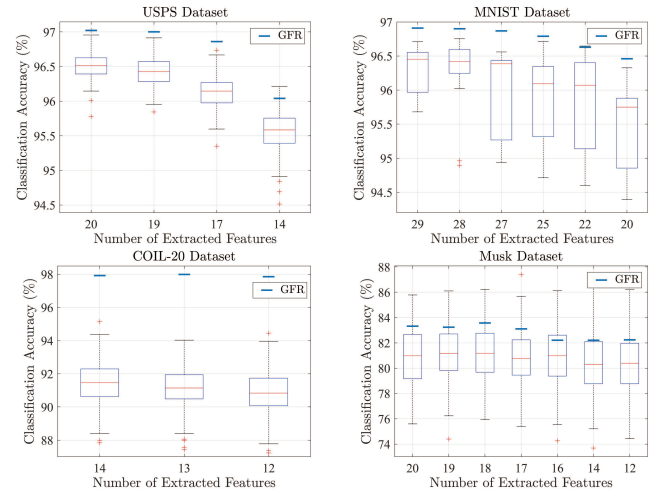


Fig. 4. Classification accuracy (%) of GFR with multilinear polynomials of degree up to 4 versus autoencoders on benchmark datasets.

epochs. For UMAP, we change `min_dist`, which is the effective minimum distance between embedded points, and `n_neighbors`, which is the size of the local neighborhood (in terms of the number of neighboring sample points) used for manifold approximation. For more details about these hyperparameters, we refer readers to [24].

Figures 3–4 compare the classification accuracy of GFR with UMAP and autoencoders, respectively. The numerical results show that GFR outperforms both UMAP and autoencoder in terms of classification accuracy. Note that on each box, the central mark indicates the median, and the bottom and top edges of the box indicate the 25<sup>th</sup> and 75<sup>th</sup> percentiles, respectively. The whiskers extend to the most extreme data points not considered outliers, and the outliers are plotted individually using the “+” marker symbol. Another important result of these experiments is the sensitivity of the autoencoders and UMAP with respect to the hyperparameters, which results in a higher uncertainty of these methods. As we see in these tables and figures, in all the experiments, the

TABLE V  
CLASSIFICATION ACCURACY (%) OF GFR WITH MULTILINEAR POLYNOMIALS OF DEGREE UP TO 2 VERSUS PCA IN SYNTHETIC DATASETS WITH 20 FEATURES

| Method | Dataset Size |              |              |              |
|--------|--------------|--------------|--------------|--------------|
|        | 1000         | 2000         | 5000         | 10000        |
| GFR    | <b>57.03</b> | <b>58.62</b> | <b>61.21</b> | <b>63.93</b> |
| PCA    | 56.43        | 57.41        | 58.43        | 59.4         |

(a) Synthetic Dataset with PTF Labels

| Method | Dataset Size |              |              |              |
|--------|--------------|--------------|--------------|--------------|
|        | 1000         | 2000         | 5000         | 10000        |
| GFR    | <b>87.72</b> | <b>89.96</b> | <b>92.92</b> | <b>95.15</b> |
| PCA    | 84.97        | 85.66        | 86.22        | 86.36        |

(b) Synthetic Dataset with LTF Labels

classification accuracy of GFR with multilinear polynomials of degree up to 4 over USPS, MNIST, and COIL-20 datasets is higher than all other algorithms. The results for the Musk data set demonstrate that none of the GFR, Kernel PCA, FastICA, PCA, and LPP are better than others in most cases, but the classification accuracy of GFR is higher than the average classification accuracy of UMAP and autoencoder. Finally, from the simulation results for the Credit Approval dataset, it is obvious that GFR's performance is better than other algorithms in most cases. Only in cases with lower extracted features, LPP performs better than GFR.

2) *GFR Vs PCA in Synthetic Datasets*: In this subsection, we evaluate GFR's performance in terms of classification accuracy in synthetic datasets. The main goal of these experiments is to show that GFR outperforms PCA in datasets with underlying normal distribution. We generate the synthetic datasets with 20 features, including 10 mutually independent random variables  $X_1, \dots, X_{10}$ , where each  $X_i$  is normally distributed with zero mean and a random variance chosen from  $\text{Unif}(0, 2)$ . For  $i \in \{11, \dots, 20\}$  we chose  $X_i = X_j X_k$  with uniformly random distinct  $j, k \in \{1, \dots, 10\}$ . We applied GFR with  $\mathcal{F}(\mathbf{z})$  being multilinear polynomials of degree up to two, and after extracting features we applied PCA to extract exactly the same number of features obtained by GFR. To evaluate their performance, we use a common method in unsupervised feature extraction [20], [51], [52], where we assign labels to the data points but never use them in the feature extraction procedure. After extracting new features, we use the labels for computing the error of binary classification tasks. To generate labels we used either a polynomial threshold function (PTF)

$$f(X_1, \dots, X_{10}) = \text{sign} \left[ \prod_{1 \leq j \leq 3} \left( b_{0,j} + \sum_{i=1}^{10} b_{i,j} X_i \right) \right],$$

with  $b_{i,j} \sim \text{Unif}(0, 1)$  and mutually independent, or a linear threshold function (LTF)

$$f(X_1, \dots, X_{10}) = \text{sign} \left[ b_0 + \sum_{i=1}^{10} b_i X_i \right],$$

with  $b_i \sim \text{Unif}(0, 1)$  and mutually independent.

We used a support vector machine classifier with a radial basis function as kernel. 5-fold cross-validation on the entire datasets is used to validate the performance of the algorithms.

We repeated the experiments for different dataset sizes, and the simulation results are shown in Table V. As we see in these tables, the classification accuracy of GFR for both LTF and PTF label functions is larger than PCA.

To show the effect of the number of extracted features on the performance of GFR and PCA, we performed another experiment with a dataset dimension of  $d = 60$ , among which 30 are independent, and LTF label functions (Table VI). The result show that GFR retains its advantage over PCA even as the number of extracted features grows.

### C. GCA's Performance

We evaluate GCA on the basis of its success in eliminating all redundant features among the principal components in synthetic datasets. Similar to Section IV, suppose  $X = \sum_{i=1}^d \rho_i Z_i$ . We let  $Z_{\ell_1}, \dots, Z_{\ell_n}$  for  $\ell_1, \dots, \ell_n \in [d]$  be mutually independent non-redundant principal components chosen from  $\mathcal{N}(0, \text{Unif}(0, n))$ , and let  $Z_{\ell_{n+1}}, \dots, Z_{\ell_d}$  be functions of  $Z_{\ell_1}, \dots, Z_{\ell_n}$ , induced by the redundancies as explained shortly. The experiments detailed below were repeated 1000 times, in each experiment GCA was applied, and its output was compared against the non-redundant principal components.

1) *Experiments for Corollary 1*: We define  $t$  0-redundancies using two types of multilinear polynomials up to degree 2 or 3, i.e.,

$$R^{(i)} = Z_{\ell_{i+n}} - k_i Z_{\ell_j} Z_{\ell_k} \text{ or } \\ R^{(i)} = Z_{\ell_{i+n}} - k_i Z_{\ell_j} Z_{\ell_k} Z_{\ell_r},$$

where  $\ell_j, \ell_k, \ell_r$  are chosen at random, and  $k_i$  is chosen<sup>8</sup> so that  $\text{Var}(Z_{\ell_{i+n}})$  is smallest among the variables participating in  $R^{(i)}$ .

We apply GCA with  $(d, n) = (30, 15)$  and  $\mathcal{F}(\mathbf{z})$  being multilinear polynomials of degree up to 2 or 3, according to the degree of the  $R^{(i)}$ 's mentioned above. In our experiments, we vary dataset sizes from 300 to 2000 and in each experiment GCA is applied, and its output is compared against the non-redundant principal components. We repeat the experiments for different dataset sizes 1000 times and the success rate of these experiments, expressed as a percentage, is reported in Table VII for the two different types of redundancies. The results show that if the dataset size is large enough (roughly 1000 in these experiments), GCA can remove all redundant principal components from the data. However, as the dataset size decreases, the approximation of the covariance matrices degrades. This degradation affects GCA's performance, particularly in datasets with redundancies of higher-degree multilinear polynomials.

To show the effect of dataset dimension on GCA's performance, we also generate synthetic datasets with  $d = 50$  features, among them  $n = 25$  are non-redundant principal components. The success rate of GCA in removing redundant principal components are shown in Table VII. It is obvious that as the dimension of datasets increases, more samples are needed to achieve similar performance compared

<sup>8</sup>For example, it is readily seen that  $k_i < \sqrt{\min\{\text{Var}(Z_{\ell_j}), \text{Var}(Z_{\ell_k})\} / (\text{Var}(Z_{\ell_j}) \text{Var}(Z_{\ell_k}))}$  suffices since the  $Z_{\ell_i}$ 's are mutually independent.



TABLE VI

CLASSIFICATION ACCURACY (%) OF GFR WITH MULTILINEAR POLYNOMIALS OF DEGREE UP TO 2 AGAINST PCA IN SYNTHETIC DATASETS WITH LTF LABELS AND 60 FEATURES, AMONG WHICH 30 ARE MUTUALLY INDEPENDENT AND NORMALLY DISTRIBUTED, AND THE REMAINING ONES ARE RANDOMLY CHOSEN MONOMIALS OF DEGREE 2 OF THE 30 INDEPENDENT ONES

| Method | Number of Extracted Features |              |              |              |              |              |              |              |              |              |
|--------|------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|        | 1                            | 2            | 3            | 4            | 5            | 10           | 15           | 20           | 25           | 30           |
| GFR    | <b>59.26</b>                 | <b>61.25</b> | <b>63.20</b> | <b>64.60</b> | <b>66.14</b> | <b>71.64</b> | <b>76.62</b> | <b>79.92</b> | <b>82.42</b> | <b>84.28</b> |
| PCA    | 59.26                        | 61.25        | 62.92        | 64.13        | 65.43        | 70.02        | 74.80        | 77.42        | 79.73        | 80.95        |

TABLE VII

THE SUCCESS RATE (%) OF GCA IN EXTRACTING CORRECT NON-REDUNDANT PRINCIPAL COMPONENTS FROM 1000 EXPERIMENTS CONDUCTED ON SYNTHETIC DATASETS WITH  $d$  FEATURES, WHERE  $n$  FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $Z_{\ell_1}, \dots, Z_{\ell_n}$ , CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0, n))$ , AND THE REMAINING  $d - n$  FEATURES ARE 0-REDUNDANCIES OF MULTILINEAR POLYNOMIALS UP TO DEGREE  $\deg(R^{(i)})$  (COROLLARY 1)

| $(d, n, \deg(R^{(i)}))$ | Dataset Size |      |      |      |      |      |      |      |      |       |       |       |       |       |
|-------------------------|--------------|------|------|------|------|------|------|------|------|-------|-------|-------|-------|-------|
|                         | 300          | 350  | 400  | 450  | 500  | 550  | 600  | 700  | 800  | 900   | 1000  | 1200  | 1500  | 2000  |
| $(30, 15, 2)$           | 65.5         | 76.0 | 83.5 | 87.5 | 93.6 | 96.2 | 97.6 | 99.0 | 99.4 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 |
| $(30, 15, 3)$           | 0            | 0    | 0.8  | 12.1 | 68.6 | 84.5 | 96.5 | 97.9 | 99.2 | 99.8  | 100.0 | 100.0 | 100.0 | 100.0 |
| $(50, 25, 2)$           | 5.1          | 32.5 | 47.9 | 62.1 | 72.7 | 75.6 | 82.3 | 92.0 | 95.7 | 98.7  | 99.0  | 99.9  | 100.0 | 100.0 |

TABLE VIII

THE SUCCESS RATE (%) OF GCA IN EXTRACTING CORRECT NON-REDUNDANT PRINCIPAL COMPONENTS FROM 1000 EXPERIMENTS CONDUCTED ON SYNTHETIC DATASETS WITH  $d$  FEATURES, WHERE  $n$  FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $Z_{\ell_1}, \dots, Z_{\ell_n}$ , CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0, n))$ , AND THE REMAINING  $d - n$  FEATURES ARE  $\epsilon$ -REDUNDANCIES (STOCHASTIC NOISE) OF MULTILINEAR POLYNOMIALS UP TO DEGREE  $\deg(R^{(i)})$  (COROLLARY 2)

| $(d, n, \deg(R^{(i)}))$ | Dataset Size |      |      |      |      |      |      |      |      |      |      |      |      |      |
|-------------------------|--------------|------|------|------|------|------|------|------|------|------|------|------|------|------|
|                         | 300          | 350  | 400  | 450  | 500  | 550  | 600  | 700  | 800  | 900  | 1000 | 1200 | 1500 | 2000 |
| $(30, 15, 2)$           | 63           | 68.9 | 75   | 81.5 | 83.2 | 85.9 | 93.9 | 96.8 | 98.2 | 98.3 | 98.7 | 98.8 | 99.9 | 100  |
| $(30, 15, 3)$           | 0            | 0    | 1.2  | 25.3 | 74.3 | 83.7 | 85.1 | 88.2 | 89.1 | 91.3 | 94   | 96.9 | 99   | 100  |
| $(50, 25, 2)$           | 0.5          | 25.5 | 44.2 | 48.6 | 56.3 | 60.1 | 65.8 | 75.1 | 81.2 | 83.7 | 90.5 | 95.3 | 100  | 100  |

TABLE IX

THE SUCCESS RATE (%) OF GCA IN EXTRACTING CORRECT NON-REDUNDANT PRINCIPAL COMPONENTS FROM 1000 EXPERIMENTS CONDUCTED ON SYNTHETIC DATASETS WITH  $d$  FEATURES, WHERE  $n$  FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $Z_{\ell_1}, \dots, Z_{\ell_n}$ , CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0, n))$ , AND THE REMAINING  $d - n$  FEATURES ARE  $\epsilon$ -REDUNDANCIES (DETERMINISTIC NOISE) OF MULTILINEAR POLYNOMIALS UP TO DEGREE  $\deg(R^{(i)})$  (COROLLARY 3)

| $(d, n, \deg(R^{(i)}))$ | Dataset Size |      |      |      |      |      |      |      |      |      |      |      |      |      |
|-------------------------|--------------|------|------|------|------|------|------|------|------|------|------|------|------|------|
|                         | 300          | 350  | 400  | 450  | 500  | 550  | 600  | 700  | 800  | 900  | 1000 | 1200 | 1500 | 2000 |
| $(30, 15, 4)$           | 0.7          | 28.7 | 47.7 | 50.2 | 59.6 | 63.7 | 70.1 | 73.2 | 82.2 | 87.9 | 91.3 | 95.2 | 97.8 | 100  |
| $(30, 15, 5)$           | 0            | 0    | 0    | 11.6 | 37.4 | 45.3 | 60.2 | 62.1 | 74.5 | 79.3 | 89.1 | 90.1 | 93.3 | 99.3 |
| $(50, 25, 4)$           | 0            | 0    | 0.1  | 13.4 | 42.1 | 46.7 | 63.1 | 67.2 | 78.9 | 81.1 | 90   | 90.1 | 95.2 | 99.5 |

to lower-dimensional datasets. We mention that even though we consider 0-redundancies in these experiments, due to numerical errors, the parameter  $\epsilon$  in Line 5 of Algorithm 3 is chosen as a small constant  $\epsilon = 10^{-4}$ .

2) *Experiments for Corollary 2:* We repeat similar experiments to Section VIII-C.1 with the addition of noise, i.e.,

$$R^{(i)} = Z_{\ell_{i+n}} - k_i Z_{\ell_j} Z_{\ell_k} + \delta_i \text{ or}$$

$$R^{(i)} = Z_{\ell_{i+n}} - k_i Z_{\ell_j} Z_{\ell_k} Z_{\ell_r} + \delta_i,$$

with mutually independent  $\delta_i \sim \mathcal{N}(0, 0.1)$ . Again Similar to section VIII-C.1, we apply GCA with  $\mathcal{F}(\mathbf{z})$  being multilinear polynomials of degree up to 2 or 3. The parameter  $\epsilon$  in Line 5 of Algorithm 3 is chosen as the empirical variance of the added noise  $\delta_i$  in each experiment. The results, which are summarized in Table VIII, demonstrate resilience of GCA to stochastic noise.

3) *Experiments for Corollary 3:* We repeat experiments similar to Section VIII-C.1, with an added mismatch between the degree of the 0-redundancies and the degree of the polynomials in  $\mathcal{F}(\mathbf{z})$ . That is, we take

$$R^{(i)} = Z_{\ell_{i+n}} - k_i Z_{\ell_j} Z_{\ell_k} Z_{\ell_r} Z_{\ell_a}, \text{ or}$$

$$R^{(i)} = Z_{\ell_{i+n}} - k_i Z_{\ell_j} Z_{\ell_k} Z_{\ell_r} Z_{\ell_a} Z_{\ell_b},$$

but apply GCA with  $\mathcal{F}(\mathbf{z})$  containing multilinear polynomials only up to degree three. The  $\epsilon$  parameter in GCA is chosen as the maximum value of

$$\mathbb{E}[(Z_{\ell_{i+n}} - \text{proj}_{\mathcal{F}(Z_{\ell_j}, Z_{\ell_k}, Z_{\ell_r}, Z_{\ell_a})} Z_{\ell_{i+n}})^2], \text{ or}$$

$$\mathbb{E}[(Z_{\ell_{i+n}} - \text{proj}_{\mathcal{F}(Z_{\ell_j}, Z_{\ell_k}, Z_{\ell_r}, Z_{\ell_a}, Z_{\ell_b})} Z_{\ell_{i+n}})^2],$$

TABLE X

NUMBER OF SELECTED FEATURES BY GFS WITH MULTILINEAR POLYNOMIALS ON BENCHMARK DATASETS FOR DIFFERENT VALUES OF THE THRESHOLD,  $\epsilon^2$

| Degree of Polynomial | $\epsilon^2$ |      |      |      |      |      |
|----------------------|--------------|------|------|------|------|------|
|                      | 0.03         | 0.04 | 0.05 | 0.06 | 0.07 | 0.08 |
| 1                    | 85           | 60   | 48   | 41   | 36   | 29   |
| 2                    | 39           | 31   | 25   | 21   | 18   | 16   |
| 3                    | 25           | 20   | 18   | 16   | 15   | 12   |
| 4                    | 24           | 17   | 14   | 13   | 12   | 11   |

(a) USPS Dataset

| Degree of Polynomial | $\epsilon^2$ |        |       |        |      |
|----------------------|--------------|--------|-------|--------|------|
|                      | 0.01         | 0.0125 | 0.015 | 0.0175 | 0.02 |
| 1                    | 98           | 76     | 60    | 50     | 40   |
| 2                    | 36           | 21     | 17    | 15     | 14   |
| 3                    | 36           | 20     | 13    | 11     | 9    |
| 4                    | 31           | 19     | 13    | 10     | 9    |

(b) COIL-20 Dataset

| Degree of Polynomial | $\epsilon^2$ |     |
|----------------------|--------------|-----|
|                      | 0.1          | 0.4 |
| 1                    | 14           | 13  |
| 2                    | 13           | 12  |
| 3                    | 13           | 12  |
| 4                    | 13           | 12  |

(c) Credit Approval Dataset

respectively. The results, which are summarized in Table IX, demonstrate resilience of GCA to so-called *deterministic* noise, i.e., the inability of  $\mathcal{F}(\mathbf{z})$  to represent the redundancies.

#### D. The Number of Selected Features Using GFS

To validate the performance of GFS, we begin by demonstrating that selecting higher degree multilinear polynomials as  $\mathcal{F}(\mathbf{z})$  leads to a significant reduction in maximum variance among the entries of  $d_j(X)$  in each iteration (i.e.,  $\max\{\text{Var}[d_j(X)_i]\}_{i=1}^d$ ). Consequently, GFS effectively detects and removes more redundant features from the datasets as the complexity of  $\mathcal{F}(\mathbf{z})$  increases. To support this claim, we apply GFS with  $\mathcal{F}(\mathbf{z})$  being multilinear polynomials of degrees at most 1, at most 2, at most 3 and at most 4, to the benchmark datasets in Table I. The number of selected features for different values of the threshold  $\epsilon^2$  and degree of the polynomials are shown in Tables X. As demonstrated in the experimental results, by increasing the degree of multilinear polynomials, GFS selects fewer features given the same threshold  $\epsilon^2$ .

#### E. Classification Accuracy of GFS

We turn to validate the performance of GFS for classification tasks on the benchmark datasets in Table I. We applied GFS with multilinear polynomial of degree up to 4 against several well-known feature selection algorithms: Multi-Cluster Feature Selection (MCFS) [31], Nonnegative Discriminative Feature Selection (NDFS) [33], Unsupervised Discriminative Feature Selection (UDFS) [34], Laplacian Score (LS) [26], Trace Ratio (TR) [30], and Fisher Score (FS) [28].

TABLE XI

CLASSIFICATION ACCURACY (%) OF GFS WITH  $\mathcal{F}(\mathbf{z})$  MULTILINEAR POLYNOMIALS OF DEGREE UP TO 4 VERSUS MCFS [31], NDFS [33], UDFS [34], LS [26], TR [30], FS [28] ON BENCHMARK DATASETS

| Method | Number of Selected Features |              |              |              |              |              |
|--------|-----------------------------|--------------|--------------|--------------|--------------|--------------|
|        | 24                          | 17           | 14           | 13           | 12           | 11           |
| GFS    | <b>92.62</b>                | <b>89.27</b> | <b>85.10</b> | <b>83.73</b> | <b>82.25</b> | <b>79.60</b> |
| MCFS   | 90.99                       | 87.37        | 84.39        | 82.81        | 79.76        | 75.13        |
| NDFS   | 89.48                       | 82.84        | 78.38        | 73.98        | 72.92        | 69.98        |
| UDFS   | 86.37                       | 78.81        | 76.56        | 75.71        | 73.06        | 69.03        |
| LS     | 85.21                       | 80.94        | 76.59        | 75.59        | 74.12        | 73.36        |
| TR     | 83.87                       | 78.48        | 74.52        | 72.10        | 71.81        | 68.71        |
| FS     | 83.87                       | 78.93        | 74.39        | 74.02        | 71.81        | 68.71        |

(a) USPS Dataset

| Method | Number of Selected Features |              |              |              |              |
|--------|-----------------------------|--------------|--------------|--------------|--------------|
|        | 31                          | 19           | 13           | 10           | 9            |
| GFS    | <b>90.76</b>                | <b>87.08</b> | <b>84.72</b> | <b>81.18</b> | <b>78.19</b> |
| MCFS   | 79.03                       | 77.57        | 67.57        | 61.18        | 59.93        |
| NDFS   | 87.78                       | 83.89        | 81.94        | 77.64        | 75.76        |
| UDFS   | 69.44                       | 63.47        | 58.75        | 53.82        | 52.36        |
| LS     | 68.47                       | 65.28        | 57.08        | 49.86        | 48.40        |
| TR     | 66.52                       | 57.85        | 43.75        | 41.25        | 39.10        |
| FS     | 59.23                       | 56.32        | 43.02        | 41.20        | 38.90        |

(b) COIL-20 Dataset

| Method | Number of Selected Features |                        |
|--------|-----------------------------|------------------------|
|        | 13                          | 12                     |
| GFS    | 84.20 ( $2^{nd}$ best)      | 83.91 ( $2^{nd}$ best) |
| MCFS   | <b>84.34</b>                | 83.62                  |
| NDFS   | 83.62                       | 83.47                  |
| UDFS   | 75.79                       | 75.22                  |
| LS     | 84.05                       | <b>84.20</b>           |
| TR     | 75.79                       | 75.51                  |
| FS     | 75.80                       | 75.51                  |

(c) Credit Approval Dataset

We used a support vector machine classifier with radial basis function as kernel, and 5-fold cross-validation on the entire dataset to validate the performance of the algorithms. To implement MCFS, NDFS, UDFS, LS, TR, and FS, we used `skfeature-chappers` package. The results, given in Table XI, demonstrate superior performance in terms of classification accuracy in the benchmark datasets in comparison to other algorithms.

#### F. Comparison Between GFS and UDFS [5]

As mentioned earlier (and shown in detail in Appendix C), GFS generalizes the Unsupervised Fourier Feature Selection (UFFS) algorithm of [5] at significantly reduced complexity. Specifically, UDFS arises as a special case of GFS when  $\mathcal{F}(\mathbf{z})$  is chosen as multilinear polynomials up to degree  $d$ . The reduction in complexity is due to our step-by-step orthogonalization process (Algorithm 1), in contrast to UDFS which orthogonalizes all multilinear polynomials. We corroborate GFS's superiority over UDFS in terms of running time, the ability to capture redundant features, and classification accuracy over synthetic datasets. The synthetic datasets we use in these experiments include 15 independent normally distributed random

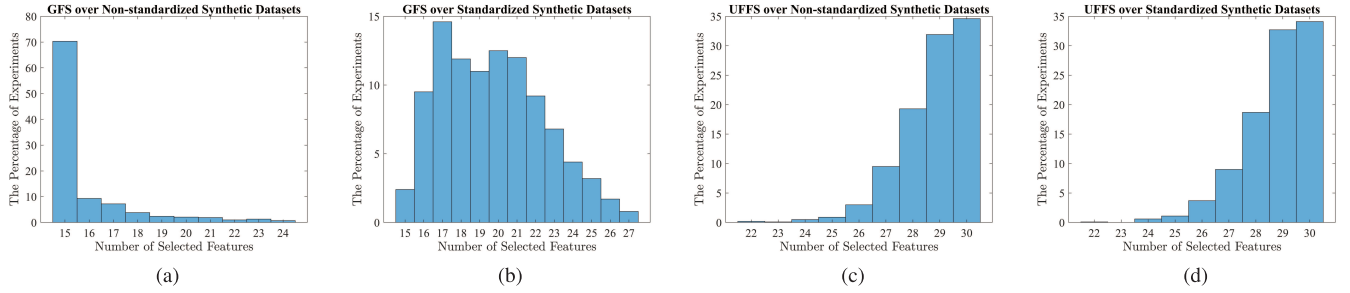


Fig. 5. The percentage of experiments, out of 1000, corresponding to the number of non-redundant features selected by GFS and UFFS over synthetic datasets with 30 features, where 15 features are non-redundant (mutually independent)  $X_{\ell_1}, \dots, X_{\ell_{15}}$  and the remaining 15 features are multilinear polynomials of these non-redundant features up to degree 3. (a) GFS over non-standardized datasets; (b) GFS over standardized datasets; (c) UFFS over non-standardized datasets; (d) UFFS over standardized datasets.

TABLE XII

RUNNING TIME RATIO OF GFS VS. UFFS, I.E.  $t_{\text{UFFS}}/t_{\text{GFS}}$ , FOR NON-STANDARDIZED AND STANDARDIZED SYNTHETIC DATASETS WITH 30 FEATURES, WHERE 15 FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $X_{\ell_1}, \dots, X_{\ell_{15}}$  CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0.5, 1))$ , AND THE REMAINING 15 FEATURES ARE MULTILINEAR POLYNOMIALS OF THESE NON-REDUNDANT FEATURES UP TO DEGREE 3

|                                     | GFS over non-standardized datasets | GFS over standardized datasets |
|-------------------------------------|------------------------------------|--------------------------------|
| UFFS over non-standardized datasets | 20.92                              | 6.99                           |
| UFFS over standardized datasets     | 22.11                              | 7.39                           |

variables  $X_{\ell_1}, \dots, X_{\ell_{15}} \sim \mathcal{N}(0, \text{Unif}(0.5, 1))$ <sup>9</sup> and 15 redundant features  $X_{\ell_{16}}, \dots, X_{\ell_{30}}$ , which are randomly and uniformly taken from  $\{X_{\ell_i} X_{\ell_j}\}_{\text{distinct } i, j \in [15]} \cup \{X_{\ell_i} X_{\ell_j} X_{\ell_k}\}_{\text{distinct } i, j, k \in [15]}$ ; the integers  $\ell_1, \dots, \ell_{30}$  are a random permutation of  $1, \dots, 30$ . We also consider 1000 sample points in each dataset. In all the following comparisons between GFS and UFFS, we choose multilinear polynomials of degree up to three as the function family  $\mathcal{F}(\mathbf{z})$  in GFS, and in UFFS we applied the heuristic proposed in [5] to orthogonalize multilinear polynomials only up to degree three<sup>10</sup>. In the experimental section of [5], the features are standardized<sup>11</sup> before applying UFFS. We apply both GFS and UFFS to the synthetic datasets and their standardized versions.

1) *Running Time:* In Table XII, the running time ratio of UFFS and GFS over synthetic datasets is compared for the non-standardized datasets and their standardized versions. We repeat the experiments for 1000 different synthetic datasets

<sup>9</sup>We use the lower bound 0.5 in the variance of independent  $X_{\ell_i}$ 's to make sure their variance is not very small and not considered as redundant in our analysis.

<sup>10</sup>Technically, UFFS requires one to orthogonalize *all* multilinear polynomials up to degree 30, and as a heuristic, [5] proposed to orthogonalize only up to a lower degree. Clearly, orthogonalizing all  $2^{30}$  multilinear polynomials to execute UFFS in its entirety would result in infeasible run times.

<sup>11</sup>I.e., the mean vector is subtracted from all points of the data, and then every feature is divided by its variance. This results in data which has mean zero, and every feature has variance 1.

and report the average values in Table XII. The results show that on average GFS is about 22 times faster than UFFS for the chosen parameters.

2) *Capturing Redundant Features:* In this section, we demonstrate that GFS can capture nonlinear redundancies better than UFFS. We perform 1000 experiments over synthetic datasets and in each of them we apply GFS and UFFS with the threshold  $\epsilon^2 = 0.01$  in Line 6 of Algorithm 4. Figure 5 shows the histograms of the number of non-redundant features selected by GFS and UFFS (out of 1000 experiments) on both non-standardized and standardized synthetic datasets. The results indicate that GFS selects fewer non-redundant features than UFFS on both non-standardized and standardized datasets, suggesting that GFS is more effective at removing redundant features compared to UFFS. In [5], the authors apply UFFS three times and report the results after the third round. For a fair comparison, we apply both GFS and UFFS only once. Another insight from these simulations is the sensitivity of UFFS to the order of features. To validate this claim, we set the non-redundant features as the first 15 features in the datasets, i.e.,  $X_{\ell_1} = X_1, \dots, X_{\ell_{15}} = X_{15}$ , and repeated the experiments 1000 times. In all cases, UFFS successfully selected the non-redundant features and removed the redundant ones. However, in real datasets, the order is unknown. GFS, on the other hand, is not sensitive to the order of features because it uses a step-by-step orthogonalization process. Moreover, as seen in Figures 5c–5d, standardization does not affect UFFS's performance, as it starts from a random feature and orthogonalizes all multilinear polynomials up to degree 3. However, in Figures 5a–5b it is evident that standardization affects GFS's performance. GFS begins with the highest variant feature and uses step-by-step orthogonalization to select the most informative features. When the features are standardized, they are supposed to have unit variances. However, due to computational errors, one of them may end up with a higher variance, even if it is initially one of the features with the smallest variance. This discrepancy is a result of the standardization process. It impacts the rest of the orthogonalization process, causing some redundant features to be selected as non-redundant. Figures 5b–5c show that even in standardized datasets, GFS outperforms UFFS in terms of capturing nonlinear redundant features.

TABLE XIII

CLASSIFICATION ACCURACY (%) OF GFS VS. UFFS OVER SYNTHETIC DATASETS WITH 30 FEATURES, WHERE 15 FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $X_{\ell_1}, \dots, X_{\ell_{15}}$  AND THE REMAINING 15 FEATURES ARE MULTILINEAR POLYNOMIALS UP TO DEGREE 3

| Method | Labels |       |
|--------|--------|-------|
|        | $f_1$  | $f_2$ |
| GFS    | 74.33  | 61.08 |
| UFFS   | 67.22  | 57.26 |

3) *Classification Accuracy*: In this section, we compare GFS and UFFS in terms of classification accuracy in synthetic datasets. The main goal of these experiments is to show that GFS outperforms UFFS in datasets with nonlinear redundancies. To evaluate their performance, we use a common method in unsupervised feature selection, where we assign labels to the data points but never use them in the feature selection procedure. After selecting non-redundant features, we use the labels for computing the error of binary classification tasks. To generate labels, we use the following polynomial threshold functions

$$f_1(X_{\ell_1}, \dots, X_{\ell_{15}}) = \text{sign} \left[ \prod_{1 \leq j \leq 2} \left( b_{0,j} + \sum_{i=1}^{15} b_{i,j} X_{\ell_i} \right) \right]$$

or

$$f_2(X_{\ell_1}, \dots, X_{\ell_{15}}) = \text{sign} \left[ \prod_{1 \leq j \leq 3} \left( b_{0,j} + \sum_{i=1}^{15} b_{i,j} X_{\ell_i} \right) \right],$$

where  $b_{i,j} \sim \text{Unif}(0,1)$  and mutually independent. In all the experiments, GFS selects fewer features than UFFS. To have a fair comparison, we select an identical number of features using GFS and UFFS. Since UFFS does not rank the features, we complete the orthogonalization process for all features and then select the ones whose orthogonalized versions exhibit the greatest variance. We apply a support vector machine classifier with a radial basis function as kernel. 5-fold cross-validation on the entire datasets is used and the results are shown in Table XIII which show that GFS outperforms UFFS in terms of classification accuracy.

### G. GFA's Performance

We evaluate GFA on the basis of its success in removing all redundant features in synthetic datasets. Similar to Section VI, suppose  $X = (X_1, \dots, X_d)^\top$ . We let  $X_{\ell_1}, \dots, X_{\ell_n}$  for  $\ell_1, \dots, \ell_n \in [d]$  be mutually independent non-redundant features chosen from  $\mathcal{N}(0, \text{Unif}(0,n))$ , and let  $X_{\ell_{n+1}}, \dots, X_{\ell_d}$  be functions of  $X_{\ell_1}, \dots, X_{\ell_n}$ , induced by the redundancies as explained shortly. The experiments detailed below were repeated 1000 times, in each experiment GFA was applied, and its output was compared against the non-redundant features.

1) *Experiments for Corollary 1*: Similar to section VIII-C.1, We define  $t$  0-redundancies using two types of multilinear polynomials up to degree 2 or 3, i.e.,

$$\begin{aligned} R^{(i)} &= X_{\ell_{i+n}} - k_i X_{\ell_j} X_{\ell_k} \text{ or} \\ R^{(i)} &= X_{\ell_{i+n}} - k_i X_{\ell_j} X_{\ell_k} X_{\ell_r}, \end{aligned}$$

where  $\ell_j, \ell_k, \ell_r$  are chosen at random, and  $k_i$  is chosen<sup>12</sup> so that  $\text{Var}(X_{\ell_{i+n}})$  is smallest among the variables participating in  $R^{(i)}$ .

We apply GFA with  $(d, n) = (30, 15)$  and  $\mathcal{F}(\mathbf{z})$  being multilinear polynomials of degree up to 2 or 3, according to the degree of the  $R^{(i)}$ 's mentioned above. In our experiments, we vary dataset sizes from 300 to 2000 and in each experiment GFA is applied, and its output is compared against the non-redundant features. We repeat the experiments for different dataset sizes 1000 times and the success rate of these experiments, expressed as a percentage, is reported in Table XIV for the two different types of redundancies. The results show that if the dataset size is large enough (roughly 800 in these experiments), GFA can remove all redundant features from the data. However, as the dataset size decreases, the approximation of the covariance matrices degrades. This degradation affects GFA's performance, particularly in datasets with redundancies of higher-degree multilinear polynomials.

To show the effect of dataset dimension on GFA's performance, we also generate synthetic datasets with  $d = 50$  features, among them  $n = 25$  are non-redundant features. The success rate of GFA in removing redundant features is shown in Table XIV. It is obvious that as the dimension of datasets increases, more samples are needed to achieve similar performance compared to lower-dimensional datasets.

2) *Experiments for Corollary 2*: We repeat similar experiments to Section VIII-G.1 with the addition of noise, i.e.,

$$\begin{aligned} R^{(i)} &= X_{\ell_{i+n}} - k_i X_{\ell_j} X_{\ell_k} + \delta_i \text{ or} \\ R^{(i)} &= X_{\ell_{i+n}} - k_i X_{\ell_j} X_{\ell_k} X_{\ell_r} + \delta_i, \end{aligned}$$

with mutually independent  $\delta_i \sim \mathcal{N}(0, 0.1)$ . Again Similar to section VIII-G.1, we apply GFA with  $\mathcal{F}(\mathbf{z})$  being multilinear polynomials of degree up to 2 or 3. The parameter  $\epsilon$  in Algorithm 5 is chosen as the empirical variance of the added noise  $\delta_i$  in each experiment. The results, which are summarized in Table XV, demonstrate resilience of GFA to stochastic noise.

3) *Experiments for Corollary 3*: We repeat experiments similar to Section VIII-G.1, with an added mismatch between the degree of the 0-redundancies and the degree of the polynomials in  $\mathcal{F}(\mathbf{z})$ . That is, we take

$$\begin{aligned} R^{(i)} &= X_{\ell_{i+n}} - k_i X_{\ell_j} X_{\ell_k} X_{\ell_r} X_{\ell_a}, \text{ or} \\ R^{(i)} &= X_{\ell_{i+n}} - k_i X_{\ell_j} X_{\ell_k} X_{\ell_r} X_{\ell_a} X_{\ell_b}, \end{aligned}$$

but apply GFA with  $\mathcal{F}(\mathbf{z})$  containing multilinear polynomials only up to degree three. The  $\epsilon$  parameter in GFA is chosen as the maximum value of

$$\begin{aligned} &\mathbb{E}[(X_{\ell_{i+n}} - \text{proj}_{\mathcal{F}(X_{\ell_j}, X_{\ell_k}, X_{\ell_r}, X_{\ell_a})} X_{\ell_{i+n}})^2], \text{ or} \\ &\mathbb{E}[(X_{\ell_{i+n}} - \text{proj}_{\mathcal{F}(X_{\ell_j}, X_{\ell_k}, X_{\ell_r}, X_{\ell_a}, X_{\ell_b})} X_{\ell_{i+n}})^2], \end{aligned}$$

respectively. The results, which are summarized in Table XVI, demonstrate resilience of GFA to so-called *deterministic* noise, i.e., the inability of  $\mathcal{F}(\mathbf{z})$  to represent the redundancies.

<sup>12</sup>For example, it is readily seen that  $k_i < \sqrt{\min\{\text{Var}(X_{\ell_j}), \text{Var}(X_{\ell_k})\} / (\text{Var}(X_{\ell_j}) \text{Var}(X_{\ell_k}))}$  suffices since the  $X_{\ell_i}$ 's are mutually independent.



TABLE XIV

THE SUCCESS RATE (%) OF GFA IN SELECTING CORRECT NON-REDUNDANT FEATURES FROM 1000 EXPERIMENTS CONDUCTED ON SYNTHETIC DATASETS WITH  $d$  FEATURES, WHERE  $n$  FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $X_{\ell_1}, \dots, X_{\ell_n}$ , CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0, n))$ , AND THE REMAINING  $d - n$  FEATURES ARE 0-REDUNDANCIES OF MULTILINEAR POLYNOMIALS UP TO DEGREE  $\deg(R^{(i)})$  (COROLLARY 1)

| $(d, n, \deg(R^{(i)}))$ | Dataset Size |      |      |      |      |      |      |      |     |     |      |      |      |      |  |
|-------------------------|--------------|------|------|------|------|------|------|------|-----|-----|------|------|------|------|--|
|                         | 300          | 350  | 400  | 450  | 500  | 550  | 600  | 700  | 800 | 900 | 1000 | 1200 | 1500 | 2000 |  |
| $(30, 15, 2)$           | 98.9         | 99.8 | 99.9 | 100  | 100  | 100  | 100  | 100  | 100 | 100 | 100  | 100  | 100  | 100  |  |
| $(30, 15, 3)$           | 0            | 0.5  | 23.6 | 58.5 | 90.1 | 100  | 100  | 100  | 100 | 100 | 100  | 100  | 100  | 100  |  |
| $(50, 25, 2)$           | 12.3         | 57   | 76.2 | 88.9 | 96.1 | 97.9 | 99.7 | 99.8 | 100 | 100 | 100  | 100  | 100  | 100  |  |

TABLE XV

THE SUCCESS RATE (%) OF GFA IN SELECTING CORRECT NON-REDUNDANT FEATURES FROM 1000 EXPERIMENTS CONDUCTED ON SYNTHETIC DATASETS WITH  $d$  FEATURES, WHERE  $n$  FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $X_{\ell_1}, \dots, X_{\ell_n}$ , CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0, n))$ , AND THE REMAINING  $d - n$  FEATURES ARE  $\epsilon$ -REDUNDANCIES (STOCHASTIC NOISE) OF MULTILINEAR POLYNOMIALS UP TO DEGREE  $\deg(R^{(i)})$  (COROLLARY 2)

| $(d, n, \deg(R^{(i)}))$ | Dataset Size |      |      |      |      |      |      |      |      |      |      |      |      |      |  |
|-------------------------|--------------|------|------|------|------|------|------|------|------|------|------|------|------|------|--|
|                         | 300          | 350  | 400  | 450  | 500  | 550  | 600  | 700  | 800  | 900  | 1000 | 1200 | 1500 | 2000 |  |
| $(30, 15, 2)$           | 82.9         | 86.6 | 88.6 | 90.8 | 92.8 | 94   | 94.4 | 96.6 | 97   | 97.9 | 98.8 | 98.8 | 99.5 | 99.9 |  |
| $(30, 15, 3)$           | 1.24         | 2.5  | 27.3 | 60.1 | 72.1 | 79.3 | 80.8 | 81.7 | 82.3 | 83.8 | 90.9 | 96   | 98.8 | 99.8 |  |
| $(50, 25, 2)$           | 8.7          | 43.2 | 52.8 | 63.8 | 72.2 | 77.1 | 82.2 | 86.9 | 91.8 | 93.9 | 94.9 | 96.2 | 99.1 | 99.8 |  |

TABLE XVI

THE SUCCESS RATE (%) OF GFA IN SELECTING CORRECT NON-REDUNDANT FEATURES FROM 1000 EXPERIMENTS CONDUCTED ON SYNTHETIC DATASETS WITH  $d$  FEATURES, WHERE  $n$  FEATURES ARE NON-REDUNDANT (MUTUALLY INDEPENDENT)  $X_{\ell_1}, \dots, X_{\ell_n}$ , CHOSEN FROM  $\mathcal{N}(0, \text{Unif}(0, n))$ , AND THE REMAINING  $d - n$  FEATURES ARE  $\epsilon$ -REDUNDANCIES (DETERMINISTIC NOISE) OF MULTILINEAR POLYNOMIALS UP TO DEGREE  $\deg(R^{(i)})$  (COROLLARY 3)

| $(d, n, \deg(R^{(i)}))$ | Dataset Size |      |      |      |      |      |      |      |      |      |      |      |      |      |  |
|-------------------------|--------------|------|------|------|------|------|------|------|------|------|------|------|------|------|--|
|                         | 300          | 350  | 400  | 450  | 500  | 550  | 600  | 700  | 800  | 900  | 1000 | 1200 | 1500 | 2000 |  |
| (30, 15, 4)             | 20.3         | 38.4 | 62.1 | 55.8 | 64.7 | 69.1 | 81.2 | 83.2 | 86.2 | 91.1 | 94.7 | 97.2 | 99.4 | 100  |  |
| (30, 15, 5)             | 0            | 3.4  | 13.5 | 22.4 | 37.4 | 49.3 | 62.5 | 63.4 | 78.9 | 83.4 | 89.9 | 91.7 | 94.3 | 98.9 |  |
| (50, 25, 4)             | 10.1         | 10.2 | 11.2 | 32.3 | 48.3 | 51.1 | 67.4 | 72.1 | 82.2 | 84.3 | 92.3 | 93.1 | 95.6 | 99.7 |  |

## IX. DISCUSSION AND FUTURE WORK

The algorithms presented herein rely on an orthogonalization process of an arbitrary function family  $\mathcal{F}(\mathbf{z})$ , whose variables are substituted one-by-one with functions of the data random variable  $X$ . While we pose few constraints on the choice of  $\mathcal{F}(\mathbf{z})$ , it is evident that informative dimension reduction strongly relies on choosing a “proper” function family for the data distribution.

The choice of  $\mathcal{F}(\mathbf{z})$  should correspond to a prior belief that the redundancies can be described, or can be well-approximated by, the functions in  $\text{span } \mathcal{F}(\mathbf{z})$ . Several clear interpretations of the required correspondence between  $\mathcal{F}(\mathbf{z})$  and  $X$  are given in Section IV (also Section VI).

More broadly, the choice of  $\mathcal{F}(\mathbf{z})$  is tightly connected to Fourier analysis, an area which studies representation of functions over orthogonal bases (w.r.t some underlying distribution). For example, it is known [53] that under the uniform distribution over the discrete hypercube  $\{\pm 1\}^t$ , the set of multilinear polynomials in  $t$  variables is an orthonormal basis to the set of functions of the form  $f : \{\pm 1\}^t \rightarrow \mathbb{R}$ ; cases where the distribution is not uniform can be treated using the GS process. Continuous variants exist (e.g., Hermite polynomials for Gaussian distribution), and the interested reader is referred

to [54]. Therefore, it is reasonable to assume that letting  $\mathcal{F}(\mathbf{z})$  be low-degree polynomials is a safe choice, and this is very clearly demonstrated in our experimental section. However, one might also suggest using Fourier basis, which is left for future work.

We once again emphasize the different approach taken in GFR and GFS vs. the approach taken in GCA and GFA. In GFR and GFS, we extract/select variance maximizers of the *reduced* data random variable  $d_j(X)$ , whereas in GCA and GFA we merely use the  $d_j(X)$  to identify which features/components need to be subtracted from  $X$  itself. The latter enables a finer understanding of the exact structure of data distributions on which it is effective, and its correspondence with  $\mathcal{F}(\mathbf{z})$ . For the former we are not able to provide such analyses, but we are able to provide entropy reduction guarantees.

For future work we propose to study more closely which redundancy structures can be accommodated using GFR, GFS, and potentially, waive the requirement for discrete alphabets in their entropy reduction guarantees. Second, we implicitly assumed that expectations can be computed exactly, which is clearly not the case in practice. Nevertheless, our extensive experiments show that this does not pose a significant barrier on many benchmark and synthetic datasets. However, it is still imperative to study convergence results, i.e., the required

dataset sizes for which empirical means are sufficiently close to the true expectations. Although GCA is proposed to eliminate redundancy among the principal components, it can be applied using any orthonormal basis other than the principal directions. In our future work, we propose to study the extension of GCA to other linear and nonlinear (kernel-based) feature extraction methods.

#### APPENDIX A PROOF OF THEOREM 4

Similar to the Proof of Theorem 1, we begin by showing that the variance of the data in all features which are not selected in smaller than  $\epsilon^2$ . To this end, let  $Z^\perp \triangleq (Z_{m+1}, \dots, Z_d)$  be the features of  $X$  that were *not* selected by GFS, in any arbitrary order. Also, suppose we complete the orthogonalization process which GFS started and obtain  $\hat{\mathcal{F}}(Z_1, \dots, Z_d)$ , as well as the respective variance vector  $\sigma_{m+1}, \dots, \sigma_d$ .

*Lemma 3:* For every  $i \in \{m+1, \dots, d\}$  we have  $\|\sigma_i\|_\infty \leq \epsilon^2$ .

The following technical statement is required for the subsequent proof of Lemma 3. For brevity we denote  $\hat{\mathcal{F}}_j \triangleq \hat{\mathcal{F}}(Z_1, \dots, Z_j)$  for every  $j \in [d]$ .

*Lemma 4:* In GFS, we have the following.

(a) For  $i < j$  in  $[d]$  we have

$$\sigma_j = \sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX]^{\otimes 2}.$$

(b) The integers  $s_1, \dots, s_m$  are distinct.

*Proof of Lemma 4.(a):* It is readily verified that  $d_{j-1}(X)$  can be written as

$$d_{j-1}(X) = d_{i-1}(X) - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f, \quad (16)$$

and that  $d_{i-1}(X)$  can be written as

$$d_{i-1}(X) = X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f. \quad (17)$$

Hence, it follows from (16) that

$$\begin{aligned} \sigma_j &= \mathbb{E}[d_{j-1}(X)^{\otimes 2}] \\ &= \mathbb{E}\left[\left(d_{i-1}(X) - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]f\right)^{\otimes 2}\right] \\ &= \sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[f d_{i-1}(X)] \otimes \mathbb{E}[Xf] \\ &\quad - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[fX] \otimes \mathbb{E}[d_{i-1}(X)f] \\ &\quad + \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]^{\otimes 2}, \end{aligned} \quad (18)$$

where the last summand follows from the orthonormality of  $\hat{\mathcal{F}}$ . Further, it follows from (17) that every  $f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}$  satisfies

$$\mathbb{E}[f d_{i-1}(X)] = \mathbb{E}\left[f \cdot \left(X - \sum_{g \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xg]g\right)\right] = \mathbb{E}[fX],$$

and therefore (18) =  $\sigma_i - \sum_{f \in \hat{\mathcal{F}}_{j-1} \setminus \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Xf]^{\otimes 2}$  as required.  $\square$

*Proof of Lemma 4.(b):* Notice that for every  $i \in [d]$  and every  $r \in [m]$  we have

$$\sigma_{r,i} = \mathbb{E}[d_{r-1}(X)_i^2]. \quad (19)$$

and since  $d_{r-1}(X) = X - \sum_{f \in \hat{\mathcal{F}}_{r-1}} \mathbb{E}[Xf]f$ , it follows that

$$(19) = \mathbb{E}\left[\left(X_i - \sum_{f \in \hat{\mathcal{F}}_{r-1}} \mathbb{E}[X_i f]f\right)^2\right] \quad (20)$$

That is, at the beginning of the  $r$ 'th iteration for any  $r$ , GFS will find the maximizer  $s_r$  over  $i$  of (20). Now, observe that if  $i = s_k$  for some  $k \in \{1, \dots, r-1\}$ , i.e., if the  $i$ 'th feature was already selected in an earlier iteration  $k$ , then  $Z_k = X_i$ . Let  $\tilde{Z}_k$  and  $\hat{Z}_k$  be the orthogonalized and orthonormalized versions of  $Z_k$ . Since  $Z_k = \tilde{Z}_k + \sum_{f \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k f]f$ , we have

$$\begin{aligned} (20) &= \mathbb{E}\left[\left(\tilde{Z}_k + \sum_{g \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k g]g - \sum_{f \in \hat{\mathcal{F}}_{r-1}} \mathbb{E}\left[\left(\tilde{Z}_k + \sum_{g \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k g]g\right)f\right]f\right)^2\right] \\ &= \mathbb{E}\left[\left(\tilde{Z}_k + \sum_{g \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k g]g - \sum_{f \in \hat{\mathcal{F}}_{r-1}} \left(\mathbb{E}[\tilde{Z}_k f] + \sum_{g \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k g]\mathbb{E}[g f]\right)f\right)^2\right] \\ &= \mathbb{E}\left[\left(\tilde{Z}_k + \sum_{g \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k g]g - \sum_{f \in \hat{\mathcal{F}}_{r-1}} \mathbb{E}[\tilde{Z}_k f]f - \sum_{g \in \hat{\mathcal{F}}_{k-1}} \mathbb{E}[Z_k g]g\right)^2\right] \\ &= \mathbb{E}\left[\left(\tilde{Z}_k - \sum_{f \in \hat{\mathcal{F}}_{r-1}} \mathbb{E}[\tilde{Z}_k f]f\right)^2\right]. \end{aligned} \quad (21)$$

Since  $\tilde{Z}_k$  is orthogonalized, it follows that

$$\mathbb{E}[\tilde{Z}_k f] = \begin{cases} 0 & \text{if } f \in \hat{\mathcal{F}}_{r-1} \setminus \{\hat{Z}_k\} \\ \|\tilde{Z}_k\| & \text{if } f = \hat{Z}_k \end{cases},$$

and hence

$$(21) = \mathbb{E}[(\tilde{Z}_k - \hat{Z}_k \cdot \|\tilde{Z}_k\|)^2] = 0,$$

It then follows that the maximization problem will not select an index  $i$  that was already selected at a previous iteration.  $\square$

*Proof of Lemma 3:* According to the stopping criterion in GFS, it follows that

$$\begin{aligned} \|\sigma_i\|_\infty &> \epsilon^2 \text{ for all } i \in [m]; \text{ and} \\ \|\sigma_{m+1}\|_\infty &\leq \epsilon^2. \end{aligned} \quad (22)$$

Assume for contradiction that there exists  $i \in \{m+1, \dots, d\}$  such that  $\|\sigma_i\|_\infty > \epsilon^2$ , and observe that  $i > m+1$ , since otherwise the existence of  $s_{m+1}$  contradicts (22). It follows from Lemma 4.(a) that

$$\sigma_i = \sigma_{m+1} - \sum_{f \in \hat{\mathcal{F}}_{i-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[fX]^{\otimes 2},$$

and therefore

$$\begin{aligned} \|\sigma_i\|_\infty &= \sigma_{i,s_i} = \sigma_{m+1,s_i} - \sum_{f \in \hat{\mathcal{F}}_{i-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[fX_{s_i}]^2 \\ &\stackrel{(22)}{\leq} \epsilon^2 - \sum_{f \in \hat{\mathcal{F}}_{i-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[fX_{s_i}]^2. \end{aligned}$$

Therefore, since  $\|\sigma_i\|_\infty > \epsilon^2$ , it follows that

$$- \sum_{f \in \hat{\mathcal{F}}_{i-1} \setminus \hat{\mathcal{F}}_m} \mathbb{E}[fX_{s_i}]^2 > 0,$$

which is a contradiction since  $\mathbb{E}[fX_{s_i}]^2 \geq 0$  for every  $f$ .  $\square$

*Proof of Theorem 4:* Let  $\{s_{m+1}, \dots, s_d\} = [d] \setminus \{s_1, \dots, s_m\}$  and recall that  $Z = X_{S_e}$  and  $Z^\perp = (Z_{m+1}, \dots, Z_d)$ . Observe that

$$H(X|Z) = H(\{Z_i\}_{i=1}^d | \{Z_i\}_{i=1}^m)$$

$$= \sum_{i=m+1}^d H(Z_i | (Z_j)_{j=1}^{i-1}), \quad (23)$$

where the last equality follows from the chain rule for information entropy. Since

$$\tilde{Z}_i = Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f, \quad (24)$$

it follows that

$$\begin{aligned} H(Z_i | (Z_j)_{j=1}^{i-1}) &= H\left(\tilde{Z}_i + \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f | (Z_j)_{j=1}^{i-1}\right) \\ &\stackrel{(a)}{=} H(\tilde{Z}_i | (Z_j)_{j=1}^i) \stackrel{(b)}{\leq} H(\tilde{Z}_i), \end{aligned} \quad (25)$$

where (a) follows since the expression  $\sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f$  is uniquely determined by the variables  $Z_1, \dots, Z_{i-1}$  (every  $f \in \hat{\mathcal{F}}_{i-1}$  is a deterministic function of  $Z_1, \dots, Z_{i-1}$ , and the coefficients  $\mathbb{E}[Z_i f]$  are constants), and (b) follows since conditioning reduces entropy. Combining (23) with (25) we have that  $H(X|Z) \leq \sum_{i=m+1}^d H(\tilde{Z}_i)$ , and hence it remains to bound  $H(\tilde{Z}_i)$  for all  $i \in \{m+1, \dots, d\}$ .

To this end let  $i \in \{m+1, \dots, d\}$ , define  $\alpha_i \triangleq \min\{|\tilde{Z}_i(\alpha)| : \alpha \in \mathcal{X}^d, \tilde{Z}_i(\alpha) \neq 0\}$  and  $\alpha_{\min} = \min\{\alpha_i\}_{i=m+1}^d$ , and observe that by Markov's inequality we have

$$\begin{aligned} \Pr(\tilde{Z}_i \neq 0) &= \Pr(|\tilde{Z}_i| \geq \alpha_i) = \Pr(\tilde{Z}_i^2 \geq \alpha_i^2) \\ &\leq \frac{\mathbb{E}[\tilde{Z}_i^2]}{\alpha_i^2} \leq \frac{\mathbb{E}[\tilde{Z}_i^2]}{\alpha_{\min}^2}. \end{aligned} \quad (26)$$

Moreover, recall that

$$\begin{aligned} d_{i-1}(X) &= X - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[X f] f, \text{ and} \\ \sigma_i &= \mathbb{E}[d_{i-1}(X)^{\otimes 2}], \end{aligned}$$

and therefore

$$\begin{aligned} \|\sigma_i\|_{\infty} &= \sigma_{i, s_i} = \mathbb{E}[d_{i-1}(X)_{s_i}^2] \\ &= \mathbb{E}\left[\left(X_{s_i} - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[X_{s_i} f] f\right)^2\right] \\ &= \mathbb{E}\left[\left(Z_i - \sum_{f \in \hat{\mathcal{F}}_{i-1}} \mathbb{E}[Z_i f] f\right)^2\right] \\ &\stackrel{(24)}{=} \mathbb{E}[\tilde{Z}_i^2]. \end{aligned}$$

Consequently, it follows that

$$(26) = \frac{\|\sigma_i\|_{\infty}}{a_{\min}^2} \stackrel{\text{Lemma 3}}{\leq} \frac{\epsilon^2}{a_{\min}^2}.$$

Now, by using the grouping rule [7, Ex. 2.27] we have

$$H(\tilde{Z}_i) \leq h_b(\epsilon^2/a_{\min}^2) + \frac{\epsilon^2}{a_{\min}^2} \log |\mathcal{X}|,$$

where  $h_b$  is the binary entropy function. Finally, using the bound  $h_b(p) \leq 2\sqrt{p(1-p)} \leq 2\sqrt{p}$ , it follows that

$$\begin{aligned} H(X|Z) &\leq \sum_{i=m+1}^d H(\tilde{Z}_i) \\ &\leq (d-m)(h_b(\epsilon^2/a_{\min}^2) + (\epsilon^2/a_{\min}^2) \log |\mathcal{X}|) \\ &\leq (d-m)(2\epsilon/a_{\min} + (\epsilon/a_{\min}) \log |\mathcal{X}|) = dO(\epsilon). \end{aligned}$$

## APPENDIX B COMPLEXITY ANALYSIS

The computation bottleneck of our algorithm is the orthogonalization process (Algorithm 1). Theorem 6 shows the complexity upper bound on our algorithms.

*Theorem 6:* Let  $c(\mathcal{F})$  be an upper bound on the time complexity of computing any  $f \in \mathcal{F}$  (once the variables  $z_j$  are known), and let  $N$ ,  $d$ , and  $m$  be the number of samples, dataset dimension, and the number of extracted features, respectively. Then, the complexity upper bound on GCA and GFR is

$$O(|\hat{\mathcal{F}}_m|^2 N c(\mathcal{F}) + |\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d^2) + d^2(N + d)). \quad (27)$$

*Proof:* The orthogonalization process requires to subtract from each  $f_i$  its projections along  $f_1, \dots, f_{i-1}$ , each of which carries a coefficient of the form  $\mathbb{E}[f_i f_j]$  for  $j < i$  which can be empirically estimated in  $O(Nc(\mathcal{F}))$ . Letting  $m$  be the number of extracted features, this sums up to  $O(Nc(\mathcal{F})|\hat{\mathcal{F}}_m|^2)$ .

The computation of the matrices  $\{\Sigma_j\}_{j=1}^m$  can be done recursively using<sup>13</sup> Lemma 2.(a). Since

$$\Sigma_{j+1} = \Sigma_j - \sum_{f \in \hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}} \mathbb{E}[fX] \mathbb{E}[X^\top f],$$

at iteration  $j$  the matrix  $\Sigma_{j+1}$  can be approximated in  $O(|\hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}|N(c(\mathcal{F}) + d) + d^2|\hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}|)$ , since each  $\mathbb{E}[fX]$  can be approximated in  $O(N(c(\mathcal{F}) + d))$ , and then  $O(|\hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}|)$  operations are required per entry. Summing over  $j \in [m]$  yields  $O(|\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d^2))$ . Therefore, all orthogonalization operations in the process of extracting  $m$  features require  $O(|\hat{\mathcal{F}}_m|^2 N c(\mathcal{F}) + |\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d^2))$ .

Outside the orthogonalization operations, GCA (Algorithm 3) requires algebraic and min/max operations, as well as covariance matrix approximation ( $\mathbb{E}[\bar{X}_j \bar{X}_j^\top]$ ), all dominated by  $O(d^2(N + d))$ . GFR (Algorithm 2) also requires solving the largest eigenvector problem, which can be done in  $O(d^3)$ , but iterative methods in  $O(d^2)$  exist. Hence, a rough complexity upper bound on GCA and GFR is

$$O(|\hat{\mathcal{F}}_m|^2 N c(\mathcal{F}) + |\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d^2) + d^2(N + d)).$$

□

Similar complexity upper bound on GFS and GFA can also be shown as follows.

*Theorem 7:* Let  $c(\mathcal{F})$  be an upper bound on the time complexity of computing any  $f \in \mathcal{F}$  (once the variables  $z_j$  are known), and let  $N$ ,  $d$ , and  $m$  be the number of samples, dataset dimension, and the number of selected features, respectively. Then, the complexity upper bound on GFA and GFS is

$$O(|\hat{\mathcal{F}}_m|^2 N c(\mathcal{F}) + |\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d) + d(N + d)). \quad (28)$$

*Proof:* As shown in the proof of Theorem 6, the computational complexity of the orthogonalization process is  $O(Nc(\mathcal{F})|\hat{\mathcal{F}}_m|^2)$ . The computation of the variance vectors  $\{\sigma_j\}_{j=1}^m$  can be done recursively

$$\sigma_{j+1} = \sigma_j - \sum_{f \in \hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}} \mathbb{E}[fX]^{\otimes 2},$$

<sup>13</sup>The distributions  $d_j$  are merely proof artifacts, and need not be computed in practice. □

at iteration  $j$  the vector  $\sigma_{j+1}$  can be approximated in  $O(|\hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}|N(c(\mathcal{F}) + d) + d|\hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}|)$ , since each  $\mathbb{E}[fX]$  can be approximated in  $O(N(c(\mathcal{F}) + d))$ , and then  $O(|\hat{\mathcal{F}}_j \setminus \hat{\mathcal{F}}_{j-1}|)$  operations are required per entry. Summing over  $j \in [m]$  yields  $O(|\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d))$ . Therefore, all orthogonalization operations in the process of extracting  $m$  features require  $O(|\hat{\mathcal{F}}_m|^2 Nc(\mathcal{F}) + |\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d))$ . Outside the orthogonalization operations, GFA (Algorithm 5) finds the highest variant feature with  $O(d(N + d))$  complexity. GFS (Algorithm 4) also requires similar computational complexity. Hence, a rough complexity upper bound on GFA and GFS is

$$O(|\hat{\mathcal{F}}_m|^2 Nc(\mathcal{F}) + |\hat{\mathcal{F}}_m|(N(c(\mathcal{F}) + d) + d) + d(N + d)).$$

□

### APPENDIX C

#### UNSUPERVISED FOURIER FEATURE SELECTION, A COMPARISON

A fascinating feature selection framework has been recently put forward by [5] (see also [6], [51]). In this framework, the random variable  $X = (X_1, \dots, X_d)$  is viewed from a Fourier-analytic perspective in the following sense. A set of *characters*  $\{\phi_S\}_{S \subseteq [d]}$  is defined as all multilinear monomials in the (centered and normalized, and possibly dependent) variables  $\{X_i\}_{i=1}^d$ , i.e.,  $\phi_S(X) = \prod_{i \in S} X_i$ . These characters are then orthogonalized as  $\{\tilde{\psi}_S\}_{S \subseteq [d]}$  with respect to the data distribution using a Gram-Schmidt process,

$$\begin{aligned} \tilde{\psi}_{S_i} &= \phi_{S_i} - \sum_{j=1}^{i-1} \mathbb{E}[\psi_{S_j} \phi_{S_i}] \psi_{S_j}, \text{ where} \\ \psi_{S_i} &= \begin{cases} \frac{\tilde{\psi}_{S_i}}{\|\tilde{\psi}_{S_i}\|} & \text{if } \|\tilde{\psi}_{S_i}\| \neq 0 \\ 0 & \text{otherwise,} \end{cases} \end{aligned} \quad (29)$$

and where the sets  $S \subseteq [d]$  are indexed in the following order:

$$\begin{aligned} &\emptyset, \{1\}, \{2\}, \{1, 2\}, \{3\}, \{1, 3\}, \{2, 3\}, \\ &\{1, 2, 3\}, \dots, \{1, 2, \dots, d\}. \end{aligned} \quad (30)$$

Then, it is shown that the Fourier norm  $\|\tilde{\psi}_j\| = \sqrt{\mathbb{E}[\tilde{\psi}_j^2]}$ , where  $\tilde{\psi}_j$  is a shorthand notation for  $\tilde{\psi}_{\{j\}}$ , can be seen as a measure for the “informativeness” of feature  $j$  in the following sense.

**Theorem 8:** [5, Thm. 1] Let  $S_\epsilon \subseteq [d]$  be the set of indices  $j$  such that  $\|\tilde{\psi}_j\| > \epsilon$ . Then  $H(X|X^{S_\epsilon}) = dO(\epsilon)$ , where  $X^{S_\epsilon} = (X_j)_{j \in S_\epsilon}$ .

That is, selecting those features whose respective Fourier norm is high, reduces the information entropy of the data, and therefore those features are “sufficiently informative.” In practice, orthogonalization is performed by approximating expectations using empirical means, then the Fourier norms  $\|\tilde{\psi}_j\|$  are calculated similarly, and the features whose norms are larger than  $\epsilon$  are selected.

In what follows we briefly explain that GFS, when choosing  $\mathcal{F}$  as the set of all multilinear polynomials, specifies to the above framework by [5], yet at reduced complexity. A key observation regarding Theorem 8, which enables GFS to subsume Algorithm 6 at reduced complexity, is that the order

#### Algorithm 6 Unsupervised Fourier Feature Selection (UFFS)

*Theorem 8 implies a feature selection algorithm, in which orthogonal characters are constructed as in (29), (30), the Fourier norms  $\|\tilde{\psi}_j\|$  are computed for every  $j$  (approximating expectations by empirical means), and the  $j$ 's whose norms are larger than a chosen threshold  $\epsilon > 0$  are selected.*

of sets in (30) is rather arbitrary, and for any permutation  $\pi$  over  $[d]$  we may set

$$\begin{aligned} \tilde{\psi}_{S_i^\pi} &= \phi_{S_i^\pi} - \sum_{j=1}^{i-1} \mathbb{E}[\psi_{S_j^\pi} \phi_{S_i^\pi}] \psi_{S_j^\pi}, \text{ where} \\ \psi_{S_i^\pi} &= \begin{cases} \frac{\tilde{\psi}_{S_i^\pi}}{\|\tilde{\psi}_{S_i^\pi}\|} & \text{if } \|\tilde{\psi}_{S_i^\pi}\| \neq 0 \\ 0 & \text{otherwise,} \end{cases} \end{aligned} \quad (31)$$

and where  $\{S_i^\pi\}_{i=1}^{2^d}$  is the order

$$\begin{aligned} &\emptyset, \{\pi(1)\}, \{\pi(2)\}, \{\pi(1), \pi(2)\}, \{\pi(3)\}, \{\pi(1), \pi(3)\}, \\ &\{\pi(2), \pi(3)\}, \{\pi(1), \pi(2), \pi(3)\}, \dots, \{\pi(1), \pi(2), \dots, \pi(d)\}, \end{aligned}$$

and the guarantees in Theorem 8 still hold true. GFS, however, finds such a permutation  $\pi$  on the fly, one element at a time; the next element to be chosen is the one which maximizes  $\{\sigma_{r,i}\}_{i=1}^d$ . Now, since

$$\sigma_{r,i} = \mathbb{E}[d_{r-1}(X)_i^2], \text{ and since}$$

$$d_{r-1}(X) = X - \sum_{\ell=1}^{2^{r-1}} \mathbb{E}[X \psi_{S_\ell^\pi}] \psi_{S_\ell^\pi}, \quad (32)$$

it follows that

$$\begin{aligned} (32) &= \mathbb{E} \left[ \left( X_i - \sum_{\ell=1}^{2^{r-1}} \mathbb{E}[X_i \psi_{S_\ell^\pi}] \psi_{S_\ell^\pi} \right)^2 \right] \\ &= \mathbb{E} \left[ \left( \phi_i - \sum_{\ell=1}^{2^{r-1}} \mathbb{E}[\phi_i \psi_{S_\ell^\pi}] \psi_{S_\ell^\pi} \right)^2 \right]. \end{aligned} \quad (33)$$

That is, at the beginning of the  $r$ 'th iteration for any  $r$ , the algorithm will find the maximizer over  $i$  of (33). Denoting this maximizer by  $j_r$ , we have that  $S_{2^{r-1}+1}^\pi = \{j_r\}$ , and hence

$$\begin{aligned} &\mathbb{E} \left[ \left( \phi_{j_r} - \sum_{\ell=1}^{2^{r-1}} \mathbb{E}[\phi_{j_r} \psi_{S_\ell^\pi}] \psi_{S_\ell^\pi} \right)^2 \right] \\ &= \mathbb{E} \left[ \left( \phi_{S_{2^{r-1}+1}^\pi} - \sum_{\ell=1}^{2^{r-1}} \mathbb{E}[\phi_{S_{2^{r-1}+1}^\pi} \psi_{S_\ell^\pi}] \psi_{S_\ell^\pi} \right)^2 \right] \\ &\stackrel{(31)}{=} \mathbb{E}[(\tilde{\psi}_{S_{2^{r-1}+1}^\pi}^\pi)^2] = \mathbb{E}[(\tilde{\psi}_{j_r}^\pi)^2] = \|\tilde{\psi}_{j_r}^\pi\|^2. \end{aligned} \quad (34)$$

That is, the algorithm will choose the feature which maximizes the Fourier norm, and will stop once the resulting maximum is less than  $\epsilon^2$ . This implies the following.

**Corollary 4:** GFS generalizes Algorithm 6, which arises as a special case when  $\mathcal{F}$  is chosen as the set of all multilinear polynomials. Yet, Algorithm 6 requires one to first compute all  $2^d$  orthogonalized characters, and only then to choose the  $i$ 's such that  $\|\tilde{\psi}_i\|$  are larger than  $\epsilon$ . In contrast, since GFS finds the orthogonalization on the fly, it only requires  $2^m$  orthogonalization operations, where  $m$  is the number of selected features; the fact that the remaining  $\|\tilde{\psi}_i\|$ 's lie below  $\epsilon$  follows from Lemma 3 in Appendix A.



The complexity benefits of GFS extend beyond setting  $\mathcal{F}(\mathbf{z})$  as the set of all multilinear polynomials. For instance, in order to reduce computational complexity, [5] propose to fix  $\mathcal{F}(\mathbf{z})$  as the set of *bounded degree*  $\ell$  multilinear monomials, of which there are  $\binom{d}{\leq \ell} \triangleq \sum_{i=0}^{\ell} \binom{d}{i}$ , and thus only this many orthogonalization operations are needed. Applying this  $\mathcal{F}(\mathbf{z})$  in GFS, one gets that only  $\binom{m}{\leq \ell}$  orthogonalizations are required. In addition, a clustering heuristic is also suggested, where the features are clustered a priori to subsets of size  $\ell$ , and orthogonalization is performed only within each cluster, thus reducing the number of orthogonalizations to  $\frac{d}{\ell} 2^\ell$ . In contrast, applying GFS on each cluster would result in  $\sum_{i=1}^{d/\ell} 2^{m_i}$  many orthogonalization, where  $m_i \leq \ell$  is the number of features that are selected from the  $i$ 'th cluster.

## APPENDIX D ALGEBRAIC REPRESENTATIONS OF THE PROPOSED ALGORITHMS

### A. Algebraic Representation of the Gram-Schmidt Orthogonalization Process

To the request of the reviewers, in order to enhance clarity and facilitate implementation from a data science perspective, we provide algebraic representations of the proposed methods. While the original formulations were expressed in a probabilistic framework to emphasize theoretical properties, such as orthogonality in the  $L^2(P_X)$  space, nonlinear redundancy in the data, and information-theoretic guarantees, the core procedures can be naturally interpreted as algebraic operation (due to Remark 2). This reformulation replaces expectations with empirical means and random variables with data vectors, enabling the algorithms to be described directly in terms of matrix operations, inner products, and projections onto sample data. The resulting presentation aligns more closely with classical techniques such as PCA and Gram-Schmidt orthogonalization, thereby making the methods more accessible for practical implementation. To replace expectations with empirical means, we use data points  $\mathbf{x}_i \in \mathbb{R}^d$  for  $i = 1, \dots, N$ , where  $N$  is the number of data points in the dataset. For example, an expression such as  $\Sigma_j = \mathbb{E}[d_j(X)d_j(X)^\top]$  is replaced by its empirical counterpart  $\Sigma_j = \frac{1}{N} \sum_{i=1}^N d_j(\mathbf{x}_i)d_j(\mathbf{x}_i)^\top$ . Similarly, for two arbitrary functions  $f$  and  $g$ , we replace  $\mathbb{E}[f(X)g(X)]$  with  $\frac{1}{N} \sum_{i=1}^N f(\mathbf{x}_i)g(\mathbf{x}_i)$ .

In Algorithm 7, we provide the algebraic representation of the Gram-Schmidt orthogonalization process. We let  $\mathbf{z} = [z_1, \dots, z_d]^\top$  be a vector of symbolic variables and use it to define the procedure. At iteration  $j$  of our algorithms, Algorithm 7 receives a set of already orthogonalized functions  $\hat{\mathcal{F}}(\mathbf{z}_{[j-1]})$ , a set of new functions  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]})$  to be orthogonalized, all previously extracted directions  $\mathbf{v}_1, \dots, \mathbf{v}_j$ , and data points  $\{\mathbf{x}_i\}_{i=1}^N$ . For each function in  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]})$ , the algorithm defines a new function  $g(\mathbf{z}_{[j]})$  (Line 6), subtracts its projection onto the previously orthogonalized functions using empirical projections (replacing  $\sum_{f \in \mathcal{T}} \mathbb{E}[gf]f$  from the probabilistic formulation; see Line 7), normalizes  $g$  by dividing by the empirical counterpart of  $\sqrt{\mathbb{E}[g^2]}$  (Line 8), and adds the result to  $\mathcal{T}$  (Line 9). Finally, we define a new symbolic variable  $d_j(\mathbf{z})$  (Line 12), compute its empirical covariance

---

**Algorithm 7** Algebraic-Orthogonalize( $\hat{\mathcal{F}}(\mathbf{z}_{[j-1]}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{\mathbf{v}_i\}_{i=1}^j, \{\mathbf{x}_i\}_{i=1}^N$ )

---

- 1: **Input:**
    - $\hat{\mathcal{F}}(\mathbf{z}_{[j-1]})$ : an orthogonalized function family.
    - $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]})$ : all the functions in  $\mathcal{F}(\mathbf{z})$  which depend on  $\mathbf{z}_{[j]}$  but not only on  $\mathbf{z}_{[j-1]}$ .
    - $\mathbf{v}_1, \dots, \mathbf{v}_j$ : extracted directions.
    - $\{\mathbf{x}_i\}_{i=1}^N$ :  $N$  data points.
  - 2: **Output:** Orthogonalized function family  $\hat{\mathcal{F}}(\mathbf{z}_{[j]})$ , and a covariance matrix  $\Sigma_{j+1}$ .
  - 3: **Initialize:**  $\mathcal{T} \leftarrow \hat{\mathcal{F}}(\mathbf{z}_{[j-1]})$ .
  - 4: Denote  $\mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}) \triangleq \{f_1(\mathbf{z}_{[j]}), \dots, f_\ell(\mathbf{z}_{[j]})\}$ , where  $f_1(\mathbf{z}_{[j]}) = z_j$ .
  - 5: **for**  $k \leftarrow 1$  **to**  $\ell$  **do**
  - 6:    $g(\mathbf{z}_{[j]}) \leftarrow f_k(\mathbf{z}_{[j]})$
  - 7:    $g(\mathbf{z}_{[j]}) \leftarrow g(\mathbf{z}_{[j]}) - \sum_{f(\mathbf{z}_{[j]}) \in \mathcal{T}} \left( \frac{1}{N} \sum_{i=1}^N (g(\mathbf{x}_i^\top \mathbf{v}_1, \dots, \mathbf{x}_i^\top \mathbf{v}_j) f(\mathbf{x}_i^\top \mathbf{v}_1, \dots, \mathbf{x}_i^\top \mathbf{v}_j)) \right) f(\mathbf{z}_{[j]})$
  - 8:    $g(\mathbf{z}_{[j]}) \leftarrow \frac{g(\mathbf{z}_{[j]})}{\sqrt{\frac{1}{N} \sum_{i=1}^N g(\mathbf{x}_i^\top \mathbf{v}_1, \dots, \mathbf{x}_i^\top \mathbf{v}_j)^2}}$
  - 9:    $\mathcal{T} \leftarrow \mathcal{T} \cup \{g(\mathbf{z}_{[j]})\}$
  - 10: **end for**
  - 11: Define  $\hat{\mathcal{F}}(\mathbf{z}_{[j]}) = \mathcal{T}$ .
  - 12: Define  $d_j(\mathbf{z}) = \mathbf{z} - \frac{1}{N} \sum_{f(\mathbf{z}) \in \mathcal{T}} \sum_{i=1}^N [\mathbf{x}_i f(\mathbf{x}_i^\top \mathbf{v}_1, \dots, \mathbf{x}_i^\top \mathbf{v}_j)] f(\mathbf{z}^\top \mathbf{v}_1, \dots, \mathbf{z}^\top \mathbf{v}_j)$ .
  - 13: Define  $\Sigma_{j+1} = \frac{1}{N} \sum_{i=1}^N d_j(\mathbf{x}_i)d_j(\mathbf{x}_i)^\top$ .
  - 14: Return  $\Sigma_{j+1}, \mathcal{T}$ .
- 

---

**Algorithm 8** Algebraic Gram-Schmidt Functional Reduction (Algebraic-GFR)

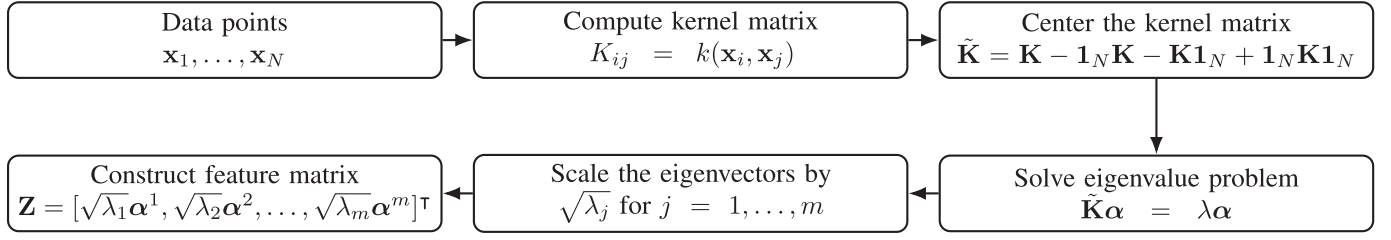
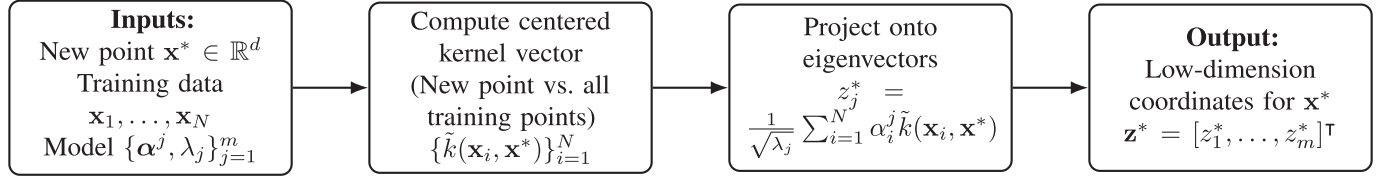
---

- 1: **Input:** A function family  $\mathcal{F}(\mathbf{z})$ , a threshold  $\epsilon > 0$ , and data points  $\{\mathbf{x}_i\}_{i=1}^N$ .
  - 2: **Output:** Extracted directions  $\mathbf{v}_1, \dots, \mathbf{v}_m$  ( $m$  is a varying number that depends on  $\mathcal{F}(\mathbf{z}), \epsilon$ ).
  - 3: **Initialize:**  $\Sigma_1 = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^\top$ .
  - 4: **for**  $j \leftarrow 1$  **to**  $d$  **do**
  - 5:   Let  $\mathbf{v}_j$  be the largest unit-norm eigenvector of  $\Sigma_j$ , i.e.,  $\mathbf{v}_j = \arg \max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \Sigma_j \mathbf{v}$ .
  - 6:   **if**  $\mathbf{v}_j^\top \Sigma_j \mathbf{v}_j \leq \epsilon^2$  **then**
  - 7:     break.
  - 8:   **end if**
  - 9:    $\Sigma_{j+1}, \hat{\mathcal{F}}(\mathbf{z}_{[j]}) = \text{Algebraic-Orthogonalize}(\hat{\mathcal{F}}(\mathbf{z}_{[j-1]}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{\mathbf{v}_i\}_{i=1}^j, \{\mathbf{x}_i\}_{i=1}^N)$ .
  - 10: **end for**
- 

matrix (Line 13), and return it as the output of Algorithm 7. We denote  $\mathbf{x}_i \in \mathbb{R}^d$ , for  $i = 1, \dots, N$ , as the data points used to compute empirical means.

### B. Algebraic Representations of GFR and GCA

In Algorithm 8, we provide the algebraic representation of GFR, corresponding to Algorithm 2 in Section III. In this version, empirical covariance matrices are used in place of their probabilistic counterparts to identify the new high-variance direction in Line 5.

Fig. 6. Algorithmic workflow for extracting features from data points  $\mathbf{x}_1, \dots, \mathbf{x}_N$  using **Kernel PCA**.Fig. 7. Workflow for applying a trained **Kernel PCA** model to a new data point  $\mathbf{x}^*$ .**Algorithm 9** Algebraic Gram-Schmidt Component Analysis (Algebraic-GCA)

- 1: **Input:** A function family  $\mathcal{F}(\mathbf{z})$ , a threshold  $\epsilon > 0$ , and data points  $\{\mathbf{x}_i\}_{i=1}^N$ .
- 2: **Output:** A set  $\mathcal{L} \subseteq [d]$  of indices of extracted principal directions.
- 3: **Initialize:**  $\Sigma_1 = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T$  and  $\mathcal{L}_0 = \emptyset$ .
- 4: **for**  $j \leftarrow 1$  **to**  $d$  **do**
- 5:   Let  $\mathcal{E}_j = \{i | \rho_i^T \Sigma_j \rho_i < \epsilon\}$ .
- 6:   **if**  $\mathcal{E}_j = [d]$  **then**
- 7:     **return**  $\mathcal{L}_{j-1}$ .
- 8:   **end if**
- 9:   Define  $g_j(\mathbf{z}) = \mathbf{z} - \sum_{i \in \mathcal{E}_j} \mathbf{z}^T \rho_i \cdot \rho_i$ .
- 10:   Let  $\ell_j = \arg \max_{i \in [d] \setminus \mathcal{E}_j} \rho_i^T \left( \frac{1}{N} \sum_{t=1}^N g_j(\mathbf{x}_t) g_j(\mathbf{x}_t)^T \right) \rho_i$  (breaking ties arbitrarily), and  $\mathcal{L}_j = \mathcal{L}_{j-1} \cup \{\ell_j\}$ .
- 11:    $\Sigma_{j+1}, \hat{\mathcal{F}}(\mathbf{z}_{[j]}) = \text{Algebraic-Orthogonalize}(\hat{\mathcal{F}}(\mathbf{z}_{[j-1]}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{\rho_i\}_{i \in \mathcal{L}_j}, \{\mathbf{x}_i\}_{i=1}^N)$
- 12: **end for**
- 13: **return**  $\mathcal{L}_d$ .

The algebraic representation of GCA, shown in Algorithm 9, is obtained by replacing the probabilistic covariance matrix of  $g_j(\mathbf{z})$ , denoted by  $\bar{X}_j$  in the original representation, with its empirical counterpart.

**C. Algebraic Representations of GFS and GFA**

As in the probabilistic representation, we replace the maximization step used to find the eigenvector  $\mathbf{v}_j$  in Line 5 of Algorithm 8 with a discrete version. The algorithm terminates if the resulting variance falls below the threshold  $\epsilon^2$ . The final output is a subset  $\mathcal{S} \subseteq [d]$  of selected features. In feature selection algorithms, it suffices to compute empirical *variance vectors*  $\sigma_j$  rather than full empirical covariance matrices  $\Sigma_j$ . Using  $\sigma_j$  in place of  $\Sigma_j$  allows for certain simplifications. For example, when Algorithm 7 is used for feature *selection* rather than feature extraction, the following modifications are

**Algorithm 10** Algebraic Gram-Schmidt Functional Selection (Algebraic-GFS)

- 1: **Input:** A function family  $\mathcal{F}(\mathbf{z})$ , a threshold  $\epsilon > 0$ , and data points  $\{\mathbf{x}_i\}_{i=1}^N$ .
- 2: **Output:** Selected features  $s_1, \dots, s_m$  ( $m$  is a varying number that depends on  $\mathcal{F}(\mathbf{z}), \epsilon$ ).
- 3: **Initialize:**  $\sigma_1 = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i^{\otimes 2}$ ,  $\mathcal{S} = \emptyset$ .
- 4: **for**  $j \leftarrow 1$  **to**  $d$  **do**
- 5:   Let  $s_j \triangleq \arg \max_{i \in [d] \setminus \mathcal{S}} \{\sigma_{j,i}\}_{i=1}^d$ , where  $\sigma_j = (\sigma_{j,i})_{i=1}^d$ .
- 6:    $\mathcal{S} = \mathcal{S} \cup \{s_j\}$ .
- 7:   **if**  $\|\sigma_j\|_\infty \leq \epsilon^2$  **then**
- 8:     **break**.
- 9:   **end if**
- 10:    $\sigma_{j+1}, \hat{\mathcal{F}}(\mathbf{z}_{[j]}) = \text{Algebraic-Orthogonalize}(\hat{\mathcal{F}}(\mathbf{z}_{[j-1]}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{\mathbf{e}_i\}_{i \in \mathcal{S}}, \{\mathbf{x}_i\}_{i=1}^N)$ .
- 11: **end for**

applied:

$$\begin{aligned}
 \text{(Line 13 of Alg. 7) Define } \Sigma_{j+1} &= \frac{1}{N} \sum_{i=1}^N d_j(\mathbf{x}_i) d_j(\mathbf{x}_i)^T \\
 &\rightarrow \text{Define: } \sigma_{j+1} = \frac{1}{N} \sum_{i=1}^N d_j(\mathbf{x}_i)^{\otimes 2}.
 \end{aligned}$$

(Line 14 of Alg. 7) Return  $\Sigma_{j+1}, \mathcal{T} \rightarrow$  Return  $\sigma_{j+1}, \mathcal{T}$ .

In Algorithm 10, we present the *Algebraic Gram-Schmidt Functional Selection* (Algebraic-GFS), in which each step  $j$  uses the empirical variance vector  $\sigma_j$  to select the most variant feature of  $d_{j-1}$ . If the maximum variance is less than the threshold  $\epsilon^2$ , the algorithm terminates. Otherwise, it continues by identifying  $s_j$  as the index of the highest-variance feature in  $d_{j-1}$ , which determines the input to the next invocation of the “Algebraic-Orthogonalize” procedure.

Following the same procedure used to derive Algebraic-GFS from its probabilistic counterpart, we obtain the *Alge-*

<sup>14</sup>Here,  $\mathbf{e}_i$  denotes the  $i$ 'th standard basis vector in  $\mathbb{R}^d$ , with a 1 in the  $i$ 'th entry and 0 elsewhere.

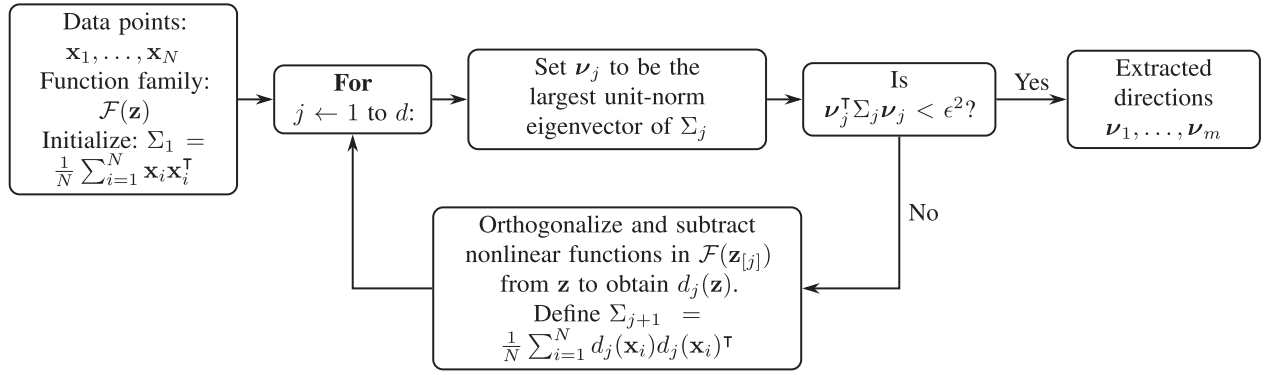
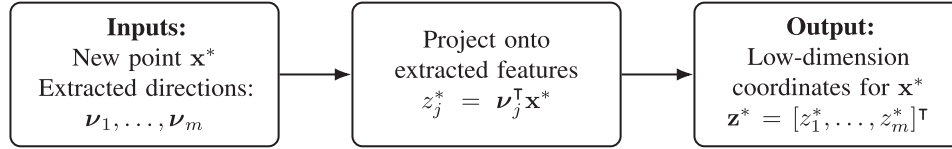
Fig. 8. Algorithmic workflow for extracting features from data points  $\mathbf{x}_1, \dots, \mathbf{x}_N$  using **GFR**.Fig. 9. Workflow for applying a trained **GFR** model to a new data point  $\mathbf{x}^*$ .

TABLE XVII

GFR COEFFICIENTS FOR THE FIRST THREE EXTRACTED FEATURES. EACH COLUMN REPRESENTS THE COEFFICIENTS OF A SINGLE EXTRACTED FEATURE, EXPRESSED AS A LINEAR COMBINATION OF THE ORIGINAL FEATURES

| Feature | First extracted feature | Second extracted feature | Third extracted feature |
|---------|-------------------------|--------------------------|-------------------------|
| $X_1$   | -0.5034008              | -0.0184953               | 0.0238543               |
| $X_2$   | 0.0094701               | -0.9979131               | 0.0608883               |
| $X_3$   | -0.0064781              | -0.0594010               | -0.9862699              |
| $X_4$   | -0.0009178              | 0.0172879                | 0.1515234               |
| $X_5$   | -0.8639764              | 0.0002652                | -0.0059973              |

**Algorithm 11** Algebraic Gram-Schmidt Feature Analysis (Algebraic-GFA)

- 1: **Input:** A function family  $\mathcal{F}(\mathbf{z})$ , a threshold  $\epsilon > 0$ , and data points  $\{\mathbf{x}_i\}_{i=1}^N$ .
- 2: **Output:** A set  $\mathcal{S} \subseteq [d]$  of indices of selected features.
- 3: **Initialize:**  $\sigma_1 = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i^{\otimes 2}$  and  $\mathcal{S} = \emptyset$ .
- 4: **for**  $j \leftarrow 1$  **to**  $d$  **do**
- 5:   Let  $\mathcal{E}_j = \{i | \sigma_{j,i} < \epsilon\}$ .
- 6:   **if**  $\mathcal{E}_j = [d]$  **then**
- 7:     **return**  $\mathcal{S}$ .
- 8:   **end if**
- 9:   Let  $s_j \triangleq \arg \max_{i \in [d] \setminus \mathcal{E}_j} \{\sigma_{j,i}\}$  and  $\mathcal{S} = \mathcal{S} \cup \{s_j\}$ .
- 10:    $\sigma_{j+1}, \hat{\mathcal{F}}(\mathbf{z}_{[j]}) = \text{Algebraic-Orthogonalize}(\hat{\mathcal{F}}(\mathbf{z}_{[j-1]}), \mathcal{F}(\mathbf{z}_{[j]}) \setminus \mathcal{F}(\mathbf{z}_{[j-1]}), \{\mathbf{e}_i\}_{i \in \mathcal{S}}, \{\mathbf{x}_i\}_{i=1}^N)$
- 11: **end for**
- 12: **return**  $\mathcal{S}_d$ .

*braic Gram-Schmidt Feature Analysis* (Algebraic-GFA) by replacing expectations with empirical means in Algorithm 5. The resulting algebraic formulation is presented in Algorithm 11.

## APPENDIX E

### KERNEL PCA: A COMPARISON

Kernel PCA is a nonlinear dimensionality reduction technique that extends standard PCA to capture complex data

TABLE XVIII

STRUCTURE OF THE EXPLICIT FEATURE MAP  $\Phi(X)$

| Total degree | Representative monomials        | Count     |
|--------------|---------------------------------|-----------|
| 0            | 1                               | 1         |
| 1            | $X_i$                           | 5         |
| 2            | $X_i^2, X_i X_j$                | 15        |
| 3            | $X_i^3, X_i^2 X_j, X_i X_j X_k$ | 35        |
| <b>Total</b> |                                 | <b>56</b> |

structures [14], [55]. The core idea of Kernel PCA is to project the input data from its original space  $\mathbb{R}^d$  into a higher-dimensional feature space  $\mathcal{H}$  using a nonlinear mapping function  $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ . In this feature space, nonlinear relationships in the original data can be represented linearly. Standard PCA is then applied within this high-dimensional space to identify the principal components that account for the maximum variance in the transformed data.

This process is made computationally efficient by leveraging the *kernel trick*. Rather than performing the explicit mapping, a kernel function,  $k(\mathbf{x}_i, \mathbf{x}_j) = \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle$ , is used to compute the inner products between all pairs of data points in the feature space directly, where  $\mathbf{x}_i$  for  $i = 1, \dots, N$  are the data points [14], [55]. The entire practical implementation, detailed in Figure 6, is built upon this principle. The first step involves computing the kernel (or Gram) matrix,  $\mathbf{K}$ , from the input data. To satisfy the zero-mean data requirement of PCA,

TABLE XIX

KERNEL PCA COEFFICIENTS FOR THE FIRST THREE EXTRACTED FEATURES ON THE EXPLICIT DEGREE THREE POLYNOMIAL FEATURE MAP. EACH COLUMN REPRESENTS THE COEFFICIENTS OF AN EXTRACTED FEATURE, EXPRESSED AS A LINEAR COMBINATION OF THE 56-DIMENSIONAL MAPPED FEATURES

| Feature map element | First extracted feature | Second extracted feature | Third extracted feature |
|---------------------|-------------------------|--------------------------|-------------------------|
| 1                   | 0                       | 0                        | 0                       |
| $X_1$               | -0.0013580              | 0.0004211                | -0.0026138              |
| $X_2$               | -0.0001010              | 0.0124174                | -0.0012818              |
| $X_3$               | 0.0000630               | -0.0000370               | -0.0009413              |
| $X_4$               | 0.0000030               | -0.0010657               | -0.0187138              |
| $X_5$               | -0.0068570              | 0.0020614                | -0.0059683              |
| $X_1^2$             | 0.0001050               | 0.0019951                | 0.0052743               |
| $X_1 X_2$           | -0.0006300              | -0.0004311               | 0.0026513               |
| $X_1 X_3$           | 0.0000030               | -0.0010657               | -0.0187138              |
| $X_1 X_4$           | 0.0003040               | -0.0008245               | -0.0038118              |
| $X_1 X_5$           | 0.0011740               | 0.0182158                | 0.0374256               |
| $X_2^2$             | 0.0002260               | 0.0063389                | 0.0098226               |
| $X_2 X_3$           | 0.0001190               | -0.0002863               | -0.0000659              |
| $X_2 X_4$           | 0.0002000               | -0.0015134               | -0.0027630              |
| $X_2 X_5$           | -0.0050300              | -0.0053237               | 0.0190429               |
| $X_3^2$             | -0.0000420              | -0.0000483               | -0.0020432              |
| $X_3 X_4$           | -0.0007890              | 0.0001343                | -0.0048543              |
| $X_3 X_5$           | 0.0002170               | -0.0041173               | -0.0589983              |
| $X_4^2$             | -0.0001380              | 0.0007376                | -0.0049381              |
| $X_4 X_5$           | 0.0022130               | -0.0051542               | -0.0131623              |
| $X_5^2$             | 0.0126570               | 0.1382663                | 0.2286677               |
| $X_1^3$             | -0.0051060              | 0.0030761                | -0.0111865              |
| $X_1^2 X_2$         | -0.0007510              | 0.0308195                | 0.0010471               |
| $X_1^2 X_3$         | 0.0003040               | -0.0008245               | -0.0038118              |
| $X_1^2 X_4$         | -0.0000720              | -0.0043660               | -0.0662884              |
| $X_1^2 X_5$         | -0.0248660              | 0.0160293                | -0.0318944              |
| $X_1 X_2^2$         | -0.0068570              | 0.0020614                | -0.0059683              |
| $X_1 X_2 X_3$       | 0.0002000               | -0.0015134               | -0.0027630              |
| $X_1 X_2 X_4$       | 0.0004590               | -0.0002668               | 0.0000969               |
| $X_1 X_2 X_5$       | -0.0033060              | 0.1562446                | -0.0067139              |
| $X_1 X_3^2$         | -0.0007890              | 0.0001343                | -0.0048543              |
| $X_1 X_3 X_4$       | -0.0001380              | 0.0007376                | -0.0049381              |
| $X_1 X_3 X_5$       | 0.0022130               | -0.0051542               | -0.0131623              |
| $X_1 X_4^2$         | -0.0026860              | 0.0003751                | -0.0222417              |
| $X_1 X_4 X_5$       | -0.0000320              | -0.0156654               | -0.2021858              |
| $X_1 X_5^2$         | -0.1462630              | 0.0607584                | -0.0996672              |
| $X_2^3$             | -0.0005290              | 0.0655799                | -0.0085468              |
| $X_2^2 X_3$         | 0.0005040               | -0.0004615               | -0.0044455              |
| $X_2^2 X_4$         | 0.0002170               | -0.0041173               | -0.0589983              |
| $X_2^2 X_5$         | -0.0422990              | 0.0081994                | -0.0067239              |
| $X_2 X_3^2$         | -0.0001000              | 0.0082728                | 0.0041120               |
| $X_2 X_3 X_4$       | -0.0004290              | 0.0006817                | 0.0039226               |
| $X_2 X_3 X_5$       | 0.0017500               | -0.0120602               | -0.0292133              |
| $X_2 X_4^2$         | -0.0003050              | 0.0182859                | 0.0198962               |
| $X_2 X_4 X_5$       | 0.0018890               | -0.0064891               | -0.0217272              |
| $X_2 X_5^2$         | -0.0050360              | 0.9688958                | -0.0977778              |
| $X_3^3$             | 0.0000150               | 0.0001309                | -0.0026999              |
| $X_3^2 X_4$         | -0.0001250              | -0.0027096               | -0.0383327              |
| $X_3^2 X_5$         | -0.0037320              | 0.0021466                | -0.0146465              |
| $X_3 X_4^2$         | 0.0000320               | -0.0013991               | -0.0135352              |
| $X_3 X_4 X_5$       | -0.0008180              | 0.0113528                | 0.0071007               |
| $X_3 X_5^2$         | 0.0147360               | -0.0430641               | -0.0692935              |
| $X_4^3$             | -0.0007210              | -0.0098587               | -0.1291472              |
| $X_4^2 X_5$         | -0.0121590              | 0.0062181                | -0.0718619              |
| $X_4 X_5^2$         | 0.0057270               | -0.0675782               | -0.9167747              |
| $X_5^3$             | -0.9876310              | -0.0146868               | 0.0139714               |

this matrix is then centered in the feature space using the formula  $\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_N \mathbf{K} - \mathbf{K} \mathbf{1}_N + \mathbf{1}_N \mathbf{K} \mathbf{1}_N$ , where  $\mathbf{1}_N$  denotes an  $N \times N$  matrix in which each entry equals  $1/N$ . Subsequently, the eigenvalue problem  $\tilde{\mathbf{K}}\alpha = \lambda\alpha$  is solved on the centered matrix to find the eigenvalues  $\{\lambda_j\}_{j=1}^m$  and their corresponding eigenvectors  $\{\alpha^j\}_{j=1}^m$ . These eigenvectors, which represent the principal axes in  $\mathcal{H}$ , are then scaled by the square root of their

eigenvalues to form the extracted feature matrix  $\mathbf{Z}$ . While these principal components are linear axes in the high-dimensional feature space, they correspond to complex, nonlinear functions of the original input features.

Once the Kernel PCA model has been trained on a dataset with data points  $\mathbf{x}_1, \dots, \mathbf{x}_N$ , it can be used to project a new, unseen data point,  $\mathbf{x}^*$ , into the same low-dimensional



space. This process, illustrated in Figure 7, requires the original training data  $\{\mathbf{x}_i\}_{i=1}^N$  as well as the eigenvectors and eigenvalues  $\{\alpha^j, \lambda_j\}_{j=1}^m$  learned during training. First, a kernel vector is computed by applying the *centered* kernel function  $\tilde{k}(\mathbf{x}_i, \mathbf{x}^*) = k(\mathbf{x}_i, \mathbf{x}^*) - \frac{1}{N} \sum_{p=1}^N k(\mathbf{x}_p, \mathbf{x}^*) - \frac{1}{N} \sum_{q=1}^N k(\mathbf{x}_i, \mathbf{x}_q) + \frac{1}{N^2} \sum_{p,q=1}^N k(\mathbf{x}_p, \mathbf{x}_q)$  between the new point and every point in the original training set. This vector, which represents the new point's relationship to the original data, is then projected onto each of the learned eigenvectors. The result  $\mathbf{z}^*$  is the set of coordinates for the new data point in the low-dimensional feature space.

Figure 8 illustrates a simplified schematic of the algebraic representation of GFR, as described in Algorithm 8. Unlike Kernel PCA, which maps the data into a higher-dimensional space to capture nonlinearity, GFR captures nonlinear dependencies through an iterative procedure. At each iteration  $j$ , it removes the projections onto all previously extracted orthogonal functions from the data, and then identifies the direction of maximum variance as the next component. This iterative structure enables GFR to capture nonlinear relationships without relying on computational techniques such as the kernel trick used in Kernel PCA. Another key distinction between GFR and Kernel PCA lies in their outputs. GFR produces linear combinations of the original features in the input space, which makes its results directly interpretable because the contribution of each feature can be observed through the coefficients in the extracted directions. In contrast, the output of Kernel PCA consists of nonlinear functions of the original features in a high-dimensional feature space  $\mathcal{H}$ , which makes interpretation more difficult due to the implicit nature of the mapping.

Finally, another key difference between GFR and Kernel PCA arises when applying the trained model to a new data point  $\mathbf{x}^*$ . As shown in Figure 9, in the case of GFR, we simply project the new data point onto the extracted directions  $\mathbf{v}_1, \dots, \mathbf{v}_m$ . In contrast, Kernel PCA requires additional computations: we must first compute the centered kernel vector between the new point and all training points, and then project this vector onto the eigenvectors obtained during training. This distinction makes GFR significantly more efficient at inference time.

**Example 4 (GFR vs. Kernel PCA):** This example compares the output of GFR with Kernel PCA on a synthetic dataset with 10,000 data points and five features, including three mutually independent Gaussian random variables  $X_1 \sim \mathcal{N}(0, 0.9)$ ,  $X_2 \sim \mathcal{N}(0, 0.8)$ ,  $X_3 \sim \mathcal{N}(0, 0.7)$ , and the remaining two features are  $X_4 = X_1 X_3$  and  $X_5 = X_1 X_2^2$ . We extract  $m = 3$  features with each method. Both rely on polynomials of total degree at most three: GFR uses this family explicitly as  $\mathcal{F}(\mathbf{z})$ , while Kernel PCA employs the same basis implicitly via its degree three feature map.

#### a) GFR Output:

GFR returns linear combinations of the original features. Table XVII lists the coefficients of the first three extracted features, where each row corresponds to one of the original features  $X_i$ , and each column shows its contribution to the respective extracted feature.

#### b) Kernel PCA Output:

Kernel PCA uses the degree three polynomial feature map

$$\Phi(X) = [1, X_1, \dots, X_5, X_1^2, X_1 X_2, \dots, X_5^2, X_1^3, X_1^2 X_2, \dots, X_5^3]^\top \in \mathbb{R}^{56}.$$

Table XVIII summarizes the structure of the feature map  $\Phi(X)$ .

After centering  $\Phi(X)$ , Kernel PCA returns three principal directions, each a linear combination of the 56 coordinates. Rather than writing the full 56-dimensional coefficient vectors as equations, which would be excessively long, we summarize them compactly in Table XIX. Table XIX lists the coefficients: the first column gives the feature map element and the next three columns give its coefficients in the first, second, and third extracted feature, respectively. Note that centering forces the bias term (the first row) to have zero coefficient.

**Remark 5:** In practice, direct mappings into high-dimensional feature spaces are rarely used, as the resulting dimensionality is often computationally prohibitive, a phenomenon referred to as the curse of dimensionality. The kernel trick circumvents this issue by implicitly realizing such mappings without constructing high-dimensional vectors. Here, we present the explicit feature map only to emphasize the structural distinction between GFR, which operates in the original feature space, and Kernel PCA, which functions in the high-dimensional feature map.

## REFERENCES

- [1] B. Yaghoobi, N. Raviv, and B. Sinopoli, "Gram-Schmidt methods for unsupervised feature selection," in *Proc. IEEE 63rd Conf. Decis. Control (CDC)*, Dec. 2024, pp. 7700–7707.
- [2] B. Yaghoobi, N. Raviv, and B. Sinopoli, "Beyond PCA: A gram-Schmidt approach to feature extraction," in *Proc. 60th Annu. Allerton Conf. Commun., Control, Comput.*, Sep. 2024, pp. 1–8.
- [3] S. Khalid, T. Khalil, and S. Nasreen, "A survey of feature selection and feature extraction techniques in machine learning," in *Proc. Sci. Inf. Conf.*, Aug. 2014, pp. 372–378.
- [4] D. G. Luenberger, *Optimization By Vector Space Methods*. Hoboken, NJ, USA: Wiley, 1997.
- [5] M. Heidari, J. K. Sreedharan, G. Shamir, and W. Szpankowski, "Sufficiently informative and relevant features: An information-theoretic and Fourier-based characterization," *IEEE Trans. Inf. Theory*, vol. 68, no. 9, pp. 6063–6077, Sep. 2022.
- [6] M. Heidari, J. K. Sreedharan, G. I. Shamir, and W. Szpankowski, "Finding relevant information via a discrete Fourier expansion," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 4181–4191.
- [7] T. M. Cover, *Elements of Information Theory*. Hoboken, NJ, USA: Wiley, 1999.
- [8] S. G. Krantz and H. R. Parks, *The Implicit Function Theorem: History, Theory, and Applications*. Cham, Switzerland: Springer, 2002.
- [9] J. M. Lee and J. M. Lee, *Smooth Manifolds*. Cham, Switzerland: Springer, 2012.
- [10] S. S. Ge, C. C. Hang, T. H. Lee, and T. Zhang, *Stable Adaptive Neural Network Control*. Cham, Switzerland: Springer, 2013.
- [11] S. S. Ge and C. Wang, "Adaptive NN control of uncertain nonlinear pure-feedback systems," *Automatica*, vol. 38, no. 4, pp. 671–682, Apr. 2002.
- [12] I. Sandberg, "Global implicit function theorems," *IEEE Trans. Circuits Syst.*, vol. CS-28, no. 2, pp. 145–149, Feb. 1981.
- [13] K. Pearson, "LIII. On lines and planes of closest fit to systems of points in space," *London, Edinburgh, Dublin Phil. Mag. J. Sci.*, vol. 2, no. 11, pp. 559–572, Nov. 1901.
- [14] S. Theodoridis, *Machine Learning: A Bayesian and Optimization Perspective*. New York, NY, USA: Academic, 2015.
- [15] T. F. Cox and M. A. Cox, *Multidimensional Scaling*. Boca Raton, FL, USA: CRC Press, 2000.

- [16] J. B. Tenenbaum, V. D. Silva, and J. C. Langford, "A global geometric framework for nonlinear dimensionality reduction," *Science*, vol. 290, no. 5500, pp. 2319–2323, Dec. 2000.
- [17] W. S. Torgerson, "Multidimensional scaling: I. theory and method," *Psychometrika*, vol. 17, no. 4, pp. 401–419, Dec. 1952.
- [18] R. Larsen, "Decomposition using maximum autocorrelation factors," *J. Chemometrics*, vol. 16, nos. 8–10, pp. 427–435, Aug. 2002.
- [19] L. Wiskott and T. J. Sejnowski, "Slow feature analysis: Unsupervised learning of invariances," *Neural Comput.*, vol. 14, no. 4, pp. 715–770, Apr. 2002.
- [20] X. He and P. Niyogi, "Locality preserving projections," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 16, 2003, pp. 153–160.
- [21] S. T. Roweis and L. K. Saul, "Nonlinear dimensionality reduction by locally linear embedding," *Science*, vol. 290, no. 5500, pp. 2323–2326, Dec. 2000.
- [22] J. V. Stone, *Independent Component Analysis: A Tutorial Introduction*. Cambridge, MA, USA: MIT Press, 2004.
- [23] G. E. Hinton and S. T. Roweis, "Stochastic neighbor embedding," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 15, 2002, pp. 857–864.
- [24] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," 2018, *arXiv:1802.03426*.
- [25] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, Jul. 2006.
- [26] X. He, D. Cai, and P. Niyogi, "Laplacian score for feature selection," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 18, 2005, pp. 507–514.
- [27] R. Huang, W. Jiang, and G. Sun, "Manifold-based constraint Laplacian score for multi-label feature selection," *Pattern Recognit. Lett.*, vol. 112, pp. 346–352, Sep. 2018.
- [28] G. Stork, P. E. Hart, and R. O. Duda, *Pattern Classification*. Hoboken, NJ, USA: Wiley, 2006.
- [29] R. J. Urbanowicz, M. Meeker, W. La Cava, R. S. Olson, and J. H. Moore, "Relief-based feature selection: Introduction and review," *J. Biomed. Informat.*, vol. 85, pp. 189–203, Sep. 2018.
- [30] F. Nie, S. Xiang, Y. Jia, C. Zhang, and S. Yan, "Trace ratio criterion for feature selection," in *Proc. Conf. Artif. Intell. (AAAI)*, vol. 2, 2008, pp. 671–676.
- [31] D. Cai, C. Zhang, and X. He, "Unsupervised feature selection for multi-cluster data," in *Proc. 16th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2010, pp. 333–342.
- [32] Y. Wang, Z. Zhang, and Y. Lin, "Multi-cluster feature selection based on isometric mapping," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 3, pp. 570–572, Mar. 2022.
- [33] Z. Li, Y. Yang, J. Liu, X. Zhou, and H. Lu, "Unsupervised feature selection using nonnegative spectral analysis," in *Proc. AAAI Conf. Artif. Intell.*, vol. 26, no. 1, 2012, pp. 1026–1032.
- [34] Y. Yang, H. T. Shen, Z. Ma, Z. Huang, and X. Zhou, "L2, l-norm regularized discriminative feature selection for unsupervised," in *Proc. 22nd Int. Joint Conf. Artif. Intell.*, 2011, pp. 1–21.
- [35] F. Nie, Z. Wang, L. Tian, R. Wang, and X. Li, "Subspace sparse discriminative feature selection," *IEEE Trans. Cybern.*, vol. 52, no. 6, pp. 4221–4233, Jun. 2022.
- [36] H. Peng, F. Long, and C. Ding, "Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 8, pp. 1226–1238, Aug. 2005.
- [37] R. Battiti, "Using mutual information for selecting features in supervised neural net learning," *IEEE Trans. Neural Netw.*, vol. 5, no. 4, pp. 537–550, Jul. 1994.
- [38] L. Yu and H. Liu, "Feature selection for high-dimensional data: A fast correlation-based filter solution," in *Proc. Int. Conf. Mach. Learn.*, 2003, pp. 856–863.
- [39] M. Dash and H. Liu, "Handling large unsupervised data via dimensionality reduction," in *Proc. ACM SIGMOD Workshop Res. Issues Data Mining Knowl. Discovery*, 1999, pp. 532–539.
- [40] N. Vandenbroucke, L. Macaire, and J.-G. Postaire, "Unsupervised color texture feature extraction and selection for soccer image segmentation," in *Proc. Int. Conf. Image Process.*, Feb. 2000, pp. 800–803.
- [41] M. Alibeigi, S. Hashemi, and A. Hamzeh, "Unsupervised feature selection based on the distribution of features attributed to imbalanced data sets," *Int. J. Artificial Intell. Expert Syst.*, vol. 2, no. 1, pp. 14–22, 2011.
- [42] P. Mitra, C. A. Murthy, and S. K. Pal, "Unsupervised feature selection using feature similarity," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 3, pp. 301–312, Mar. 2002.
- [43] P.-Y. Zhou and K. C. C. Chan, "An unsupervised attribute clustering algorithm for unsupervised feature selection," in *Proc. IEEE Int. Conf. Data Sci. Adv. Analytics (DSAA)*, Oct. 2015, pp. 1–7.
- [44] P. Padungweang, C. Lursinsap, and K. Sunat, "Univariate filter technique for unsupervised feature selection using a new Laplacian score based local nearest neighbors," in *Proc. Asia-Pacific Conf. Inf. Process.*, vol. 2, Jul. 2009, pp. 196–200.
- [45] A. Saxena, N. R. Pal, and M. Vora, "Evolutionary methods for unsupervised feature selection using Sammon's stress function," *Fuzzy Inf. Eng.*, vol. 2, no. 3, pp. 229–247, Sep. 2010.
- [46] R. Vidal, Y. Ma, and S. Sastry, "Generalized principal component analysis (GPCA)," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 12, pp. 1945–1959, Dec. 2005.
- [47] H. Hsu, S. Salamatian, and F. P. Calmon, "Generalizing correspondence analysis for applications in machine learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9347–9362, Dec. 2022.
- [48] F. d. P. Calmon, A. Makhdoumi, M. Médard, M. Varia, M. Christiansen, and K. R. Duffy, "Principal inertia components and applications," *IEEE Trans. Inf. Theory*, vol. 63, no. 8, pp. 5011–5038, Aug. 2017.
- [49] D. Dua and C. Graff. (2017). *Uci Machine Learning Repository*. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [50] J. Li et al., "Feature selection: A data perspective," *ACM Comput. surveys (CSUR)*, vol. 50, no. 6, p. 94, 2017.
- [51] M. Heidari, J. Sreedharan, G. Shamir, and W. Szpankowski, "Information sufficiency via Fourier expansion," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2021, pp. 2774–2779.
- [52] B. Schölkopf, A. J. Smola, and K. Müller, "Kernel principal component analysis," in *Proc. 7th Int. Conf. Artif. Neural Networks-ICANN*, 1997, pp. 583–588.
- [53] R. O'Donnell, *Analysis of Boolean Functions*. Cambridge, U.K.: Cambridge Univ. Press, 2014.
- [54] M. E. H. Ismail and R. Zhang, "A review of multivariate orthogonal polynomials," *J. Egyptian Math. Soc.*, vol. 25, no. 2, pp. 91–110, Apr. 2017.
- [55] B. Schölkopf and A. J. Smola, *Learning With Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Cambridge, MA, USA: MIT Press, 2002.

**Bahram Yaghooti** (Graduate Student Member, IEEE) received the B.Sc. and M.Sc. degrees in mechanical engineering from the Sharif University of Technology, Tehran, Iran. He is currently pursuing the Ph.D. degree with the Department of Electrical and Systems Engineering, Washington University in St. Louis, St. Louis, MO, USA. His research interests include information theory, control theory, and machine learning.

**Netanel Raviv** (Senior Member, IEEE) received the B.Sc. degree in mathematics and computer science and the M.Sc. and Ph.D. degrees in computer science from Technion, Israel, in 2010, 2013, and 2017, respectively. He is currently an Assistant Professor with the Department of Computer Science and Engineering, Washington University in St. Louis, St. Louis, MO, USA. His research interests include applications of coding theory to privacy, distributed computations, and machine learning. He was an awardee of the IBM Ph.D. Fellowship, the First Prize in the Feder Family Competition for best student work in communication technology, and the Lester-Deutsche Post-Doctoral Fellowship.

**Bruno Sinopoli** (Fellow, IEEE) received the Dr.Eng. degree from the University of Padova, Padua, Italy, in 1998, and the M.S. and Ph.D. degrees in electrical engineering from the University of California at Berkeley, Berkeley, CA, USA, in 2003 and 2005, respectively. He was a Post-Doctoral Researcher with Stanford University. From 2007 to 2019, he was a member of the Faculty with Carnegie Mellon University, where he was a Professor with the Department of Electrical and Computer Engineering with courtesy appointments in mechanical engineering and with the Robotics Institute and the Co-Director of the Smart Infrastructure Institute. He is currently the Das Family Distinguished Professor with Washington University in St. Louis, St. Louis, MO, USA, where he is also the Founding Director of the Center for Trustworthy AI in Cyber-Physical Systems and the Chair of the Electrical and Systems Engineering Department. His research interests include modeling, analysis, and design of resilient cyber-physical systems with applications to smart interdependent infrastructure systems, such as energy and transportation, the Internet of Things, and control of computing systems.